

Portable Backgammon Match Clock V 2.0 Code Walkthrough

Updated firmware analysis with detailed figures

Target board: Elecrow CrowPanel ESP32 HMI 5.0-inch / DIS07050H

Source file: `BackgammonMatchClock(3).ino`

Firmware lines analyzed: 4104

Version label: V 2.0

Focus: Segment-by-segment explanation, hardware wiring, perfboard layout, UI feature map.

Output: LaTeX source, PDF, images, and full code appendix.

Contents

1	Executive Summary	2
2	What Changed in V 2.0	2
3	Detailed Figures	2
3.1	Firmware architecture	3
3.2	Hardware wiring overview	4
3.3	Perfboard physical layout	5
3.4	UI feature map from code coordinates	6
3.5	Font and color rendering model	7
4	Hardware Wiring and Pin Use	8
4.1	Important wiring rules	8
4.2	LCD RGB pins reserved by the panel	8
5	Segment-by-Segment Code Walkthrough	8
5.1	1. File header, libraries, optional fonts, and pin map	8
5.2	2. Core enums, button structures, Rects, and display wrapper	11
5.3	3. Global state, UI coordinate map, and forward declarations	20
5.4	4. Arduino startup and runtime loop	22
5.5	5. Button and touch input handling	23
5.6	6. Clock engine and timing rules	28
5.7	7. Scoring, Crawford logic, match lifecycle, and menu changes	31
5.8	8. Display renderer and main screens	35
5.9	9. Name editor, checker-color selection, timer panels, and player panels	39
5.10	10. Splash screen and generated graphic helpers	44
5.11	11. I2C devices: expanders, fuel gauge, RTC, and touch controller	46
5.12	12. Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	51
6	Top-Level Function Inventory	59
7	Command/API Appendix	67
A	Full V 2.0 Source Code	67

1 Executive Summary

This V 2.0 firmware implements a portable backgammon match clock and scoreboard for an 800 x 480 ESP32-S3 CrowPanel. The firmware combines match scoring, Crawford/post-Crawford rules, game/match timer scope, simple delay, Bronstein-style delay behavior, Fischer increment, touch UI, physical-button support through GPIO and PCF8575, persistent settings, player names, checker-color selection, match log storage, DS3231 real-time clock support, MAX17048 battery gauge support, splash graphics, and a custom RGB LCD drawing wrapper.

The main architectural pattern is: **inputs change state; state marks the display dirty; the renderer redraws the current screen from that state.** This keeps clock rules, match rules, hardware access, and drawing code mostly separated.

2 What Changed in V 2.0

Compared with the earlier versions, this source file is more visually developed. It adds or expands optional GFXfont support, compiled splash-image support, score/match-target font paths, checker-color selection during name editing, player-color persistence, and a clearer split between timer panels, score panels, and general UI text.

Important V 2.0 design ideas:

- **Fonts are optional at compile time.** The code checks for font headers and falls back safely if they are not present.
- **Score identity and active-clock emphasis are separate.** Checker colors identify each player in the score panels, while timer inversion/highlighting communicates whose clock is active.
- **The UI is coordinate-driven.** The major screens use explicit 800 x 480 rectangles, which makes it possible to generate accurate UI maps from the code.
- **Hardware is modular.** The I2C bus can support the hub, RTC, fuel gauge, touch, and button expander while the firmware continues to run if optional devices are missing.

3 Detailed Figures

3.1 Firmware architecture

Backgammon Match Clock V 2.0 - Firmware Architecture

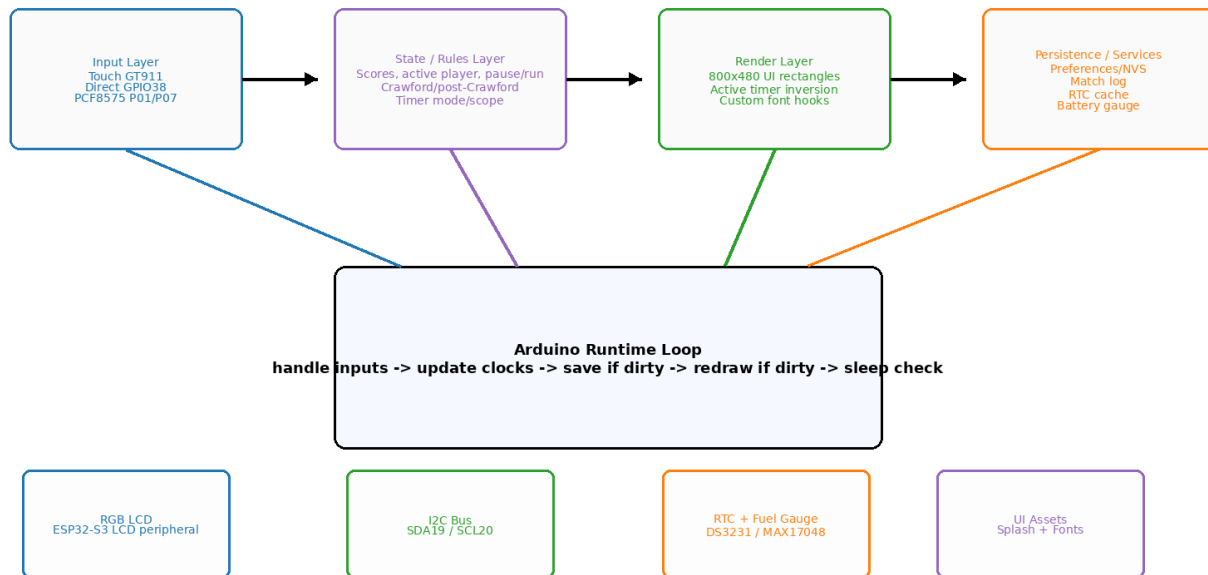


Figure 1: Firmware architecture map for V 2.0. The sketch is organized around input, rules/state, rendering, and persistence/services.

3.2 Hardware wiring overview

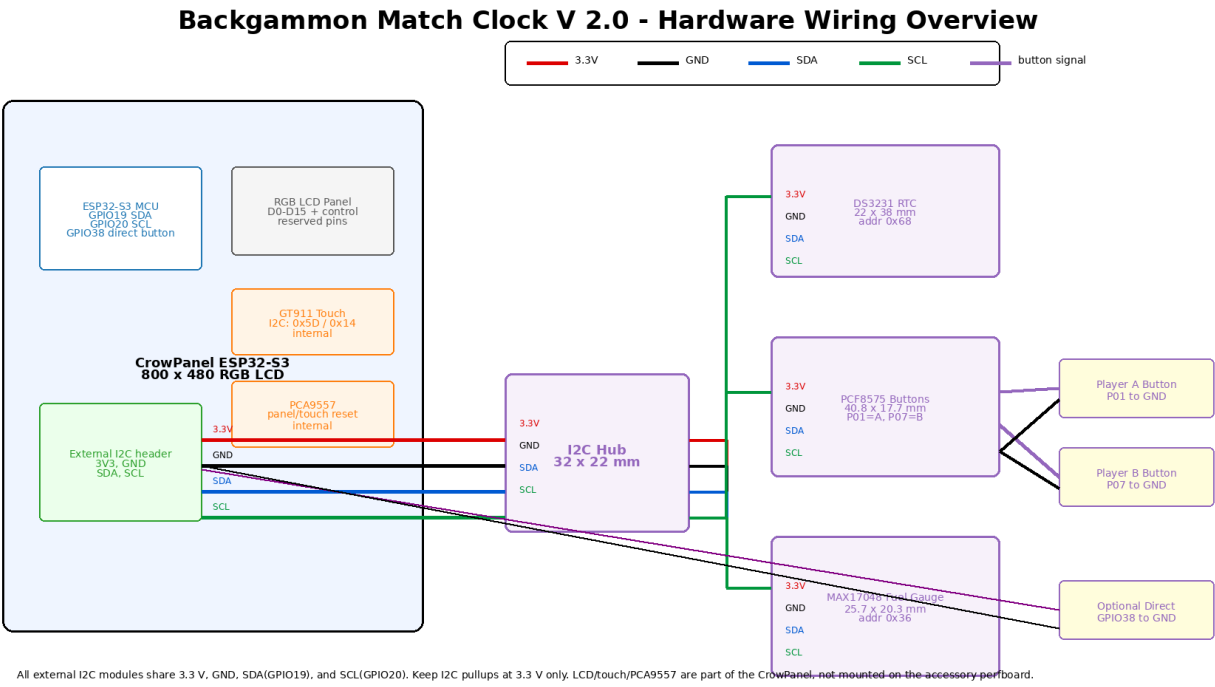


Figure 2: Hardware wiring overview showing the CrowPanel, external I2C bus, I2C hub, DS3231 RTC, PCF8575 button expander, MAX17048 fuel gauge, and button connections.

3.3 Perfboard physical layout

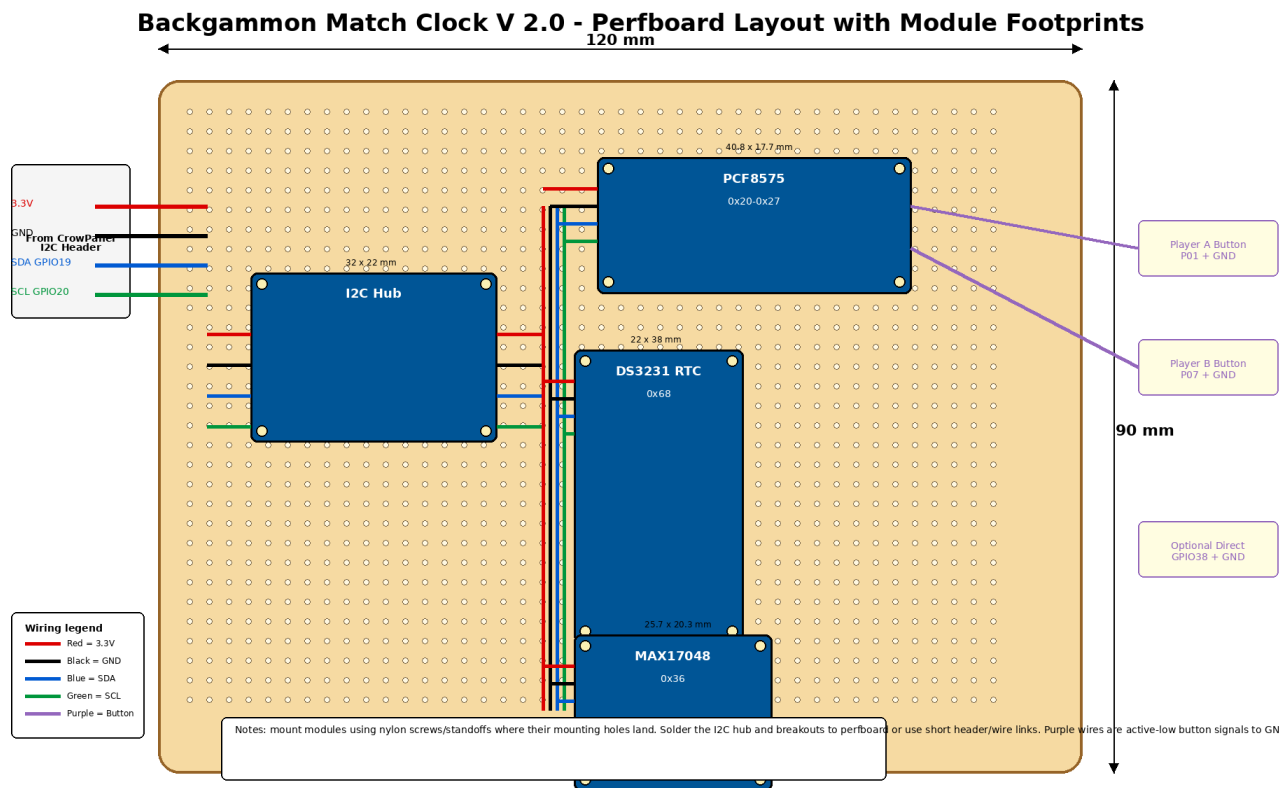
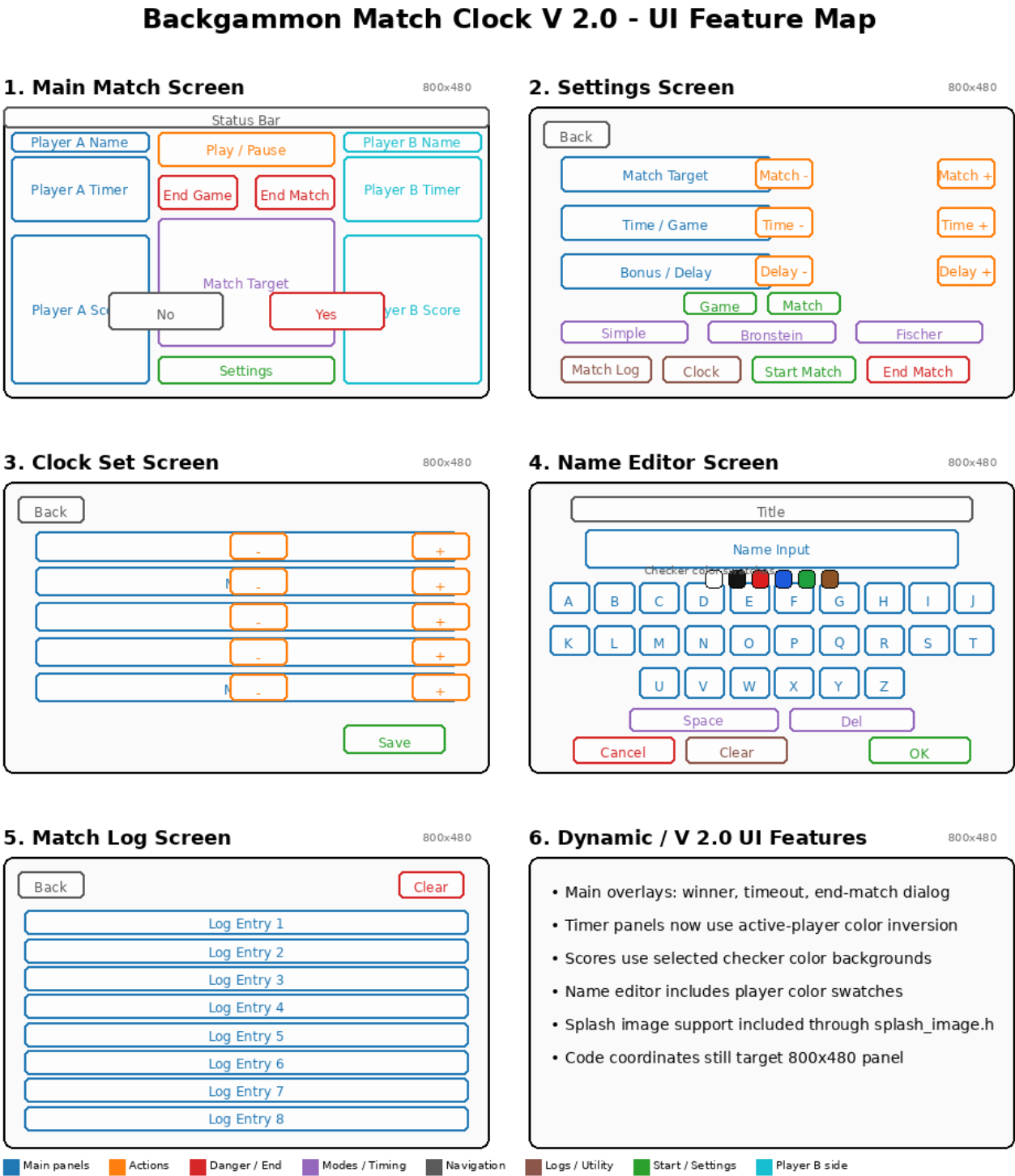


Figure 3: Perfboard layout using the component sizes provided: DS3231 22 x 38 mm, PCF8575 40.8 x 17.7 mm, MAX17048 25.7 x 20.3 mm, and I2C hub 32 x 22 mm.

3.4 UI feature map from code coordinates



3.5 Font and color rendering model

Backgammon Match Clock V 2.0 - Font and Color Rendering Model



Figure 5: Visual summary of how V 2.0 separates timer rendering, general UI text, large score rendering, and checker-color identity.

4 Hardware Wiring and Pin Use

4.1 Important wiring rules

The display is an RGB-parallel panel. Many ESP32-S3 pins are already committed to the LCD bus, so they should not be reused for external buttons or accessories. Use I2C accessories whenever possible. The firmware starts the shared I2C bus on **SDA = GPIO19** and **SCL = GPIO20**. The direct Player A button remains available on **GPIO38 to GND**; Player B is expected through the PCF8575 unless a future free GPIO is selected.

Signal / device	Firmware pin/address	Notes
Direct Player A button	GPIO38	Momentary button to GND, active-low.
Direct Player B button	disabled direct GPIO / PCF8575 P07	Use the PCF8575 expander for Player B.
I2C SDA	GPIO19	Shared by touch, PCA9557, PCF8575, MAX17048, DS3231, and hub.
I2C SCL	GPIO20	Shared I2C clock line; 3.3 V pullups only.
PCF8575 button expander	address scan 0x20-0x27	P01 = Player A, P07 = Player B.
MAX17048 fuel gauge	0x36	Battery voltage and state-of-charge monitor.
DS3231 RTC	0x68	Date/time source for match log entries.
Backlight	GPIO2	Built-in TFT backlight control; do not reuse externally.

4.2 LCD RGB pins reserved by the panel

LCD role	ESP32-S3 GPIO used by firmware
Data bus	D0=8, D1=3, D2=46, D3=9, D4=1, D5=5, D6=6, D7=7, D8=15, D9=16, D10=4, D11=45, D12=48, D13=47, D14=21, D15=14
Control/timing	DE=40, VSYNC=41, HSYNC=39, PCLK=0
Backlight	GPIO2

5 Segment-by-Segment Code Walkthrough

Each subsection below contains the actual source segment from `BackgammonMatchClock(3).ino`, followed by explanation. The code listings are not meant to replace the appendix; they give the most important surrounding context for each functional area.

5.1 1. File header, libraries, optional fonts, and pin map

What this segment does. Defines the project purpose, pulls in Arduino/ESP32/LCD/I2C headers, and declares optional font include paths. This version also includes `splash_image.h` and multiple GFXfont fallbacks, so the firmware can compile whether or not custom fonts are present.

Why this structure is used. The top of the sketch is the hardware and build-configuration contract. The 255 sentinel means not wired / disabled. Keeping font availability behind `__has_include` lets you experiment with fonts without breaking builds on machines that do not yet have those headers.

Source lines 1–235.

```
1  /*
2   Portable Backgammon Match Clock and Scoreboard
3   Target: Elecrow CrowPanel ESP32 HMI 5.0-inch display (DIS07050H)
4
5   Display library:
```

```

6   This sketch uses the ESP32-S3 RGB LCD peripheral for the built-in 800x480
7   RGB-parallel panel.
8
9   Clock model:
10  - At the start of each game, no clock is running.
11  - Pressing Player A's large button ends A's turn and starts B's clock.
12  - Pressing Player B's large button ends B's turn and starts A's clock.
13  - Optional delay/increment is supported. Delay is Bronstein/simple delay:
14    main time does not tick down until the active turn exceeds delaySeconds.
15  - Increment, if enabled, is added to the player whose turn was ended.
16
17  Crawford model:
18  - When either player reaches match_target - 1 and Crawford has not been used,
19    the following game is marked Crawford and the cube is disabled.
20  - After that game ends, all later games are post-Crawford and cube is enabled.
21  */
22
23  #include <Arduino.h>
24  #include <Wire.h>
25  #include <stdarg.h>
26  #include <Preferences.h>
27  #include <esp_sleep.h>
28  #include <esp_heap_caps.h>
29  #include <esp_lcd_panel_ops.h>
30  #include <esp_lcd_panel_rgb.h>
31  #include "splash_image.h"
32
33  #if __has_include(<gfxfont.h>)
34  #include <gfxfont.h>
35  #else
36  typedef struct {
37      uint16_t bitmapOffset;
38      uint8_t width;
39      uint8_t height;
40      uint8_t xAdvance;
41      int8_t xOffset;
42      int8_t yOffset;
43  } GFXglyph;
44
45  typedef struct {
46      uint8_t *bitmap;
47      GFXglyph *glyph;
48      uint16_t first;
49      uint16_t last;
50      uint16_t yAdvance;
51  } GFXfont;
52  #endif
53
54  #if __has_include("FreeMonoBold24pt7b.h")
55  #include "FreeMonoBold24pt7b.h"
56  #define BGMC_HAS_CLOCK_FONT 1
57  #elif __has_include(<Fonts/FreeMonoBold24pt7b.h>)
58  #include <Fonts/FreeMonoBold24pt7b.h>
59  #define BGMC_HAS_CLOCK_FONT 1
60  #else
61  #define BGMC_HAS_CLOCK_FONT 0
62  #endif
63
64  #if __has_include("FreeSans18pt7b.h")
65  #include "FreeSans18pt7b.h"
66  #define BGMC_HAS_UI_FONT 1
67  #elif __has_include(<Fonts/FreeSans18pt7b.h>)
68  #include <Fonts/FreeSans18pt7b.h>
69  #define BGMC_HAS_UI_FONT 1
70  #else
71  #define BGMC_HAS_UI_FONT 0
72  #endif
73
74  #if __has_include("FreeSans12pt7b.h")
75  #include "FreeSans12pt7b.h"
76  #define BGMC_HAS_SMALL_UI_FONT 1
77  #elif __has_include(<Fonts/FreeSans12pt7b.h>)
78  #include <Fonts/FreeSans12pt7b.h>
79  #define BGMC_HAS_SMALL_UI_FONT 1
80  #else
81  #define BGMC_HAS_SMALL_UI_FONT 0
82  #endif
83
84  #if __has_include("FreeSansBold96pt7b.h")
85  #include "FreeSansBold96pt7b.h"
86  #define BGMC_HAS_SCORE_96_FONT 1
87  #elif __has_include(<Fonts/FreeSansBold96pt7b.h>)
88  #include <Fonts/FreeSansBold96pt7b.h>
89  #define BGMC_HAS_SCORE_96_FONT 1
90  #else
91  #define BGMC_HAS_SCORE_96_FONT 0

```

```

92 #endif
93
94 #if __has_include("collegb96pt7b.h")
95 #include "collegb96pt7b.h"
96 #define BGMC_HAS_COLLEGE_SCORE_FONT 1
97 #else
98 #define BGMC_HAS_COLLEGE_SCORE_FONT 0
99 #endif
100
101 #if __has_include("FreeSansBold48pt7b.h")
102 #include "FreeSansBold48pt7b.h"
103 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
104 #elif __has_include(<Fonts/FreeSansBold48pt7b.h>)
105 #include <Fonts/FreeSansBold48pt7b.h>
106 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
107 #else
108 #define BGMC_HAS_MATCH_TARGET_48_FONT 0
109 #endif
110
111 #if __has_include("FreeSansBold24pt7b.h")
112 #include "FreeSansBold24pt7b.h"
113 #define BGMC_HAS_SCORE_FONT 1
114 #elif __has_include(<Fonts/FreeSansBold24pt7b.h>)
115 #include <Fonts/FreeSansBold24pt7b.h>
116 #define BGMC_HAS_SCORE_FONT 1
117 #else
118 #define BGMC_HAS_SCORE_FONT 0
119 #endif
120
121 // -----
122 // DIS07050H display-first pin map. External buttons can be added later on
123 // available expansion pins; avoid the LCD RGB bus pins below.
124 // -----
125 static const uint8_t PIN_PLAYER_A_BUTTON = 38; // GPIO_D IO38: one-button clock toggle to GND.
126 static const uint8_t PIN_PLAYER_B_BUTTON = 255;
127 static const uint8_t PIN_SCORE_A_PLUS = 255;
128 static const uint8_t PIN_SCORE_A_MINUS = 255;
129 static const uint8_t PIN_SCORE_B_PLUS = 255;
130 static const uint8_t PIN_SCORE_B_MINUS = 255;
131 static const uint8_t PIN_MENU_SELECT = 255;
132 static const uint8_t PIN_MENU_UP = 255;
133 static const uint8_t PIN_MENU_DOWN = 255;
134 static const uint8_t PIN_BUZZER = 255;
135 static const uint8_t PIN_TFT_BACKLIGHT = 2;
136 static const uint8_t PIN_BATTERY_ADC = 255;
137 static const uint8_t PIN_CHARGE_DETECT = 255; // Optional charger STAT pin, active-low when wired.
138
139 static const uint16_t PANEL_WIDTH = 800;
140 static const uint16_t PANEL_HEIGHT = 480;
141 static const uint8_t PIN_LCD_D0 = 8;
142 static const uint8_t PIN_LCD_D1 = 3;
143 static const uint8_t PIN_LCD_D2 = 46;
144 static const uint8_t PIN_LCD_D3 = 9;
145 static const uint8_t PIN_LCD_D4 = 1;
146 static const uint8_t PIN_LCD_D5 = 5;
147 static const uint8_t PIN_LCD_D6 = 6;
148 static const uint8_t PIN_LCD_D7 = 7;
149 static const uint8_t PIN_LCD_D8 = 15;
150 static const uint8_t PIN_LCD_D9 = 16;
151 static const uint8_t PIN_LCD_D10 = 4;
152 static const uint8_t PIN_LCD_D11 = 45;
153 static const uint8_t PIN_LCD_D12 = 48;
154 static const uint8_t PIN_LCD_D13 = 47;
155 static const uint8_t PIN_LCD_D14 = 21;
156 static const uint8_t PIN_LCD_D15 = 14;
157 static const uint8_t PIN_LCD_DE = 40;
158 static const uint8_t PIN_LCD_VSYNC = 41;
159 static const uint8_t PIN_LCD_HSYNC = 39;
160 static const uint8_t PIN_LCD_PCLK = 0;
161 static const uint8_t PIN_I2C_SDA = 19;
162 static const uint8_t PIN_I2C_SCL = 20;
163 static const uint8_t PCF8575_PLAY_PAUSE_PIN = 0; // PCF8575 P00, button to GND.
164 static const uint8_t PCF8575_PLAYER_A_PIN = 1; // PCF8575 P01, button to GND.
165 static const uint8_t PCF8575_PLAYER_B_PIN = 7; // PCF8575 P07, button to GND.
166 static const uint8_t MAX17048_I2C_ADDRESS = 0x36;
167 static const uint8_t DS3231_I2C_ADDRESS = 0x68;
168
169 // Rotary encoder placeholders. This sketch uses up/down buttons by default.
170 static const uint8_t PIN_ENCODER_A = 255;
171 static const uint8_t PIN_ENCODER_B = 255;
172
173 // Battery divider calibration placeholders.
174 static const float BATTERY_ADC_REF_VOLTS = 3.30f;
175 static const float BATTERY_DIVIDER_RATIO = 2.00f; // Vbat = Vadc * ratio.
176 static const float BATTERY_EMPTY_VOLTS = 3.30f;
177 static const float BATTERY_FULL_VOLTS = 4.20f;

```

```

178 // Display/backlight defaults.
179 static const uint8_t DEFAULT_BRIGHTNESS = 210; // 0..255 PWM duty.
180 static const bool TFT_BACKLIGHT_ACTIVE_HIGH = true;
181 static const uint16_t ILI9341_BLACK = 0x0000;
182 static const uint16_t ILI9341_WHITE = 0xFFFF;
183 static const uint16_t ILI9341_RED = 0xF800;
184 static const uint16_t ILI9341_GREEN = 0x07E0;
185 static const uint16_t ILI9341_BLUE = 0x001F;
186 static const uint16_t ILI9341_CYAN = 0x07FF;
187 static const uint16_t ILI9341_YELLOW = 0xFFE0;
188 static const uint16_t TFT_BACKGROUND = ILI9341_BLACK;
189 static const uint16_t TFT_PANEL = 0x2104;
190 static const uint16_t TFT_PANEL_ALT = 0x3186;
191 static const uint16_t TFT_BORDER = 0x7BEF;
192 static const uint16_t TFT_BUTTON_BORDER = 0xD6BA;
193 static const uint16_t TFT_TEXT = ILI9341_WHITE;
194 static const uint16_t TFT_TEXT_DARK = ILI9341_BLACK;
195 static const uint16_t TFT_ACTIVE_TEXT = 0x4208;
196 static const uint16_t TFT_MUTED = 0xBDF7;
197 static const uint16_t TFT_ACCENT_A = ILI9341_CYAN;
198 static const uint16_t TFT_ACCENT_B = 0xFD20;
199 static const uint16_t TFT_MAGENTA = 0xF81F;
200 static const uint16_t TFT_WARN = ILI9341_YELLOW;
201 static const uint16_t TFT_DANGER = ILI9341_RED;
202 static const uint16_t TFT_OK = ILI9341_GREEN;
203 static const uint16_t PLAYER_COLOR_RED = 0x7800; // #7C0A02 approximated in RGB565.
204 static const uint16_t PLAYER_COLOR_BLUE = 0x194A; // #1C2951 approximated in RGB565.
205 static const uint16_t PLAYER_COLOR_GREEN = 0x19A5; // #1C352D approximated in RGB565.
206 static const uint16_t PLAYER_COLOR_YELLOW = 0xFD20; // #FFA700 approximated in RGB565.
207 static const uint16_t PLAYER_COLOR_GRAY = 0x4208;
208 static const uint16_t PLAYER_COLOR_WHITE = ILI9341_WHITE;
209
210 // Time behavior.
211 static const uint32_t DEFAULT_INITIAL_TIME_SECONDS = 15UL * 60UL;
212 static const uint8_t DEFAULT_DELAY_SECONDS = 12;
213 static const uint8_t DEFAULT_INCREMENT_SECONDS = 0;
214 static const uint32_t LOW_TIME_WARNING_SECONDS = 30;
215 static const uint32_t CRITICAL_TIME_WARNING_SECONDS = 10;
216 static const uint32_t SLEEP_AFTER_IDLE_MS = 10UL * 60UL * 1000UL;
217 static const uint32_t LCD_PIXEL_CLOCK_HZ = 20000000;
218 static const uint16_t LCD_BOUNCE_BUFFER_LINES = 10;
219 static const uint8_t PLAYER_NAME_MAX_LEN = 10;
220
221 // Debounce intervals. Large buttons stay snappy; score/menu buttons are slower.
222 static const uint16_t PLAYER_BUTTON_DEBOUNCE_MS = 25;
223 static const uint16_t SMALL_BUTTON_DEBOUNCE_MS = 90;
224 static const uint16_t DISPLAY_REFRESH_MS = 1000;
225 static const uint16_t SETTINGS_SAVE_DELAY_MS = 1500;
226 static const char *FIRMWARE_VERSION = "lcd-refresh-2026-06-07";
227 static const uint8_t MATCH_LOG_MAX_ENTRIES = 8;
228
229 void logLine(const char *message);
230 void logPrintf(const char *format, ...);
231
232 // -----
233 // Types
234 // -----
235

```

5.2 2. Core enums, button structures, Rects, and display wrapper

What this segment does. Declares the application vocabulary and low-level drawing support: player identity, timing mode, timer scope, device mode, menu item, debounced buttons, expander buttons, hit-test rectangles, touch points, and the CrowPanel RGB display wrapper.

Why this structure is used. The display is not a normal SPI TFT; it is an RGB-parallel panel driven through ESP32-S3 LCD peripheral APIs. The wrapper lets the rest of the code call simple drawing functions while hiding the row-buffer and LCD panel details.

Source lines 236–931.

```

1 enum Player : uint8_t {
2     PLAYER_A = 0,
3     PLAYER_B = 1,
4     PLAYER_NONE = 255
5 };
6
7 enum TimingMode : uint8_t {
8     SIMPLE_DELAY,

```

```

9   BRONSTEIN_DELAY,
10  FISCHER_INCREMENT
11 };
12
13 enum TimerScope : uint8_t {
14     TIMER_BY_GAME,
15     TIMER_BY_MATCH
16 };
17
18 enum DeviceMode : uint8_t {
19     MODE_MENU,
20     MODE_MATCH,
21     MODE_MATCH_OVER,
22     MODE_TIMEOUT,
23     MODE_NAME_EDIT,
24     MODE_LOG,
25     MODE_CLOCK_SET,
26     MODE_CONFIRM_END_MATCH
27 };
28
29 enum MenuItem : uint8_t {
30     MENU_MATCH_TARGET,
31     MENU_INITIAL_TIME,
32     MENU_DELAY,
33     MENU_INCREMENT,
34     MENU_BRIGHTNESS,
35     MENU_START_MATCH,
36     MENU_RESET_MATCH,
37     MENU_COUNT
38 };
39
40 struct DebouncedButton {
41     uint8_t pin;
42     bool activeLow;
43     bool stablePressed;
44     bool lastRawPressed;
45     bool pressedEvent;
46     uint32_t lastChangeMs;
47     uint16_t debounceMs;
48
49     void begin(uint8_t buttonPin, uint16_t debounce, bool useActiveLow = true) {
50         pin = buttonPin;
51         activeLow = useActiveLow;
52         stablePressed = false;
53         lastRawPressed = false;
54         pressedEvent = false;
55         lastChangeMs = 0;
56         debounceMs = debounce;
57
58         if (pin != 255) {
59             pinMode(pin, activeLow ? INPUT_PULLUP : INPUT);
60         }
61     }
62
63     void update(uint32_t nowMs) {
64         pressedEvent = false;
65         if (pin == 255) {
66             return;
67         }
68
69         bool rawPressed = activeLow ? (digitalRead(pin) == LOW) : (digitalRead(pin) == HIGH);
70         if (rawPressed != lastRawPressed) {
71             lastRawPressed = rawPressed;
72             lastChangeMs = nowMs;
73         }
74
75         if ((nowMs - lastChangeMs) >= debounceMs && rawPressed != stablePressed) {
76             stablePressed = rawPressed;
77             if (stablePressed) {
78                 pressedEvent = true;
79             }
80         }
81     }
82
83     bool wasPressed() const {
84         return pressedEvent;
85     }
86 };
87
88 struct ExpanderButton {
89     uint8_t bit;
90     bool stablePressed;
91     bool lastRawPressed;
92     bool pressedEvent;
93     uint32_t lastChangeMs;
94     uint16_t debounceMs;

```

```

95
96 void begin(uint8_t expanderBit, uint16_t debounce) {
97     bit = expanderBit;
98     stablePressed = false;
99     lastRawPressed = false;
100     pressedEvent = false;
101     lastChangeMs = 0;
102     debounceMs = debounce;
103 }
104
105 void update(uint32_t nowMs, uint16_t portState) {
106     pressedEvent = false;
107     if (bit >= 16) {
108         return;
109     }
110
111     bool rawPressed = (portState & (1U << bit)) == 0;
112     if (rawPressed != lastRawPressed) {
113         lastRawPressed = rawPressed;
114         lastChangeMs = nowMs;
115     }
116
117     if ((nowMs - lastChangeMs) >= debounceMs && rawPressed != stablePressed) {
118         stablePressed = rawPressed;
119         if (stablePressed) {
120             pressedEvent = true;
121         }
122     }
123 }
124
125 bool wasPressed() const {
126     return pressedEvent;
127 }
128 };
129
130 struct Rect {
131     int16_t x;
132     int16_t y;
133     int16_t w;
134     int16_t h;
135
136     bool contains(int16_t px, int16_t py) const {
137         return px >= x && py >= y && px < x + w && py < y + h;
138     }
139
140     bool containsPadded(int16_t px, int16_t py, int16_t padX, int16_t padY) const {
141         return px >= x - padX && py >= y - padY && px < x + w + padX && py < y + h + padY;
142     }
143 };
144
145 struct TouchPoint {
146     bool touched;
147     uint16_t x;
148     uint16_t y;
149 };
150
151 // -----
152 // Globals
153 // -----
154 class CrowPanelRGBDisplay {
155 public:
156     CrowPanelRGBDisplay(uint8_t csPin, uint8_t dcPin, uint8_t rstPin)
157         : cs(csPin), dc(dcPin), rst(rstPin) {}
158
159     void begin() {
160         (void)cs;
161         (void)dc;
162         (void)rst;
163
164         esp_lcd_rgb_panel_config_t panelConfig = {};
165         panelConfig.clk_src = LCD_CLK_SRC_DEFAULT;
166         panelConfig.timings.pclk_hz = LCD_PIXEL_CLOCK_HZ;
167         panelConfig.timings.h_res = PANEL_WIDTH;
168         panelConfig.timings.v_res = PANEL_HEIGHT;
169         panelConfig.timings.hsync_pulse_width = 4;
170         panelConfig.timings.hsync_back_porch = 43;
171         panelConfig.timings.hsync_front_porch = 8;
172         panelConfig.timings.vsync_pulse_width = 4;
173         panelConfig.timings.vsync_back_porch = 12;
174         panelConfig.timings.vsync_front_porch = 8;
175         panelConfig.timings.flags.hsync_idle_low = 0;
176         panelConfig.timings.flags.vsync_idle_low = 0;
177         panelConfig.timings.flags.de_idle_high = 0;
178         panelConfig.timings.flags.pclk_active_neg = 1;
179         panelConfig.timings.flags.pclk_idle_high = 0;
180         panelConfig.data_width = 16;

```

```

181 panelConfig.bits_per_pixel = 16;
182 panelConfig.num_fbs = 1;
183 panelConfig.bounce_buffer_size_px = PANEL_WIDTH * LCD_BOUNCE_BUFFER_LINES;
184 panelConfig.dma_burst_size = 32;
185 panelConfig.hsycn_gpio_num = PIN_LCD_HSYNC;
186 panelConfig.vsync_gpio_num = PIN_LCD_VSYNC;
187 panelConfig.de_gpio_num = PIN_LCD_DE;
188 panelConfig.pclk_gpio_num = PIN_LCD_PCLK;
189 panelConfig.disp_gpio_num = -1;
190 panelConfig.data_gpio_nums[0] = PIN_LCD_D0;
191 panelConfig.data_gpio_nums[1] = PIN_LCD_D1;
192 panelConfig.data_gpio_nums[2] = PIN_LCD_D2;
193 panelConfig.data_gpio_nums[3] = PIN_LCD_D3;
194 panelConfig.data_gpio_nums[4] = PIN_LCD_D4;
195 panelConfig.data_gpio_nums[5] = PIN_LCD_D5;
196 panelConfig.data_gpio_nums[6] = PIN_LCD_D6;
197 panelConfig.data_gpio_nums[7] = PIN_LCD_D7;
198 panelConfig.data_gpio_nums[8] = PIN_LCD_D8;
199 panelConfig.data_gpio_nums[9] = PIN_LCD_D9;
200 panelConfig.data_gpio_nums[10] = PIN_LCD_D10;
201 panelConfig.data_gpio_nums[11] = PIN_LCD_D11;
202 panelConfig.data_gpio_nums[12] = PIN_LCD_D12;
203 panelConfig.data_gpio_nums[13] = PIN_LCD_D13;
204 panelConfig.data_gpio_nums[14] = PIN_LCD_D14;
205 panelConfig.data_gpio_nums[15] = PIN_LCD_D15;
206 panelConfig.flags.fb_in_psram = 1;
207
208 esp_err_t err = esp_lcd_new_rgb_panel(&panelConfig, &panelHandle);
209 logPrintf("esp_lcd_new_rgb_panel: %d\n", err);
210 if (err != ESP_OK) {
211     panelHandle = nullptr;
212     return;
213 }
214
215 err = esp_lcd_panel_reset(panelHandle);
216 logPrintf("esp_lcd_panel_reset: %d\n", err);
217 err = esp_lcd_panel_init(panelHandle);
218 logPrintf("esp_lcd_panel_init: %d\n", err);
219
220 lineBuffer = (uint16_t *)heap_caps_malloc(PANEL_WIDTH * sizeof(uint16_t), MALLOC_CAP_INTERNAL | MALLOC_CAP_8BIT);
221 if (lineBuffer == nullptr) {
222     lineBuffer = (uint16_t *)heap_caps_malloc(PANEL_WIDTH * sizeof(uint16_t), MALLOC_CAP_8BIT);
223 }
224 logPrintf("LCD line buffer: %s\n", lineBuffer == nullptr ? "FAILED" : "OK");
225 }
226
227 void setRotation(uint8_t rotation) {
228     (void)rotation;
229     screenWidth = PANEL_WIDTH;
230     screenHeight = PANEL_HEIGHT;
231 }
232
233 void fillScreen(uint16_t color) {
234     fillRect(0, 0, screenWidth, screenHeight, color);
235 }
236
237 void setTextSize(uint8_t size) {
238     textSize = (size == 0) ? 1 : size;
239 }
240
241 void setTextColor(uint16_t color, uint16_t background) {
242     textColor = color;
243     textBackground = background;
244 }
245
246 void setFont(const GFXfont *font = nullptr) {
247     gfxFont = font;
248 }
249
250 void setCursor(int16_t x, int16_t y) {
251     cursorX = x;
252     cursorY = y;
253 }
254
255 void print(const String &text) {
256     print(text.c_str());
257 }
258
259 void print(const char *text) {
260     if (text == nullptr) {
261         return;
262     }
263     while (*text) {
264         drawChar(*text++);
265     }
266 }

```

```

267
268 void print(int value) {
269     char buffer[16];
270     snprintf(buffer, sizeof(buffer), "%d", value);
271     print(buffer);
272 }
273
274 void print(unsigned int value) {
275     char buffer[16];
276     snprintf(buffer, sizeof(buffer), "%u", value);
277     print(buffer);
278 }
279
280 void print(unsigned long value) {
281     char buffer[20];
282     snprintf(buffer, sizeof(buffer), "%lu", value);
283     print(buffer);
284 }
285
286 void printf(const char *format, ...) {
287     char buffer[80];
288     va_list args;
289     va_start(args, format);
290     vsnprintf(buffer, sizeof(buffer), format, args);
291     va_end(args);
292     print(buffer);
293 }
294
295 int16_t width() const { return screenWidth; }
296 int16_t height() const { return screenHeight; }
297
298 void fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color) {
299     if (w <= 0 || h <= 0 || x >= screenWidth || y >= screenHeight) {
300         return;
301     }
302     if (x < 0) {
303         w += x;
304         x = 0;
305     }
306     if (y < 0) {
307         h += y;
308         y = 0;
309     }
310     if (x + w > screenWidth) {
311         w = screenWidth - x;
312     }
313     if (y + h > screenHeight) {
314         h = screenHeight - y;
315     }
316     if (w <= 0 || h <= 0) {
317         return;
318     }
319
320     if (panelHandle == nullptr || lineBuffer == nullptr) {
321         return;
322     }
323
324     for (int16_t col = 0; col < w; col++) {
325         lineBuffer[col] = color;
326     }
327
328     for (int16_t row = 0; row < h; row++) {
329         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + w, y + row + 1, lineBuffer);
330     }
331 }
332
333 void drawRGB565Image(int16_t x, int16_t y, uint16_t w, uint16_t h, const uint16_t *pixels) {
334     if (panelHandle == nullptr || lineBuffer == nullptr || pixels == nullptr) {
335         return;
336     }
337
338     uint16_t drawWidth = w;
339     uint16_t drawHeight = h;
340     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
341         return;
342     }
343     if (x + drawWidth > screenWidth) {
344         drawWidth = screenWidth - x;
345     }
346     if (y + drawHeight > screenHeight) {
347         drawHeight = screenHeight - y;
348     }
349
350     for (uint16_t row = 0; row < drawHeight; row++) {
351         const uint32_t sourceOffset = (uint32_t)row * (uint32_t)w;
352         for (uint16_t col = 0; col < drawWidth; col++) {

```

```

353     lineBuffer[col] = pgm_read_word(&pixels[sourceOffset + col]);
354 }
355 esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + drawWidth, y + row + 1, lineBuffer);
356 }
357 }
358
359 void drawRGB565ImageScaled2x(int16_t x, int16_t y, uint16_t w, uint16_t h, const uint16_t *pixels) {
360     if (panelHandle == nullptr || lineBuffer == nullptr || pixels == nullptr) {
361         return;
362     }
363
364     uint16_t drawWidth = w * 2;
365     uint16_t drawHeight = h * 2;
366     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
367         return;
368     }
369     if (x + drawWidth > screenWidth) {
370         drawWidth = screenWidth - x;
371     }
372     if (y + drawHeight > screenHeight) {
373         drawHeight = screenHeight - y;
374     }
375
376     for (uint16_t row = 0; row < drawHeight; row++) {
377         const uint32_t sourceOffset = (uint32_t)(row / 2) * (uint32_t)w;
378         for (uint16_t col = 0; col < drawWidth; col++) {
379             lineBuffer[col] = pgm_read_word(&pixels[sourceOffset + (col / 2)]);
380         }
381         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + drawWidth, y + row + 1, lineBuffer);
382     }
383 }
384
385 void drawRoundRect(int16_t x, int16_t y, int16_t w, int16_t h, int16_t radius, uint16_t color) {
386     (void)radius;
387     fillRect(x, y, w, 1, color);
388     fillRect(x, y + h - 1, w, 1, color);
389     fillRect(x, y, 1, h, color);
390     fillRect(x + w - 1, y, 1, h, color);
391 }
392
393 void getTextBounds(const String &text, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
394     if (gfxFont != nullptr) {
395         getGfxTextBounds(text, x, y, x1, y1, w, h);
396         return;
397     }
398     *x1 = 0;
399     *y1 = 0;
400     *w = text.length() * 6 * textSize;
401     *h = 8 * textSize;
402 }
403
404 private:
405 void writeCommand(uint8_t command) {
406     (void)command;
407 }
408
409 void writeData(uint8_t data) {
410     (void)data;
411 }
412
413 void writeData16(uint16_t data) {
414     (void)data;
415 }
416
417 void setAddressWindow(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1) {
418     writeCommand(0x2A);
419     writeData16(x0);
420     writeData16(x1);
421     writeCommand(0x2B);
422     writeData16(y0);
423     writeData16(y1);
424     writeCommand(0x2C);
425 }
426
427 void drawPixel(int16_t x, int16_t y, uint16_t color) {
428     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
429         return;
430     }
431     fillRect(x, y, 1, 1, color);
432 }
433
434 void getGfxTextBounds(const String &text, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
435     int16_t minX = 32767;
436     int16_t minY = 32767;
437     int16_t maxX = -32768;
438     int16_t maxY = -32768;

```

```

439 int16_t cursor = x;
440 uint16_t first = pgm_read_word(&gfxFont->first);
441 uint16_t last = pgm_read_word(&gfxFont->last);
442 const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&gfxFont->glyph);
443
444 for (uint16_t i = 0; i < text.length(); i++) {
445     char c = text.charAt(i);
446     if (c < first || c > last) {
447         continue;
448     }
449     GFXglyph glyph;
450     memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
451     if (glyph.width > 0 && glyph.height > 0) {
452         int16_t gx1 = cursor + (glyph.xOffset * textSize);
453         int16_t gy1 = y + (glyph.yOffset * textSize);
454         int16_t gx2 = gx1 + ((int16_t)glyph.width * textSize) - 1;
455         int16_t gy2 = gy1 + ((int16_t)glyph.height * textSize) - 1;
456         if (gx1 < minX) minX = gx1;
457         if (gy1 < minY) minY = gy1;
458         if (gx2 > maxX) maxX = gx2;
459         if (gy2 > maxY) maxY = gy2;
460     }
461     cursor += (int16_t)glyph.xAdvance * textSize;
462 }
463
464 if (maxX < minX || maxY < minY) {
465     *x1 = x;
466     *y1 = y;
467     *w = 0;
468     *h = 0;
469     return;
470 }
471
472 *x1 = minX;
473 *y1 = minY;
474 *w = maxX - minX + 1;
475 *h = maxY - minY + 1;
476 }
477
478 const char *glyphRow(char c, uint8_t row) const {
479     if (c >= 'a' && c <= 'z') {
480         c -= 32;
481     }
482     static const char blank[7][6] = {"00000", "00000", "00000", "00000", "00000", "00000", "00000"};
483     static const char unknown[7][6] = {"11111", "10001", "00010", "00100", "00100", "00000", "00100"};
484     #define GLYPH(ch, a,b,c,d,e,f,g) case ch: { static const char p[7][6] = {a,b,c,d,e,f,g}; return p[row]; }
485     switch (c) {
486         GLYPH(' ', "00000", "00000", "00000", "00000", "00000", "00000")
487         GLYPH('0', "01110", "10001", "10011", "10101", "11001", "10001", "01110")
488         GLYPH('1', "00100", "01100", "00100", "00100", "00100", "00100", "01110")
489         GLYPH('2', "01110", "10001", "00001", "00010", "00100", "01000", "11111")
490         GLYPH('3', "11110", "00001", "00001", "01110", "00001", "00001", "11110")
491         GLYPH('4', "00010", "00110", "01010", "10010", "11111", "00010", "00010")
492         GLYPH('5', "11111", "10000", "10000", "11110", "00001", "00001", "11110")
493         GLYPH('6', "01110", "10000", "10000", "11110", "10001", "10001", "01110")
494         GLYPH('7', "11111", "00001", "00010", "00100", "01000", "01000", "01000")
495         GLYPH('8', "01110", "10001", "10001", "01110", "10001", "10001", "01110")
496         GLYPH('9', "01110", "10001", "10001", "01111", "00001", "00001", "01110")
497         GLYPH('A', "01110", "10001", "10001", "11111", "10001", "10001", "10001")
498         GLYPH('B', "11110", "10001", "10001", "11110", "10001", "10001", "11110")
499         GLYPH('C', "01111", "10000", "10000", "10000", "10000", "10000", "01111")
500         GLYPH('D', "11110", "10001", "10001", "10001", "10001", "10001", "11110")
501         GLYPH('E', "11111", "10000", "10000", "11110", "10000", "10000", "11111")
502         GLYPH('F', "11111", "10000", "10000", "11110", "10000", "10000", "10000")
503         GLYPH('G', "01111", "10000", "10000", "10011", "10001", "10001", "01111")
504         GLYPH('H', "10001", "10001", "10001", "11111", "10001", "10001", "10001")
505         GLYPH('I', "01110", "00100", "00100", "00100", "00100", "00100", "01110")
506         GLYPH('J', "00111", "00010", "00010", "00010", "00010", "10010", "01100")
507         GLYPH('K', "10001", "10010", "10100", "11000", "10100", "10010", "10001")
508         GLYPH('L', "10000", "10000", "10000", "10000", "10000", "10000", "11111")
509         GLYPH('M', "10001", "11011", "10101", "10101", "10001", "10001", "10001")
510         GLYPH('N', "10001", "11001", "10101", "10011", "10001", "10001", "10001")
511         GLYPH('O', "01110", "10001", "10001", "10001", "10001", "10001", "01110")
512         GLYPH('P', "11110", "10001", "10001", "11110", "10000", "10000", "10000")
513         GLYPH('Q', "01110", "10001", "10001", "10001", "10101", "10010", "01101")
514         GLYPH('R', "11110", "10001", "10001", "11110", "10100", "10010", "10001")
515         GLYPH('S', "01111", "10000", "10000", "01110", "00001", "00001", "11110")
516         GLYPH('T', "11111", "00100", "00100", "00100", "00100", "00100", "00100")
517         GLYPH('U', "10001", "10001", "10001", "10001", "10001", "10001", "01110")
518         GLYPH('V', "10001", "10001", "10001", "10001", "10001", "01010", "00100")
519         GLYPH('W', "10001", "10001", "10001", "10101", "10101", "10101", "01010")
520         GLYPH('X', "10001", "10001", "01010", "00100", "01010", "10001", "10001")
521         GLYPH('Y', "10001", "10001", "01010", "00100", "00100", "00100", "00100")
522         GLYPH('Z', "11111", "00001", "00010", "00100", "01000", "10000", "11111")
523         GLYPH(':', "00000", "00100", "00100", "00000", "00100", "00100", "00000")
524         GLYPH('.', "00000", "00000", "00000", "00000", "00000", "01100", "01100")

```

```

525 GLYPH('-', "00000", "00000", "00000", "11111", "00000", "00000", "00000")
526 GLYPH('/', "00001", "00010", "00010", "00100", "01000", "01000", "10000")
527 GLYPH('+', "00000", "00100", "00100", "11111", "00100", "00100", "00000")
528 GLYPH('%', "11001", "11010", "00010", "00100", "01000", "01011", "10011")
529 default:
530     return (c == '\0') ? blank[row] : unknown[row];
531 }
532 #undef GLYPH
533 }
534
535 void drawChar(char c) {
536     if (c == '\n') {
537         cursorX = 0;
538         cursorY += gfxFont == nullptr ? (8 * textSize) : (pgm_read_word(&gfxFont->yAdvance) * textSize);
539         return;
540     }
541
542     if (gfxFont != nullptr) {
543         drawGfxChar(c);
544         return;
545     }
546
547     for (uint8_t row = 0; row < 7; row++) {
548         const char *bits = glyphRow(c, row);
549         for (uint8_t col = 0; col < 5; col++) {
550             uint16_t color = (bits[col] == '1') ? textColor : textBackground;
551             fillRect(cursorX + col * textSize, cursorY + row * textSize, textSize, textSize, color);
552         }
553     }
554     fillRect(cursorX + 5 * textSize, cursorY, textSize, 7 * textSize, textBackground);
555     cursorX += 6 * textSize;
556 }
557
558 void drawGfxChar(char c) {
559     uint16_t first = pgm_read_word(&gfxFont->first);
560     uint16_t last = pgm_read_word(&gfxFont->last);
561     if (c < first || c > last) {
562         return;
563     }
564
565     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&gfxFont->glyph);
566     const uint8_t *bitmap = (const uint8_t *)pgm_read_ptr(&gfxFont->bitmap);
567     GFXglyph glyph;
568     memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
569
570     uint16_t bitIndex = 0;
571     uint16_t bitmapOffset = glyph.bitmapOffset;
572     uint8_t bits = 0;
573     for (uint8_t yy = 0; yy < glyph.height; yy++) {
574         for (uint8_t xx = 0; xx < glyph.width; xx++) {
575             if ((bitIndex & 7) == 0) {
576                 bits = pgm_read_byte(&bitmap[bitmapOffset++]);
577             }
578             if (bits & 0x80) {
579                 fillRect(cursorX + (glyph.xOffset + xx) * textSize, cursorY + (glyph.yOffset + yy) * textSize,
580                     textSize, textSize, textColor);
581             }
582             bits <<= 1;
583             bitIndex++;
584         }
585     }
586     cursorX += (int16_t)glyph.xAdvance * textSize;
587 }
588
589 uint8_t cs;
590 uint8_t dc;
591 uint8_t rst;
592 int16_t screenWidth = PANEL_WIDTH;
593 int16_t screenHeight = PANEL_HEIGHT;
594 int16_t cursorX = 0;
595 int16_t cursorY = 0;
596 uint16_t textColor = ILI9341_WHITE;
597 uint16_t textBackground = ILI9341_BLACK;
598 uint8_t textSize = 1;
599 const GFXfont *gfxFont = nullptr;
600 esp_lcd_panel_handle_t panelHandle = nullptr;
601 uint16_t *lineBuffer = nullptr;
602 };
603
604 CrowPanelRGBDisplay tft(255, 255, 255);
605 Preferences prefs;
606
607 DebouncedButton btnPlayerA;
608 DebouncedButton btnPlayerB;
609 DebouncedButton btnScoreAPlus;
610 DebouncedButton btnScoreAMinus;

```

```

611 DebouncedButton btnScoreBPlus;
612 DebouncedButton btnScoreBMinus;
613 DebouncedButton btnSelect;
614 DebouncedButton btnUp;
615 DebouncedButton btnDown;
616 ExpanderButton expanderBtnPlayerA;
617 ExpanderButton expanderBtnPlayerB;
618 ExpanderButton expanderBtnPlayPause;
619
620 int playerA_score = 0;
621 int playerB_score = 0;
622 int match_target = 9;
623 uint32_t playerA_time_remaining = DEFAULT_INITIAL_TIME_SECONDS * 1000UL;
624 uint32_t playerB_time_remaining = DEFAULT_INITIAL_TIME_SECONDS * 1000UL;
625 uint32_t match_elapsed_seconds = 0;
626 Player active_player = PLAYER_NONE;
627 bool clock_running = false;
628 bool crawford_used = false;
629 bool crawford_active = false;
630 bool post_crawford = false;
631 bool cube_enabled = true;
632 uint16_t game_number = 0;
633
634 uint32_t initial_time_seconds = DEFAULT_INITIAL_TIME_SECONDS;
635 uint8_t delay_seconds = DEFAULT_DELAY_SECONDS;
636 uint8_t increment_seconds = DEFAULT_INCREMENT_SECONDS;
637 uint8_t brightness = DEFAULT_BRIGHTNESS;
638 TimingMode timingMode = SIMPLE_DELAY;
639 TimerScope timerScope = TIMER_BY_MATCH;
640 String playerA_name = "PLAYER A";
641 String playerB_name = "PLAYER B";
642 uint16_t playerA_color = PLAYER_COLOR_BLUE;
643 uint16_t playerB_color = PLAYER_COLOR_RED;
644 String matchLog[MATCH_LOG_MAX_ENTRIES];
645 uint8_t matchLogCount = 0;
646
647 DeviceMode mode = MODE_MENU;
648 MenuItem currentMenuItem = MENU_MATCH_TARGET;
649 Player timeoutPlayer = PLAYER_NONE;
650 Player matchWinner = PLAYER_NONE;
651 Player editingNameFor = PLAYER_NONE;
652 String editingName;
653 uint16_t editingPlayerColor = PLAYER_COLOR_BLUE;
654
655 bool pending_crawford_next_game = false;
656 bool game_in_progress = false;
657 bool settingsDirty = false;
658 bool displayDirty = true;
659 bool displayModeKnown = false;
660 DeviceMode lastDrawMode = MODE_MENU;
661 uint32_t settingsDirtyMs = 0;
662 uint32_t lastTickMs = 0;
663 uint32_t turnStartMs = 0;
664 uint32_t pausedTurnElapsedMs = 0;
665 uint32_t lastDisplayMs = 0;
666 uint32_t lastActivityMs = 0;
667 uint32_t lastLowTimeBeepMs = 0;
668 uint8_t gt911Address = 0;
669 bool touchWasDown = false;
670 uint16_t touchStartX = 0;
671 uint16_t touchStartY = 0;
672 uint16_t touchLastX = 0;
673 uint16_t touchLastY = 0;
674 uint16_t touchMinY = 0;
675 uint16_t touchMaxY = 0;
676 uint32_t touchStartMs = 0;
677 uint32_t touchLastSeenMs = 0;
678 uint32_t touchLastHandledMs = 0;
679 uint8_t pca9557Address = 0;
680 uint8_t pcf8575Address = 0;
681 bool max17048Found = false;
682 bool ds3231Found = false;
683 bool lastTouchSeen = false;
684 float lastChargeCheckVoltage = 0.0f;
685 float lastChargeCheckPercent = 0.0f;
686 bool chargeTrendRising = false;
687 uint32_t lastChargeCheckMs = 0;
688 uint32_t chargeIndicatorUntilMs = 0;
689 String cachedRtcStatus = "DATE UNSET";
690 uint32_t lastRtcStatusMs = 0;
691 uint16_t editingRtcYear = 2026;
692 uint8_t editingRtcMonth = 1;
693 uint8_t editingRtcDay = 1;
694 uint8_t editingRtcHour = 12;
695 uint8_t editingRtcMinute = 0;

```

5.3 3. Global state, UI coordinate map, and forward declarations

What this segment does. Creates match state variables, UI rectangles, player names, player checker colors, log storage, hardware status flags, and function prototypes.

Why this structure is used. The Rect constants are the code-level source of truth for the 800x480 UI. The player color indexes are a V 2.0-style identity feature: the chosen checker color can be applied to a player score panel and persisted.

Source lines 932–1107.

```

1  const int MATCH_TARGETS[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21};
2  const uint8_t MATCH_TARGET_COUNT = sizeof(MATCH_TARGETS) / sizeof(MATCH_TARGETS[0]);
3
4  const Rect REGION_STATUS = {0, 0, 800, 34};
5  const Rect REGION_HEADER_A = {12, 42, 228, 32};
6  const Rect REGION_TIMER_A = {12, 82, 228, 108};
7  const Rect REGION_SCORE_A = {12, 210, 228, 246};
8  const Rect REGION_PLAY_PAUSE = {255, 42, 290, 56};
9  const Rect REGION_END_GAME = {255, 113, 130, 56};
10 const Rect REGION_END_MATCH = {415, 113, 130, 56};
11 const Rect REGION_MATCH_TARGET = {255, 184, 290, 211};
12 const Rect REGION_SETTINGS = {255, 410, 290, 46};
13 const Rect REGION_HEADER_B = {560, 42, 228, 32};
14 const Rect REGION_TIMER_B = {560, 82, 228, 108};
15 const Rect REGION_SCORE_B = {560, 210, 228, 246};
16 const Rect HIT_PLAY_PAUSE = {245, 36, 310, 70};
17 const Rect HIT_END_GAME = {245, 106, 150, 70};
18 const Rect HIT_END_MATCH = {405, 106, 150, 70};
19 const Rect CONFIRM_NO = {172, 306, 190, 62};
20 const Rect CONFIRM_YES = {438, 306, 190, 62};
21
22 const Rect SETTINGS_MATCH_MINUS = {372, 86, 96, 48};
23 const Rect SETTINGS_MATCH_PLUS = {672, 86, 96, 48};
24 const Rect SETTINGS_TIME_MINUS = {372, 166, 96, 48};
25 const Rect SETTINGS_TIME_PLUS = {672, 166, 96, 48};
26 const Rect SETTINGS_DELAY_MINUS = {372, 246, 96, 48};
27 const Rect SETTINGS_DELAY_PLUS = {672, 246, 96, 48};
28 const Rect SETTINGS_MATCH_ROW_MINUS = {52, 82, 348, 58};
29 const Rect SETTINGS_MATCH_ROW_PLUS = {400, 82, 348, 58};
30 const Rect SETTINGS_TIME_ROW_MINUS = {52, 162, 348, 58};
31 const Rect SETTINGS_TIME_ROW_PLUS = {400, 162, 348, 58};
32 const Rect SETTINGS_DELAY_ROW_MINUS = {52, 242, 348, 58};
33 const Rect SETTINGS_DELAY_ROW_PLUS = {400, 242, 348, 58};
34 const Rect SETTINGS_SCOPE_GAME = {255, 306, 120, 36};
35 const Rect SETTINGS_SCOPE_MATCH = {392, 306, 120, 36};
36 const Rect SETTINGS_MODE_SIMPLE = {52, 352, 210, 38};
37 const Rect SETTINGS_MODE_BRONSTEIN = {295, 352, 210, 38};
38 const Rect SETTINGS_MODE_FISCHER = {538, 352, 210, 38};
39 const Rect SETTINGS_LOG = {52, 410, 150, 44};
40 const Rect SETTINGS_CLOCK = {220, 410, 130, 44};
41 const Rect SETTINGS_START = {368, 410, 170, 44};
42 const Rect SETTINGS_END = {556, 410, 170, 44};
43 const Rect SETTINGS_BACK = {24, 24, 108, 44};
44 const Rect LOG_BACK = {24, 24, 108, 44};
45 const Rect LOG_CLEAR = {650, 24, 108, 44};
46 const Rect CLOCK_BACK = {24, 24, 108, 44};
47 const Rect CLOCK_SAVE = {560, 400, 168, 48};
48 const Rect CLOCK_YEAR_MINUS = {372, 84, 96, 44};
49 const Rect CLOCK_YEAR_PLUS = {672, 84, 96, 44};
50 const Rect CLOCK_MONTH_MINUS = {372, 142, 96, 44};
51 const Rect CLOCK_MONTH_PLUS = {672, 142, 96, 44};
52 const Rect CLOCK_DAY_MINUS = {372, 200, 96, 44};
53 const Rect CLOCK_DAY_PLUS = {672, 200, 96, 44};
54 const Rect CLOCK_HOUR_MINUS = {372, 258, 96, 44};
55 const Rect CLOCK_HOUR_PLUS = {672, 258, 96, 44};
56 const Rect CLOCK_MINUTE_MINUS = {372, 316, 96, 44};
57 const Rect CLOCK_MINUTE_PLUS = {672, 316, 96, 44};
58
59 // -----
60 // Forward declarations requested by the firmware structure.
61 // -----
62 void drawDisplay();
63 void drawTimerPanels();
64 void drawTimerPanel(Player player, const Rect &region);
65 void drawSplashImage();
66 void fillTriangleRows(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color);
67 void fillCircleRows(int16_t cx, int16_t cy, int16_t radius, uint16_t color);
68 void drawCenteredInRect(const String &text, const Rect &region, uint8_t textSize, uint16_t color, uint16_t background);
69 void drawScoreInRect(int score, const Rect &region, uint16_t color, uint16_t background);
70 void drawMatchTargetInRect(int target, const Rect &region, uint16_t color, uint16_t background);
71 void drawGfxTextScaledInRect(const GFXfont *font, const String &text, const Rect &region, uint16_t color, uint8_t scaleNum, uint8_t
    ↪ scaleDen);

```

```

72 void getGfxTextBoundsScaled(const GFXfont *font, const String &text, uint8_t scaleNum, uint8_t scaleDen, int16_t *x1, int16_t *y1,
    ↪ uint16_t *w, uint16_t *h);
73 void drawGfxTextScaled(const GFXfont *font, const String &text, int16_t x, int16_t y, uint16_t color, uint8_t scaleNum, uint8_t scaleDen)
    ↪ ;
74 void drawMatchWinnerBanner();
75 void drawTimeoutBanner();
76 void drawNameEditor();
77 void drawMatchLog();
78 void drawClockSettings();
79 void drawEndMatchConfirm();
80 void openClockSettings();
81 void handleClockSettingsTap(uint16_t x, uint16_t y);
82 void handleEndMatchConfirmTap(uint16_t x, uint16_t y);
83 void adjustRtcField(uint8_t field, int delta);
84 void openNameEditor(Player player);
85 void handleNameEditorTap(uint16_t x, uint16_t y);
86 uint16_t colorForPlayer(Player player);
87 uint16_t textColorForPlayerColor(uint16_t color);
88 uint16_t playerColorOption(uint8_t index);
89 uint8_t playerColorOptionCount();
90 void savePlayerNames();
91 void recordMatchLog();
92 void clearMatchLog();
93 String matchLogDate();
94 String matchLogPart(const String &entry, uint8_t part);
95 void handleButtons();
96 void handleTouch();
97 void handleTouchTap(uint16_t x, uint16_t y);
98 void handleTouchSwipe(uint16_t startX, uint16_t startY, uint16_t endX, uint16_t endY);
99 void updateClock();
100 void adjustScore(Player player, int delta);
101 void checkMatchState();
102 void startNewGame();
103 void endCurrentGame();
104 void markCrawfordUsed();
105 void forceEndMatch();
106 void resetMatch();
107 void enterMenu();
108 void handleMenu();
109
110 // Helper declarations.
111 void loadSettings();
112 void saveSettingsIfNeeded();
113 void markSettingsDirty();
114 void markDisplayDirty();
115 void setBrightness(uint8_t duty);
116 void wakeActivity();
117 void maybeSleep();
118 void beep(uint16_t frequency, uint16_t durationMs);
119 void playButtonBeep();
120 void playTimeoutBeep();
121 void playMatchOverBeep();
122 void pauseClock();
123 void resumeClock();
124 void togglePlayPause();
125 void switchClockTo(Player nextPlayer);
126 void addIncrementTo(Player player);
127 bool applyTurnElapsed(Player player, uint32_t nowMs);
128 uint32_t visibleReserveMs(Player player, uint32_t nowMs);
129 uint32_t *reserveFor(Player player);
130 uint32_t matchClockSeconds();
131 uint32_t startingClockMs();
132 String formatTimeMs(uint32_t milliseconds);
133 String timingModeLabel();
134 String playPauseLabel();
135 void drawCentered(const String &text, int16_t y, uint8_t textSize, uint16_t color);
136 void drawPlayerPanel(Player player, int16_t x, int16_t y, int16_t w, int16_t h);
137 void drawStatusBar();
138 void drawChargeBolt(int16_t x, int16_t y, uint16_t color, uint16_t background);
139 void drawStartupTest();
140 void initCrowPanelExpander();
141 void initButtonExpander();
142 void initFuelGauge();
143 void initRtcClock();
144 bool pca9557Write(uint8_t address, uint8_t reg, uint8_t value);
145 uint8_t pca9557Read(uint8_t address, uint8_t reg);
146 bool pcf8575Write(uint8_t address, uint16_t value);
147 uint16_t pcf8575Read(uint8_t address);
148 bool max17048Read16(uint8_t reg, uint16_t *value);
149 float max17048CellVoltage();
150 float max17048StateOfCharge();
151 bool ds3231ReadTime(uint8_t *second, uint8_t *minute, uint8_t *hour, uint8_t *day, uint8_t *month, uint16_t *year);
152 bool ds3231SetTime(uint8_t second, uint8_t minute, uint8_t hour, uint8_t day, uint8_t month, uint16_t year);
153 bool rtcDateLooksSet(uint8_t day, uint8_t month, uint16_t year);
154 uint8_t daysInMonth(uint8_t month, uint16_t year);
155 uint8_t bcdToDec(uint8_t value);

```

```

156 uint8_t decToBcd(uint8_t value);
157 String rtcStatusText();
158 bool updateRtcStatusCache(bool force);
159 void scanI2C(const char *label);
160 void resetTouchController(uint8_t outputValue, uint8_t configValue);
161 void initTouch();
162 TouchPoint readTouch();
163 bool gt911Read(uint16_t reg, uint8_t *data, uint8_t len);
164 bool gt911Write(uint16_t reg, uint8_t value);
165 String formatTime(uint32_t seconds);
166 String matchStateLabel();
167 int batteryPercent();
168 bool chargingIndicatorOn();
169 void changeMenuValue(int delta);
170 void nextMenuItem();
171 void previousMenuItem();
172 uint8_t findCurrentTargetIndex();
173
174 // -----
175 // Arduino entry points
176 // -----

```

5.4 4. Arduino startup and runtime loop

What this segment does. setup initializes serial output, display, splash, preferences, I2C devices, RTC/fuel gauge/touch, buttons, and the starting screen. loop repeatedly reads inputs, updates clocks, saves pending settings, redraws dirty UI, and checks sleep.

Why this structure is used. The core loop avoids constantly redrawing the display. Instead, input and timer changes mark the display dirty, and rendering happens only when needed. This is important for responsiveness and battery operation.

Source lines 1108–1207.

```

1 void setup() {
2   Serial.begin(115200);
3   Serial0.begin(115200);
4   delay(200);
5   logLine("");
6   logLine("Backgammon Match Clock booting");
7   logPrintf("Firmware: %s\n", FIRMWARE_VERSION);
8   logPrintf("PSRAM found: %s\n", psramFound() ? "yes" : "no");
9   logPrintf("Free PSRAM: %u bytes\n", ESP.getFreePsram());
10  logPrintf("Total PSRAM: %u bytes\n", ESP.getPsramSize());
11  if (!psramFound() || ESP.getPsramSize() < 800000) {
12    logLine("ERROR: PSRAM is not enabled. Set Tools > PSRAM to OPI PSRAM and upload again.");
13  }
14  logLine("Buttons are optional for startup; the menu should draw with no buttons connected.");
15  logPrintf("DIS07050H RGB panel: %ux%u, BL=%u, PCLK=%u HSYNC=%u VSYNC=%u DE=%u\n",
16    PANEL_WIDTH, PANEL_HEIGHT, PIN_TFT_BACKLIGHT, PIN_LCD_PCLK,
17    PIN_LCD_HSYNC, PIN_LCD_VSYNC, PIN_LCD_DE);
18
19  btnPlayerA.begin(PIN_PLAYER_A_BUTTON, PLAYER_BUTTON_DEBOUNCE_MS);
20  btnPlayerB.begin(PIN_PLAYER_B_BUTTON, PLAYER_BUTTON_DEBOUNCE_MS);
21  btnScoreAPlus.begin(PIN_SCORE_A_PLUS, SMALL_BUTTON_DEBOUNCE_MS);
22  btnScoreAMinus.begin(PIN_SCORE_A_MINUS, SMALL_BUTTON_DEBOUNCE_MS);
23  btnScoreBPlus.begin(PIN_SCORE_B_PLUS, SMALL_BUTTON_DEBOUNCE_MS);
24  btnScoreBMinus.begin(PIN_SCORE_B_MINUS, SMALL_BUTTON_DEBOUNCE_MS);
25  btnSelect.begin(PIN_MENU_SELECT, SMALL_BUTTON_DEBOUNCE_MS);
26  btnUp.begin(PIN_MENU_UP, SMALL_BUTTON_DEBOUNCE_MS);
27  btnDown.begin(PIN_MENU_DOWN, SMALL_BUTTON_DEBOUNCE_MS);
28  expanderBtnPlayerA.begin(PCF8575_PLAYER_A_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
29  expanderBtnPlayerB.begin(PCF8575_PLAYER_B_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
30  expanderBtnPlayPause.begin(PCF8575_PLAY_PAUSE_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
31
32  if (PIN_BUZZER != 255) {
33    pinMode(PIN_BUZZER, OUTPUT);
34    digitalWrite(PIN_BUZZER, LOW);
35  }
36  if (PIN_CHARGE_DETECT != 255) {
37    pinMode(PIN_CHARGE_DETECT, INPUT_PULLUP);
38  }
39
40  pinMode(PIN_TFT_BACKLIGHT, OUTPUT);
41  setBrightness(255);
42
43  analogReadResolution(12);
44
45  initCrowPanelExpander();

```

```

46  initButtonExpander();
47  initFuelGauge();
48  initRtcClock();
49  initTouch();
50  tft.begin();
51  tft.setRotation(1);
52  drawStartupTest();
53
54  loadSettings();
55  setBrightness(brightness);
56  resetMatch();
57  mode = MODE_MATCH;
58  markDisplayDirty();
59
60  lastTickMs = millis();
61  lastActivityMs = lastTickMs;
62  updateRtcStatusCache(true);
63  drawDisplay();
64  displayDirty = false;
65  lastDisplayMs = lastTickMs;
66 }
67
68 void loop() {
69   handleTouch();
70   handleButtons();
71
72   if (mode == MODE_MATCH) {
73     updateClock();
74   }
75
76   saveSettingsIfNeeded();
77   maybeSleep();
78
79   uint32_t nowMs = millis();
80   if (!displayDirty && nowMs - lastRtcStatusMs >= 1000UL) {
81     bool rtcChanged = updateRtcStatusCache(false);
82     if (rtcChanged && (mode == MODE_MATCH || mode == MODE_MATCH_OVER || mode == MODE_TIMEOUT)) {
83       drawStatusBar();
84     }
85   }
86
87   bool timeDisplayDue = (clock_running && nowMs - lastDisplayMs >= DISPLAY_REFRESH_MS);
88   if (displayDirty) {
89     drawDisplay();
90     lastDisplayMs = nowMs;
91     displayDirty = false;
92   } else if (timeDisplayDue) {
93     drawTimerPanels();
94     lastDisplayMs = nowMs;
95   }
96 }
97
98 // -----
99 // Button handling
100 // -----

```

5.5 5. Button and touch input handling

What this segment does. Reads direct/expander buttons and GT911 touch input, then translates raw user actions into player switching, score changes, menu edits, clock edits, name edits, and swipe navigation.

Why this structure is used. Separating hardware input from match rules makes the code easier to maintain. Touch rectangles are based on the same coordinate constants used for drawing, so the visual layout and tap targets stay aligned.

Source lines 1208–1614.

```

1  void handleButtons() {
2    uint32_t nowMs = millis();
3
4    btnPlayerA.update(nowMs);
5    btnPlayerB.update(nowMs);
6    btnScoreAPlus.update(nowMs);
7    btnScoreAMinus.update(nowMs);
8    btnScoreBPlus.update(nowMs);
9    btnScoreBMinus.update(nowMs);
10   btnSelect.update(nowMs);
11   btnUp.update(nowMs);
12   btnDown.update(nowMs);
13

```

```

14 uint16_t expanderState = (pcf8575Address == 0) ? 0xFFFF : pcf8575Read(pcf8575Address);
15 expanderBtnPlayerA.update(nowMs, expanderState);
16 expanderBtnPlayerB.update(nowMs, expanderState);
17 expanderBtnPlayPause.update(nowMs, expanderState);
18
19 bool gpioPlayerAPressed = btnPlayerA.wasPressed();
20 bool gpioPlayerBPressed = btnPlayerB.wasPressed();
21 bool expanderPlayerAPressed = expanderBtnPlayerA.wasPressed();
22 bool expanderPlayerBPressed = expanderBtnPlayerB.wasPressed();
23 bool expanderPlayPausePressed = expanderBtnPlayPause.wasPressed();
24
25 bool anyPress = gpioPlayerAPressed || gpioPlayerBPressed ||
26                 expanderPlayerAPressed || expanderPlayerBPressed ||
27                 expanderPlayPausePressed ||
28                 btnScoreAPlus.wasPressed() || btnScoreAMinus.wasPressed() ||
29                 btnScoreBPlus.wasPressed() || btnScoreBMinus.wasPressed() ||
30                 btnSelect.wasPressed() || btnUp.wasPressed() || btnDown.wasPressed();
31
32 if (anyPress) {
33     wakeActivity();
34     playButtonBeep();
35 }
36
37 if (mode == MODE_MENU) {
38     handleMenu();
39     return;
40 }
41
42 if (mode == MODE_NAME_EDIT) {
43     if (btnSelect.wasPressed()) {
44         mode = MODE_MATCH;
45         editingNameFor = PLAYER_NONE;
46         markDisplayDirty();
47     }
48     return;
49 }
50
51 if (mode == MODE_LOG) {
52     if (btnSelect.wasPressed()) {
53         mode = MODE_MENU;
54         markDisplayDirty();
55     }
56     return;
57 }
58
59 if (mode == MODE_CONFIRM_END_MATCH) {
60     if (btnSelect.wasPressed()) {
61         mode = MODE_MATCH;
62         markDisplayDirty();
63     }
64     return;
65 }
66
67 // The menu/select button pauses or resumes menu access during a match.
68 if (btnSelect.wasPressed()) {
69     enterMenu();
70     return;
71 }
72
73 if (mode == MODE_MATCH) {
74     if (expanderPlayPausePressed) {
75         togglePlayPause();
76         return;
77     }
78
79     if (gpioPlayerAPressed) {
80         if (clock_running && active_player == PLAYER_A) {
81             switchClockTo(PLAYER_B);
82         } else {
83             switchClockTo(PLAYER_A);
84         }
85     }
86     if (gpioPlayerBPressed) {
87         switchClockTo(PLAYER_A);
88     }
89     if (expanderPlayerAPressed) {
90         switchClockTo(PLAYER_B);
91     }
92     if (expanderPlayerBPressed) {
93         switchClockTo(PLAYER_A);
94     }
95
96     if (btnScoreAPlus.wasPressed()) {
97         adjustScore(PLAYER_A, +1);
98     }
99     if (btnScoreAMinus.wasPressed()) {

```

```

100     adjustScore(PPLAYER_A, -1);
101 }
102 if (btnScoreBPlus.wasPressed()) {
103     adjustScore(PPLAYER_B, +1);
104 }
105 if (btnScoreBMinus.wasPressed()) {
106     adjustScore(PPLAYER_B, -1);
107 }
108 }
109
110 // After timeout or match over, select returns to menu so the device can reset.
111 if ((mode == MODE_TIMEOUT || mode == MODE_MATCH_OVER) && btnSelect.wasPressed()) {
112     enterMenu();
113 }
114 }
115
116 void handleTouch() {
117     const uint32_t TOUCH_RELEASE_GRACE_MS = 140;
118     const uint32_t TOUCH_REPEAT_GUARD_MS = 220;
119     const int16_t TOUCH_SWIPE_THRESHOLD = 32;
120
121     TouchPoint point = readTouch();
122     uint32_t nowMs = millis();
123
124     if (point.touched) {
125         touchLastX = point.x;
126         touchLastY = point.y;
127         if (point.y < touchMinY) {
128             touchMinY = point.y;
129         }
130         if (point.y > touchMaxY) {
131             touchMaxY = point.y;
132         }
133         touchLastSeenMs = nowMs;
134         lastTouchSeen = true;
135         if (!touchWasDown) {
136             touchWasDown = true;
137             touchStartX = point.x;
138             touchStartY = point.y;
139             touchMinY = point.y;
140             touchMaxY = point.y;
141             touchStartMs = nowMs;
142         }
143         return;
144     }
145
146     if (!touchWasDown) {
147         return;
148     }
149
150     // The GT911 can briefly report no touch while a finger is still moving.
151     // Waiting a moment here keeps swipes from being split into tiny taps.
152     if (nowMs - touchLastSeenMs < TOUCH_RELEASE_GRACE_MS) {
153         return;
154     }
155
156     touchWasDown = false;
157     wakeActivity();
158
159     if (nowMs - touchLastHandledMs < TOUCH_REPEAT_GUARD_MS) {
160         return;
161     }
162     touchLastHandledMs = nowMs;
163
164     if (mode == MODE_MENU || mode == MODE_LOG || mode == MODE_NAME_EDIT || mode == MODE_CLOCK_SET) {
165         handleTouchTap(touchLastX, touchLastY);
166         return;
167     }
168
169     int16_t dx = (int16_t)touchLastX - (int16_t)touchStartX;
170     int16_t upDistance = (int16_t)touchStartY - (int16_t)touchMinY;
171     int16_t downDistance = (int16_t)touchMaxY - (int16_t)touchStartY;
172     int16_t dy = (downDistance > upDistance) ? downDistance : -upDistance;
173     if (abs(dy) >= TOUCH_SWIPE_THRESHOLD && abs(dy) > abs(dx)) {
174         uint16_t gestureEndY = touchStartY;
175         if (dy > 0) {
176             uint32_t candidate = (uint32_t)touchStartY + (uint32_t)abs(dy);
177             gestureEndY = (candidate >= PANEL_HEIGHT) ? PANEL_HEIGHT - 1 : (uint16_t)candidate;
178         } else {
179             gestureEndY = (touchStartY > abs(dy)) ? touchStartY - abs(dy) : 0;
180         }
181         handleTouchSwipe(touchStartX, touchStartY, touchLastX, gestureEndY);
182     } else {
183         handleTouchTap(touchLastX, touchLastY);
184     }
185 }

```

```

186
187 void handleTouchTap(uint16_t x, uint16_t y) {
188     if (mode == MODE_NAME_EDIT) {
189         handleNameEditorTap(x, y);
190         return;
191     }
192
193     if (mode == MODE_CLOCK_SET) {
194         handleClockSettingsTap(x, y);
195         return;
196     }
197
198     if (mode == MODE_CONFIRM_END_MATCH) {
199         handleEndMatchConfirmTap(x, y);
200         return;
201     }
202
203     if (mode == MODE_LOG) {
204         if (LOG_BACK.containsPadded(x, y, 18, 14)) {
205             mode = MODE_MENU;
206             markDisplayDirty();
207             return;
208         }
209         if (LOG_CLEAR.containsPadded(x, y, 18, 14)) {
210             clearMatchLog();
211             markDisplayDirty();
212             return;
213         }
214         return;
215     }
216
217     if (mode == MODE_MENU) {
218         if (SETTINGS_BACK.containsPadded(x, y, 18, 14)) {
219             mode = MODE_MATCH;
220             markDisplayDirty();
221             return;
222         }
223
224         if (SETTINGS_MATCH_MINUS.containsPadded(x, y, 40, 18) ||
225             SETTINGS_MATCH_ROW_MINUS.contains(x, y)) {
226             currentMenuItem = MENU_MATCH_TARGET;
227             changeMenuValue(-1);
228             return;
229         }
230         if (SETTINGS_MATCH_PLUS.containsPadded(x, y, 40, 18) ||
231             SETTINGS_MATCH_ROW_PLUS.contains(x, y)) {
232             currentMenuItem = MENU_MATCH_TARGET;
233             changeMenuValue(+1);
234             return;
235         }
236         if (SETTINGS_TIME_MINUS.containsPadded(x, y, 40, 18) ||
237             SETTINGS_TIME_ROW_MINUS.contains(x, y)) {
238             currentMenuItem = MENU_INITIAL_TIME;
239             changeMenuValue(-1);
240             return;
241         }
242         if (SETTINGS_TIME_PLUS.containsPadded(x, y, 40, 18) ||
243             SETTINGS_TIME_ROW_PLUS.contains(x, y)) {
244             currentMenuItem = MENU_INITIAL_TIME;
245             changeMenuValue(+1);
246             return;
247         }
248         if (SETTINGS_DELAY_MINUS.containsPadded(x, y, 40, 18) ||
249             SETTINGS_DELAY_ROW_MINUS.contains(x, y)) {
250             currentMenuItem = MENU_DELAY;
251             changeMenuValue(-1);
252             return;
253         }
254         if (SETTINGS_DELAY_PLUS.containsPadded(x, y, 40, 18) ||
255             SETTINGS_DELAY_ROW_PLUS.contains(x, y)) {
256             currentMenuItem = MENU_DELAY;
257             changeMenuValue(+1);
258             return;
259         }
260         if (SETTINGS_SCOPE_GAME.containsPadded(x, y, 18, 10)) {
261             timerScope = TIMER_BY_GAME;
262             playerA_time_remaining = startingClockMs();
263             playerB_time_remaining = startingClockMs();
264             markSettingsDirty();
265             markDisplayDirty();
266             return;
267         }
268         if (SETTINGS_SCOPE_MATCH.containsPadded(x, y, 18, 10)) {
269             timerScope = TIMER_BY_MATCH;
270             playerA_time_remaining = startingClockMs();
271             playerB_time_remaining = startingClockMs();

```

```

272     markSettingsDirty();
273     markDisplayDirty();
274     return;
275 }
276 if (SETTINGS_MODE_SIMPLE.containsPadded(x, y, 12, 10)) {
277     timingMode = SIMPLE_DELAY;
278     markSettingsDirty();
279     markDisplayDirty();
280     return;
281 }
282 if (SETTINGS_MODE_BRONSTEIN.containsPadded(x, y, 12, 10)) {
283     timingMode = BRONSTEIN_DELAY;
284     markSettingsDirty();
285     markDisplayDirty();
286     return;
287 }
288 if (SETTINGS_MODE_FISCHER.containsPadded(x, y, 12, 10)) {
289     timingMode = FISCHER_INCREMENT;
290     markSettingsDirty();
291     markDisplayDirty();
292     return;
293 }
294 if (SETTINGS_LOG.containsPadded(x, y, 18, 14)) {
295     mode = MODE_LOG;
296     markDisplayDirty();
297     return;
298 }
299 if (SETTINGS_CLOCK.containsPadded(x, y, 18, 14)) {
300     openClockSettings();
301     return;
302 }
303 if (SETTINGS_START.containsPadded(x, y, 18, 14)) {
304     resetMatch();
305     startNewGame();
306     mode = MODE_MATCH;
307     markDisplayDirty();
308     return;
309 }
310 if (SETTINGS_END.containsPadded(x, y, 18, 14)) {
311     resetMatch();
312     mode = MODE_MATCH;
313     markDisplayDirty();
314     return;
315 }
316 return;
317 }
318
319 bool timerAHit = REGION_TIMER_A.containsPadded(x, y, 12, 14) ||
320     REGION_HEADER_A.containsPadded(x, y, 12, 10);
321 bool timerBHit = REGION_TIMER_B.containsPadded(x, y, 12, 14) ||
322     REGION_HEADER_B.containsPadded(x, y, 12, 10);
323
324 if (!clock_running && REGION_HEADER_A.containsPadded(x, y, 12, 10)) {
325     openNameEditor(PPLAYER_A);
326     return;
327 }
328
329 if (!clock_running && REGION_HEADER_B.containsPadded(x, y, 12, 10)) {
330     openNameEditor(PPLAYER_B);
331     return;
332 }
333
334 if (mode == MODE_MATCH && (REGION_SETTINGS.contains(x, y) || REGION_MATCH_TARGET.contains(x, y))) {
335     enterMenu();
336     return;
337 }
338
339 if (HIT_PLAY_PAUSE.contains(x, y)) {
340     togglePlayPause();
341     return;
342 }
343
344 if (HIT_END_GAME.contains(x, y)) {
345     if (mode == MODE_TIMEOUT) {
346         mode = MODE_MATCH;
347         startNewGame();
348         return;
349     }
350     if (game_in_progress || clock_running) {
351         endCurrentGame();
352     }
353     return;
354 }
355
356 if (HIT_END_MATCH.contains(x, y)) {
357     if (mode == MODE_MATCH_OVER) {

```

```

358     resetMatch();
359     startNewGame();
360 } else {
361     pauseClock();
362     mode = MODE_CONFIRM_END_MATCH;
363     markDisplayDirty();
364 }
365 return;
366 }
367
368 if (timerAHit) {
369     switchClockTo(PPLAYER_B);
370     return;
371 }
372
373 if (timerBHit) {
374     switchClockTo(PPLAYER_A);
375     return;
376 }
377
378 if (!clock_running && REGION_SCORE_A.contains(x, y)) {
379     int delta = (y < REGION_SCORE_A.y + (REGION_SCORE_A.h / 2)) ? +1 : -1;
380     adjustScore(PPLAYER_A, delta);
381     return;
382 }
383
384 if (!clock_running && REGION_SCORE_B.contains(x, y)) {
385     int delta = (y < REGION_SCORE_B.y + (REGION_SCORE_B.h / 2)) ? +1 : -1;
386     adjustScore(PPLAYER_B, delta);
387     return;
388 }
389 }
390
391 void handleTouchSwipe(uint16_t startX, uint16_t startY, uint16_t endX, uint16_t endY) {
392     if ((mode != MODE_MATCH && mode != MODE_TIMEOUT) || clock_running) {
393         return;
394     }
395
396     int delta = (endY < startY) ? +1 : -1;
397     bool scoreA = REGION_SCORE_A.contains(startX, startY);
398     bool scoreB = REGION_SCORE_B.contains(startX, startY);
399
400     if (!scoreA && !scoreB) {
401         return;
402     }
403
404     adjustScore(scoreA ? PPLAYER_A : PPLAYER_B, delta);
405     markDisplayDirty();
406 }

```

5.6 6. Clock engine and timing rules

What this segment does. Implements pause/resume, play/pause toggle, turn switching, visible reserve time, simple/Bronstein delay, Fischer increment, and timer scope.

Why this structure is used. Time is tracked in milliseconds to avoid rounding errors. The code calculates what the active player sees differently from what is stored so the delay period can be displayed correctly.

Source lines 1615–1827.

```

1 void pauseClock() {
2     if (!clock_running || active_player == PPLAYER_NONE) {
3         return;
4     }
5
6     uint32_t nowMs = millis();
7     pausedTurnElapsedMs = nowMs - turnStartMs;
8     clock_running = false;
9     lastTickMs = nowMs;
10    markDisplayDirty();
11 }
12
13 void resumeClock() {
14     if (clock_running || active_player == PPLAYER_NONE || matchWinner != PPLAYER_NONE || timeoutPlayer != PPLAYER_NONE) {
15         return;
16     }
17
18     uint32_t nowMs = millis();
19     turnStartMs = nowMs - pausedTurnElapsedMs;
20     lastTickMs = nowMs;

```

```

21     clock_running = true;
22     markDisplayDirty();
23 }
24
25 void togglePlayPause() {
26     if (mode != MODE_MATCH) {
27         return;
28     }
29
30     if (clock_running) {
31         pauseClock();
32     } else if (!game_in_progress) {
33         startNewGame();
34     } else {
35         resumeClock();
36     }
37 }
38
39 void switchClockTo(Player nextPlayer) {
40     uint32_t nowMs = millis();
41
42     if (mode != MODE_MATCH || matchWinner != PLAYER_NONE || timeoutPlayer != PLAYER_NONE) {
43         return;
44     }
45
46     if (!game_in_progress) {
47         startNewGame();
48     }
49
50     // Once the clock is running, only the active player's own large button
51     // should end the turn. Presses on the inactive side are ignored so an
52     // accidental bump cannot reset the current player's delay window.
53     if (clock_running && active_player == nextPlayer) {
54         return;
55     }
56
57     if (clock_running && active_player != PLAYER_NONE && active_player != nextPlayer) {
58         if (!applyTurnElapsed(active_player, nowMs)) {
59             return;
60         }
61     }
62
63     active_player = nextPlayer;
64     clock_running = true;
65     turnStartMs = nowMs;
66     pausedTurnElapsedMs = 0;
67     lastTickMs = nowMs;
68     markDisplayDirty();
69 }
70
71 // -----
72 // Clock behavior
73 // -----
74 void updateClock() {
75     if (!clock_running || active_player == PLAYER_NONE) {
76         lastTickMs = millis();
77         return;
78     }
79
80     uint32_t nowMs = millis();
81     uint32_t elapsedMs = nowMs - lastTickMs;
82     if (elapsedMs < 1000) {
83         return;
84     }
85
86     uint32_t wholeSeconds = elapsedMs / 1000;
87     lastTickMs += wholeSeconds * 1000;
88     match_elapsed_seconds += wholeSeconds;
89
90     if (visibleReserveMs(active_player, nowMs) == 0) {
91         uint32_t *remaining = reserveFor(active_player);
92         if (remaining != nullptr) {
93             *remaining = 0;
94         }
95         clock_running = false;
96         timeoutPlayer = active_player;
97         active_player = PLAYER_NONE;
98         if (crawford_active) {
99             markCrawfordUsed();
100         }
101         game_in_progress = false;
102         mode = MODE_TIMEOUT;
103         playTimeoutBeep();
104         markDisplayDirty();
105         return;
106     }

```

```

107 // Low-time warning. The critical zone beeps more often.
108 uint32_t visibleSeconds = visibleReserveMs(active_player, nowMs) / 1000UL;
109 if (visibleSeconds <= LOW_TIME_WARNING_SECONDS) {
110     uint32_t interval = (visibleSeconds <= CRITICAL_TIME_WARNING_SECONDS) ? 1000UL : 5000UL;
111     if (nowMs - lastLowTimeBeepMs >= interval) {
112         beep(2200, 45);
113         lastLowTimeBeepMs = nowMs;
114     }
115 }
116 }
117 }
118
119 void addIncrementTo(Player player) {
120     if (increment_seconds == 0) {
121         return;
122     }
123
124     if (player == PLAYER_A) {
125         playerA_time_remaining += (uint32_t)increment_seconds * 1000UL;
126     } else if (player == PLAYER_B) {
127         playerB_time_remaining += (uint32_t)increment_seconds * 1000UL;
128     }
129     markDisplayDirty();
130 }
131
132 bool applyTurnElapsed(Player player, uint32_t nowMs) {
133     uint32_t *reserve = reserveFor(player);
134     if (reserve == nullptr) {
135         return true;
136     }
137
138     uint32_t elapsedMs = nowMs - turnStartMs;
139     uint32_t delayOrIncrementMs = (uint32_t)delay_seconds * 1000UL;
140     uint32_t deductionMs = 0;
141
142     if (timingMode == FISCHER_INCREMENT) {
143         deductionMs = elapsedMs;
144     } else {
145         deductionMs = (elapsedMs > delayOrIncrementMs) ? (elapsedMs - delayOrIncrementMs) : 0;
146     }
147
148     if (deductionMs >= *reserve) {
149         *reserve = 0;
150         clock_running = false;
151         timeoutPlayer = player;
152         active_player = PLAYER_NONE;
153         if (crawford_active) {
154             markCrawfordUsed();
155         }
156         game_in_progress = false;
157         mode = MODE_TIMEOUT;
158         playTimeoutBeep();
159         markDisplayDirty();
160         return false;
161     }
162
163     *reserve -= deductionMs;
164     if (timingMode == FISCHER_INCREMENT) {
165         *reserve += delayOrIncrementMs;
166     }
167     return true;
168 }
169
170 uint32_t visibleReserveMs(Player player, uint32_t nowMs) {
171     uint32_t *reserve = reserveFor(player);
172     if (reserve == nullptr) {
173         return 0;
174     }
175     if (!clock_running || active_player != player) {
176         return *reserve;
177     }
178
179     uint32_t elapsedMs = nowMs - turnStartMs;
180     uint32_t delayOrIncrementMs = (uint32_t)delay_seconds * 1000UL;
181     uint32_t deductionMs = 0;
182
183     if (timingMode == SIMPLE_DELAY) {
184         deductionMs = (elapsedMs > delayOrIncrementMs) ? (elapsedMs - delayOrIncrementMs) : 0;
185     } else {
186         deductionMs = elapsedMs;
187     }
188
189     return (deductionMs >= *reserve) ? 0 : (*reserve - deductionMs);
190 }
191
192 uint32_t *reserveFor(Player player) {

```

```

193 if (player == PLAYER_A) {
194     return &playerA_time_remaining;
195 }
196 if (player == PLAYER_B) {
197     return &playerB_time_remaining;
198 }
199 return nullptr;
200 }
201
202 uint32_t matchClockSeconds() {
203     return initial_time_seconds * (uint32_t)match_target;
204 }
205
206 uint32_t startingClockMs() {
207     uint32_t seconds = (timerScope == TIMER_BY_MATCH) ? matchClockSeconds() : initial_time_seconds;
208     return seconds * 1000UL;
209 }
210
211 // -----
212 // Score and match state
213 // -----

```

5.7 7. Scoring, Crawford logic, match lifecycle, and menu changes

What this segment does. Handles score changes, Crawford/post-Crawford transitions, new games, match end/reset, settings navigation, and match target changes.

Why this structure is used. This centralizes the rules-sensitive parts of backgammon match play. Crawford state and cube availability are updated together so score changes do not leave the match in an inconsistent state.

Source lines 1828–2137.

```

1 void adjustScore(Player player, int delta) {
2     if ((mode != MODE_MATCH && mode != MODE_TIMEOUT) || clock_running) {
3         return;
4     }
5
6     bool scorerIsOpponentOfCrawfordPlayer =
7         (match_target > 1) &&
8         ((player == PLAYER_A && playerB_score == match_target - 1) ||
9          (player == PLAYER_B && playerA_score == match_target - 1));
10
11     if (delta > 0 && (crawford_active || scorerIsOpponentOfCrawfordPlayer)) {
12         // Any score recorded during the Crawford game ends Crawford immediately.
13         // Also catch the real scoreboard pattern: the one-away player did not
14         // score, so the opponent's score changed and the next game is post-Crawford.
15         markCrawfordUsed();
16     }
17
18     if (delta > 0 && game_in_progress) {
19         // A positive score adjustment is treated as the end of the current game.
20         // Manual corrections with minus buttons do not advance Crawford state.
21         endCurrentGame();
22     }
23
24     int *score = (player == PLAYER_A) ? &playerA_score : &playerB_score;
25     *score += delta;
26     if (*score < 0) {
27         *score = 0;
28     }
29     if (*score > match_target) {
30         *score = match_target;
31     }
32
33     checkMatchState();
34     markDisplayDirty();
35
36     if (matchWinner != PLAYER_NONE || mode == MODE_MATCH_OVER) {
37         return;
38     }
39
40     // New games are counted only when startNewGame() is explicitly called.
41     // Scoring corrections should not advance the game counter.
42 }
43
44 void checkMatchState() {
45     if (playerA_score >= match_target || playerB_score >= match_target) {
46         matchWinner = (playerA_score >= match_target) ? PLAYER_A : PLAYER_B;
47         clock_running = false;
48         active_player = PLAYER_NONE;

```

```

49     game_in_progress = false;
50     recordMatchLog();
51     mode = MODE_MATCH_OVER;
52     playMatchOverBeep();
53     markDisplayDirty();
54     return;
55 }
56
57 if (crawford_used) {
58     pending_crawford_next_game = false;
59     if (!crawford_active) {
60         post_crawford = true;
61         cube_enabled = true;
62     }
63     return;
64 }
65
66 // Crawford is only meaningful for match targets above 1.
67 if (match_target > 1 && !crawford_used && !crawford_active) {
68     bool someoneAtCrawfordScore =
69         (playerA_score == match_target - 1) || (playerB_score == match_target - 1);
70
71     pending_crawford_next_game = someoneAtCrawfordScore;
72 }
73 }
74
75 void startNewGame() {
76     if (mode == MODE_MATCH_OVER) {
77         return;
78     }
79
80     game_number++;
81     game_in_progress = true;
82     clock_running = false;
83     active_player = PLAYER_NONE;
84     timeoutPlayer = PLAYER_NONE;
85     if (timerScope == TIMER_BY_GAME) {
86         playerA_time_remaining = startingClockMs();
87         playerB_time_remaining = startingClockMs();
88     }
89     pausedTurnElapsedMs = 0;
90     lastLowTimeBeepMs = 0;
91
92     if (pending_crawford_next_game && !crawford_used) {
93         crawford_active = true;
94         post_crawford = false;
95         cube_enabled = false;
96         pending_crawford_next_game = false;
97     } else if (crawford_used) {
98         crawford_active = false;
99         post_crawford = true;
100        cube_enabled = true;
101    } else {
102        crawford_active = false;
103        post_crawford = false;
104        cube_enabled = true;
105    }
106
107    lastTickMs = millis();
108    turnStartMs = lastTickMs;
109    markDisplayDirty();
110 }
111
112 void endCurrentGame() {
113     clock_running = false;
114     active_player = PLAYER_NONE;
115     pausedTurnElapsedMs = 0;
116
117     if (crawford_active) {
118         markCrawfordUsed();
119     }
120
121     game_in_progress = false;
122     markDisplayDirty();
123 }
124
125 void markCrawfordUsed() {
126     crawford_active = false;
127     crawford_used = true;
128     post_crawford = true;
129     pending_crawford_next_game = false;
130     cube_enabled = true;
131 }
132
133 void forceEndMatch() {
134     clock_running = false;

```

```

135     active_player = PLAYER_NONE;
136     pausedTurnElapsedMs = 0;
137     game_in_progress = false;
138
139     matchWinner = (playerA_score == playerB_score) ? PLAYER_NONE :
140                   ((playerA_score > playerB_score) ? PLAYER_A : PLAYER_B);
141     recordMatchLog();
142     mode = MODE_MATCH_OVER;
143     playMatchOverBeep();
144     markDisplayDirty();
145 }
146
147 void resetMatch() {
148     playerA_score = 0;
149     playerB_score = 0;
150     playerA_time_remaining = startingClockMs();
151     playerB_time_remaining = startingClockMs();
152     match_elapsed_seconds = 0;
153     active_player = PLAYER_NONE;
154     clock_running = false;
155     pausedTurnElapsedMs = 0;
156     crawford_used = false;
157     crawford_active = false;
158     post_crawford = false;
159     cube_enabled = true;
160     game_number = 0;
161     timeoutPlayer = PLAYER_NONE;
162     matchWinner = PLAYER_NONE;
163     pending_crawford_next_game = false;
164     game_in_progress = false;
165     mode = MODE_MATCH;
166     markDisplayDirty();
167 }
168
169 // -----
170 // Menu behavior
171 // -----
172 void enterMenu() {
173     clock_running = false;
174     active_player = PLAYER_NONE;
175     mode = MODE_MENU;
176     markDisplayDirty();
177 }
178
179 void handleMenu() {
180     if (btnPlayerA.wasPressed() || btnUp.wasPressed()) {
181         changeMenuValue(+1);
182     }
183
184     if (btnPlayerB.wasPressed() || btnDown.wasPressed()) {
185         changeMenuValue(-1);
186     }
187
188     if (btnScoreAPlus.wasPressed() || btnScoreBPlus.wasPressed()) {
189         nextMenuItem();
190     }
191
192     if (btnScoreAMinus.wasPressed() || btnScoreBMinus.wasPressed()) {
193         previousMenuItem();
194     }
195
196     if (btnSelect.wasPressed()) {
197         if (currentMenuItem == MENU_START_MATCH) {
198             resetMatch();
199             startNewGame();
200         } else if (currentMenuItem == MENU_RESET_MATCH) {
201             resetMatch();
202             enterMenu();
203         } else {
204             nextMenuItem();
205         }
206     }
207 }
208
209 void changeMenuValue(int delta) {
210     switch (currentMenuItem) {
211     case MENU_MATCH_TARGET: {
212         int index = findCurrentTargetIndex();
213         index += delta;
214         if (index < 0) {
215             index = MATCH_TARGET_COUNT - 1;
216         }
217         if (index >= MATCH_TARGET_COUNT) {
218             index = 0;
219         }
220         match_target = MATCH_TARGETS[index];

```

```

221     playerA_time_remaining = startingClockMs();
222     playerB_time_remaining = startingClockMs();
223     markSettingsDirty();
224     break;
225 }
226
227 case MENU_INITIAL_TIME: {
228     int32_t next = (int32_t)initial_time_seconds + (delta * 30);
229     if (next < 60) {
230         next = 60;
231     }
232     if (next > 99L * 60L) {
233         next = 99L * 60L;
234     }
235     initial_time_seconds = (uint32_t)next;
236     playerA_time_remaining = startingClockMs();
237     playerB_time_remaining = startingClockMs();
238     markSettingsDirty();
239     break;
240 }
241
242 case MENU_DELAY: {
243     int next = (int)delay_seconds + delta;
244     if (next < 0) {
245         next = 0;
246     }
247     if (next > 60) {
248         next = 60;
249     }
250     delay_seconds = (uint8_t)next;
251     markSettingsDirty();
252     break;
253 }
254
255 case MENU_INCREMENT: {
256     int next = (int)increment_seconds + delta;
257     if (next < 0) {
258         next = 0;
259     }
260     if (next > 60) {
261         next = 60;
262     }
263     increment_seconds = (uint8_t)next;
264     markSettingsDirty();
265     break;
266 }
267
268 case MENU_BRIGHTNESS: {
269     int next = (int)brightness + (delta * 15);
270     if (next < 15) {
271         next = 15;
272     }
273     if (next > 255) {
274         next = 255;
275     }
276     brightness = (uint8_t)next;
277     setBrightness(brightness);
278     markSettingsDirty();
279     break;
280 }
281
282 default:
283     break;
284 }
285 markDisplayDirty();
286 }
287
288 void nextMenuItem() {
289     currentMenuItem = (MenuItem)((((uint8_t)currentMenuItem + 1) % MENU_COUNT));
290     markDisplayDirty();
291 }
292
293 void previousMenuItem() {
294     uint8_t item = (uint8_t)currentMenuItem;
295     currentMenuItem = (MenuItem)((item == 0) ? (MENU_COUNT - 1) : (item - 1));
296     markDisplayDirty();
297 }
298
299 uint8_t findCurrentTargetIndex() {
300     for (uint8_t i = 0; i < MATCH_TARGET_COUNT; i++) {
301         if (MATCH_TARGETS[i] == match_target) {
302             return i;
303         }
304     }
305     return 4; // Default to match to 9.
306 }

```

```

307 // -----
308 // Display
309 // -----
310

```

5.8 8. Display renderer and main screens

What this segment does. drawDisplay selects the active screen and draws the main match screen, settings screen, match winner banner, timeout banner, confirmation dialog, match log, and clock settings screen.

Why this structure is used. The UI is rendered from state rather than drawn ad hoc after each input. This makes the application predictable: when a flag changes, the screen is rebuilt according to the current state.

Source lines 2138–2489.

```

1 void drawDisplay() {
2     bool modeChanged = !displayModeKnown || mode != lastDrawMode;
3     if (modeChanged) {
4         tft.fillScreen(TFT_BACKGROUND);
5     }
6     displayModeKnown = true;
7     lastDrawMode = mode;
8
9     if (mode == MODE_NAME_EDIT) {
10         drawNameEditor();
11         return;
12     }
13
14     if (mode == MODE_LOG) {
15         drawMatchLog();
16         return;
17     }
18
19     if (mode == MODE_CLOCK_SET) {
20         drawClockSettings();
21         return;
22     }
23
24     if (mode == MODE_MENU) {
25         tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
26         tft.setTextSize(3);
27         tft.setCursor(248, 28);
28         tft.print("SETTINGS");
29
30         tft.fillRect(SETTINGS_BACK.x, SETTINGS_BACK.y, SETTINGS_BACK.w, SETTINGS_BACK.h, TFT_PANEL);
31         tft.drawRoundRect(SETTINGS_BACK.x, SETTINGS_BACK.y, SETTINGS_BACK.w, SETTINGS_BACK.h, 6, TFT_BORDER);
32         tft.setTextSize(2);
33         tft.setCursor(44, 38);
34         tft.print("BACK");
35
36         tft.fillRect(52, 82, 696, 58, TFT_PANEL);
37         tft.fillRect(52, 162, 696, 58, TFT_PANEL);
38         tft.fillRect(52, 242, 696, 58, TFT_PANEL);
39         tft.drawRoundRect(52, 82, 696, 58, 6, TFT_BORDER);
40         tft.drawRoundRect(52, 162, 696, 58, 6, TFT_BORDER);
41         tft.drawRoundRect(52, 242, 696, 58, 6, TFT_BORDER);
42
43         tft.setTextColor(TFT_TEXT, TFT_PANEL);
44         tft.setTextSize(2);
45         tft.setCursor(76, 102);
46         tft.print("MATCH TARGET");
47         tft.setCursor(536, 102);
48         tft.printf("%d", match_target);
49
50         tft.setCursor(76, 182);
51         tft.print("TIME / GAME");
52         tft.setCursor(520, 182);
53         tft.print(formatTime(initial_time_seconds));
54         tft.setTextColor(TFT_MUTED, TFT_PANEL);
55         tft.setTextSize(1);
56         tft.setCursor(76, 207);
57         tft.print("TOTAL ");
58         tft.print(formatTime(matchClockSeconds()));
59         tft.setTextColor(TFT_TEXT, TFT_PANEL);
60         tft.setTextSize(2);
61
62         tft.setCursor(76, 262);
63         tft.print("BONUS / DELAY");
64         tft.setCursor(536, 262);
65         tft.printf("%u", delay_seconds);

```

```

66
67     const Rect controls[] = {
68         SETTINGS_MATCH_MINUS, SETTINGS_MATCH_PLUS,
69         SETTINGS_TIME_MINUS, SETTINGS_TIME_PLUS,
70         SETTINGS_DELAY_MINUS, SETTINGS_DELAY_PLUS
71     };
72     const char *symbols[] = {"-", "+", "-", "+", "-", "+"};
73     for (uint8_t i = 0; i < 6; i++) {
74         tft.fillRect(controls[i].x, controls[i].y, controls[i].w, controls[i].h, TFT_PANEL_ALT);
75         tft.drawRoundRect(controls[i].x, controls[i].y, controls[i].w, controls[i].h, 6, TFT_BORDER);
76         tft.setTextColor(TFT_TEXT, TFT_PANEL_ALT);
77         tft.setTextSize(3);
78         tft.setCursor(controls[i].x + 40, controls[i].y + 14);
79         tft.print(symbols[i]);
80     }
81
82     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
83     tft.setTextSize(2);
84     tft.setCursor(76, 315);
85     tft.print("TIMER");
86     tft.fillRect(SETTINGS_SCOPE_GAME.x, SETTINGS_SCOPE_GAME.y, SETTINGS_SCOPE_GAME.w, SETTINGS_SCOPE_GAME.h,
87         timerScope == TIMER_BY_GAME ? TFT_OK : TFT_PANEL_ALT);
88     tft.drawRoundRect(SETTINGS_SCOPE_GAME.x, SETTINGS_SCOPE_GAME.y, SETTINGS_SCOPE_GAME.w, SETTINGS_SCOPE_GAME.h, 6, TFT_BORDER);
89     drawCenteredInRect("GAME", SETTINGS_SCOPE_GAME, 2, timerScope == TIMER_BY_GAME ? ILI9341_BLACK : TFT_TEXT,
90         timerScope == TIMER_BY_GAME ? TFT_OK : TFT_PANEL_ALT);
91     tft.fillRect(SETTINGS_SCOPE_MATCH.x, SETTINGS_SCOPE_MATCH.y, SETTINGS_SCOPE_MATCH.w, SETTINGS_SCOPE_MATCH.h,
92         timerScope == TIMER_BY_MATCH ? TFT_OK : TFT_PANEL_ALT);
93     tft.drawRoundRect(SETTINGS_SCOPE_MATCH.x, SETTINGS_SCOPE_MATCH.y, SETTINGS_SCOPE_MATCH.w, SETTINGS_SCOPE_MATCH.h, 6, TFT_BORDER);
94     drawCenteredInRect("MATCH", SETTINGS_SCOPE_MATCH, 2, timerScope == TIMER_BY_MATCH ? ILI9341_BLACK : TFT_TEXT,
95         timerScope == TIMER_BY_MATCH ? TFT_OK : TFT_PANEL_ALT);
96
97     const Rect modeButtons[] = {SETTINGS_MODE_SIMPLE, SETTINGS_MODE_BRONSTEIN, SETTINGS_MODE_FISCHER};
98     const char *modeLabels[] = {"SIMPLE", "BRONSTEIN", "FISCHER"};
99     for (uint8_t i = 0; i < 3; i++) {
100         bool selected = (uint8_t)timingMode == i;
101         tft.fillRect(modeButtons[i].x, modeButtons[i].y, modeButtons[i].w, modeButtons[i].h,
102             selected ? TFT_OK : TFT_PANEL_ALT);
103         tft.drawRoundRect(modeButtons[i].x, modeButtons[i].y, modeButtons[i].w, modeButtons[i].h, 6, TFT_BORDER);
104         drawCenteredInRect(modeLabels[i], modeButtons[i], 2, selected ? ILI9341_BLACK : TFT_TEXT,
105             selected ? TFT_OK : TFT_PANEL_ALT);
106     }
107
108     tft.fillRect(SETTINGS_LOG.x, SETTINGS_LOG.y, SETTINGS_LOG.w, SETTINGS_LOG.h, TFT_PANEL_ALT);
109     tft.drawRoundRect(SETTINGS_LOG.x, SETTINGS_LOG.y, SETTINGS_LOG.w, SETTINGS_LOG.h, 6, TFT_BORDER);
110     drawCenteredInRect("MATCH LOG", SETTINGS_LOG, 2, TFT_TEXT, TFT_PANEL_ALT);
111
112     tft.fillRect(SETTINGS_CLOCK.x, SETTINGS_CLOCK.y, SETTINGS_CLOCK.w, SETTINGS_CLOCK.h, TFT_PANEL_ALT);
113     tft.drawRoundRect(SETTINGS_CLOCK.x, SETTINGS_CLOCK.y, SETTINGS_CLOCK.w, SETTINGS_CLOCK.h, 6, TFT_BORDER);
114     drawCenteredInRect("CLOCK", SETTINGS_CLOCK, 2, TFT_TEXT, TFT_PANEL_ALT);
115
116     tft.fillRect(SETTINGS_START.x, SETTINGS_START.y, SETTINGS_START.w, SETTINGS_START.h, TFT_OK);
117     tft.fillRect(SETTINGS_END.x, SETTINGS_END.y, SETTINGS_END.w, SETTINGS_END.h, TFT_DANGER);
118     tft.drawRoundRect(SETTINGS_START.x, SETTINGS_START.y, SETTINGS_START.w, SETTINGS_START.h, 6, TFT_BORDER);
119     tft.drawRoundRect(SETTINGS_END.x, SETTINGS_END.y, SETTINGS_END.w, SETTINGS_END.h, 6, TFT_BORDER);
120     drawCenteredInRect("START MATCH", SETTINGS_START, 2, ILI9341_BLACK, TFT_OK);
121     drawCenteredInRect("END MATCH", SETTINGS_END, 2, TFT_TEXT, TFT_DANGER);
122
123     return;
124 }
125
126 drawStatusBar();
127
128 bool winnerA = (mode == MODE_MATCH_OVER && matchWinner == PLAYER_A);
129 bool winnerB = (mode == MODE_MATCH_OVER && matchWinner == PLAYER_B);
130 uint16_t headerABg = winnerA ? ILI9341_WHITE : TFT_PANEL;
131 uint16_t headerBBg = winnerB ? ILI9341_WHITE : TFT_PANEL;
132 uint16_t headerAText = winnerA ? TFT_TEXT_DARK : TFT_TEXT;
133 uint16_t headerBText = winnerB ? TFT_TEXT_DARK : TFT_TEXT;
134
135 tft.fillRect(REGION_HEADER_A.x, REGION_HEADER_A.y, REGION_HEADER_A.w, REGION_HEADER_A.h, headerABg);
136 tft.fillRect(REGION_HEADER_B.x, REGION_HEADER_B.y, REGION_HEADER_B.w, REGION_HEADER_B.h, headerBBg);
137 drawTimerPanel(PLAYER_A, REGION_TIMER_A);
138 tft.fillRect(REGION_PLAY_PAUSE.x, REGION_PLAY_PAUSE.y, REGION_PLAY_PAUSE.w, REGION_PLAY_PAUSE.h, TFT_PANEL);
139 tft.fillRect(REGION_END_GAME.x, REGION_END_GAME.y, REGION_END_GAME.w, REGION_END_GAME.h, TFT_PANEL);
140 tft.fillRect(REGION_END_MATCH.x, REGION_END_MATCH.y, REGION_END_MATCH.w, REGION_END_MATCH.h,
141     mode == MODE_MATCH_OVER ? TFT_DANGER : TFT_PANEL);
142 tft.fillRect(REGION_SCORE_A.x, REGION_SCORE_A.y, REGION_SCORE_A.w, REGION_SCORE_A.h, playerA_color);
143 tft.fillRect(REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y, REGION_MATCH_TARGET.w, REGION_MATCH_TARGET.h, TFT_PANEL);
144 tft.fillRect(REGION_SETTINGS.x, REGION_SETTINGS.y, REGION_SETTINGS.w, REGION_SETTINGS.h, TFT_PANEL);
145 tft.fillRect(REGION_SCORE_B.x, REGION_SCORE_B.y, REGION_SCORE_B.w, REGION_SCORE_B.h, playerB_color);
146
147 tft.drawRoundRect(REGION_HEADER_A.x, REGION_HEADER_A.y, REGION_HEADER_A.w, REGION_HEADER_A.h, 4, TFT_ACCENT_A);
148 tft.drawRoundRect(REGION_HEADER_B.x, REGION_HEADER_B.y, REGION_HEADER_B.w, REGION_HEADER_B.h, 4, TFT_ACCENT_B);
149 tft.drawRoundRect(REGION_PLAY_PAUSE.x, REGION_PLAY_PAUSE.y, REGION_PLAY_PAUSE.w, REGION_PLAY_PAUSE.h, 6, TFT_BUTTON_BORDER);
150 tft.drawRoundRect(REGION_END_GAME.x, REGION_END_GAME.y, REGION_END_GAME.w, REGION_END_GAME.h, 6, TFT_BUTTON_BORDER);
151 tft.drawRoundRect(REGION_END_MATCH.x, REGION_END_MATCH.y, REGION_END_MATCH.w, REGION_END_MATCH.h, 6, TFT_BUTTON_BORDER);

```

```

152 drawTimerPanel(PLAYER_B, REGION_TIMER_B);
153 tft.drawRoundRect(REGION_SCORE_A.x, REGION_SCORE_A.y, REGION_SCORE_A.w, REGION_SCORE_A.h, 6, TFT_ACCENT_A);
154 tft.drawRoundRect(REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y, REGION_MATCH_TARGET.w, REGION_MATCH_TARGET.h, 6, TFT_BUTTON_BORDER);
155 tft.drawRoundRect(REGION_SETTINGS.x, REGION_SETTINGS.y, REGION_SETTINGS.w, REGION_SETTINGS.h, 6, TFT_BUTTON_BORDER);
156 tft.drawRoundRect(REGION_SCORE_B.x, REGION_SCORE_B.y, REGION_SCORE_B.w, REGION_SCORE_B.h, 6, TFT_ACCENT_B);
157
158 tft.setTextSize(2);
159 tft.setTextColor(TFT_TEXT, TFT_PANEL);
160 drawCenteredInRect(playerA_name, REGION_HEADER_A, playerA_name.length() > 9 ? 1 : 2, headerAText, headerABg);
161 drawCenteredInRect(playerB_name, REGION_HEADER_B, playerB_name.length() > 9 ? 1 : 2, headerBText, headerBBg);
162
163 drawCenteredInRect(playPauseLabel(), REGION_PLAY_PAUSE, 2, TFT_TEXT, TFT_PANEL);
164
165 drawCenteredInRect(mode == MODE_TIMEOUT ? "New Game" : "End Game", REGION_END_GAME, 2, TFT_TEXT, TFT_PANEL);
166 drawCenteredInRect(mode == MODE_MATCH_OVER ? "Reset" : "End Match", REGION_END_MATCH, 2, TFT_TEXT, mode == MODE_MATCH_OVER ? TFT_DANGER
    ↪ : TFT_PANEL);
167 tft.setTextColor(TFT_TEXT, TFT_PANEL);
168
169 drawScoreInRect(playerA_score, REGION_SCORE_A, textColorForPlayerColor(playerA_color), playerA_color);
170 drawScoreInRect(playerB_score, REGION_SCORE_B, textColorForPlayerColor(playerB_color), playerB_color);
171
172 drawCenteredInRect("Match to:", {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 22, REGION_MATCH_TARGET.w, 26}, 2, TFT_TEXT, TFT_PANEL)
    ↪ ;
173 drawMatchTargetInRect(match_target, {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 56, REGION_MATCH_TARGET.w, 74}, TFT_TEXT, TFT_PANEL
    ↪ );
174 drawCenteredInRect(String("Game ") + String(game_number), {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 136, REGION_MATCH_TARGET.w,
    ↪ 28}, 2, TFT_TEXT, TFT_PANEL);
175 String stateLabel = "";
176 uint16_t stateColor = TFT_MAGENTA;
177 if (mode == MODE_MATCH_OVER) {
178     stateLabel = "Match Ended";
179     stateColor = TFT_WARN;
180 } else if (crawford_active || pending_crawford_next_game) {
181     stateLabel = "CRAWFORD";
182 } else if (post_crawford || crawford_used) {
183     stateLabel = "POST CRAWFORD";
184 }
185 drawCenteredInRect(stateLabel, {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 166, REGION_MATCH_TARGET.w, 30}, 2, stateColor,
    ↪ TFT_PANEL);
186
187 tft.setTextColor(TFT_TEXT, TFT_PANEL);
188 drawCenteredInRect("SETTINGS", REGION_SETTINGS, 2, TFT_TEXT, TFT_PANEL);
189
190 if (mode == MODE_TIMEOUT) {
191     drawTimeoutBanner();
192 }
193
194 if (mode == MODE_CONFIRM_END_MATCH) {
195     drawEndMatchConfirm();
196 }
197
198 }
199
200 void drawMatchWinnerBanner() {
201     const Rect banner = {6, 204, 788, 260};
202     const Rect nameLine = {24, 270, 752, 76};
203     const Rect winnerLine = {24, 348, 752, 76};
204     String winnerName = (matchWinner == PLAYER_A) ? playerA_name : playerB_name;
205
206     tft.fillRect(banner.x, banner.y, banner.w, banner.h, TFT_PANEL);
207     tft.drawRoundRect(banner.x, banner.y, banner.w, banner.h, 6, TFT_OK);
208     drawCenteredInRect(winnerName, nameLine, winnerName.length() > 8 ? 6 : 7, TFT_TEXT, TFT_PANEL);
209     drawCenteredInRect("IS THE WINNER", winnerLine, 5, TFT_OK, TFT_PANEL);
210 }
211
212 void drawTimeoutBanner() {
213     const Rect banner = {42, 224, 716, 174};
214     Player loser = timeoutPlayer;
215     String loserName = (loser == PLAYER_A) ? playerA_name : playerB_name;
216
217     tft.fillRect(banner.x, banner.y, banner.w, banner.h, TFT_PANEL);
218     tft.drawRoundRect(banner.x, banner.y, banner.w, banner.h, 6, TFT_DANGER);
219     drawCenteredInRect(loserName, {banner.x + 16, banner.y + 24, banner.w - 32, 48}, loserName.length() > 8 ? 4 : 5, TFT_TEXT, TFT_PANEL);
220     drawCenteredInRect("RAN OUT OF TIME", {banner.x + 16, banner.y + 80, banner.w - 32, 36}, 3, TFT_DANGER, TFT_PANEL);
221     drawCenteredInRect("LOSES ONE POINT", {banner.x + 16, banner.y + 122, banner.w - 32, 34}, 3, TFT_WARN, TFT_PANEL);
222 }
223
224 void drawEndMatchConfirm() {
225     const Rect dialog = {118, 132, 564, 236};
226
227     tft.fillRect(dialog.x, dialog.y, dialog.w, dialog.h, TFT_PANEL);
228     tft.drawRoundRect(dialog.x, dialog.y, dialog.w, dialog.h, 8, TFT_BUTTON_BORDER);
229
230     drawCenteredInRect("END MATCH?", {dialog.x, dialog.y + 28, dialog.w, 34}, 3, TFT_WARN, TFT_PANEL);
231     drawCenteredInRect("This will finish the match", {dialog.x, dialog.y + 84, dialog.w, 24}, 2, TFT_TEXT, TFT_PANEL);
232     drawCenteredInRect("and record the current leader.", {dialog.x, dialog.y + 112, dialog.w, 24}, 2, TFT_TEXT, TFT_PANEL);

```

```

233 tft.fillRect(CONFIRM_NO.x, CONFIRM_NO.y, CONFIRM_NO.w, CONFIRM_NO.h, TFT_PANEL_ALT);
234 tft.drawRoundRect(CONFIRM_NO.x, CONFIRM_NO.y, CONFIRM_NO.w, CONFIRM_NO.h, 6, TFT_BUTTON_BORDER);
235 drawCenteredInRect("NO", CONFIRM_NO, 3, TFT_TEXT, TFT_PANEL_ALT);
236
237
238 tft.fillRect(CONFIRM_YES.x, CONFIRM_YES.y, CONFIRM_YES.w, CONFIRM_YES.h, TFT_DANGER);
239 tft.drawRoundRect(CONFIRM_YES.x, CONFIRM_YES.y, CONFIRM_YES.w, CONFIRM_YES.h, 6, TFT_BUTTON_BORDER);
240 drawCenteredInRect("YES", CONFIRM_YES, 3, TFT_TEXT, TFT_DANGER);
241 }
242
243 void drawMatchLog() {
244     tft.fillScreen(TFT_BACKGROUND);
245
246     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
247     tft.setTextSize(3);
248     tft.setCursor(286, 28);
249     tft.print("MATCH LOG");
250
251     tft.fillRect(LOG_BACK.x, LOG_BACK.y, LOG_BACK.w, LOG_BACK.h, TFT_PANEL);
252     tft.drawRoundRect(LOG_BACK.x, LOG_BACK.y, LOG_BACK.w, LOG_BACK.h, 6, TFT_BORDER);
253     drawCenteredInRect("BACK", LOG_BACK, 2, TFT_TEXT, TFT_PANEL);
254
255     tft.fillRect(LOG_CLEAR.x, LOG_CLEAR.y, LOG_CLEAR.w, LOG_CLEAR.h, TFT_DANGER);
256     tft.drawRoundRect(LOG_CLEAR.x, LOG_CLEAR.y, LOG_CLEAR.w, LOG_CLEAR.h, 6, TFT_BORDER);
257     drawCenteredInRect("CLEAR", LOG_CLEAR, 2, TFT_TEXT, TFT_DANGER);
258
259     if (matchLogCount == 0) {
260         drawCenteredInRect("NO MATCHES YET", {120, 210, 560, 70}, 3, TFT_MUTED, TFT_BACKGROUND);
261         return;
262     }
263
264     for (uint8_t i = 0; i < matchLogCount && i < MATCH_LOG_MAX_ENTRIES; i++) {
265         const int16_t y = 88 + (i * 46);
266         const Rect row = {34, y, 732, 40};
267         String date = matchLogPart(matchLog[i], 0);
268         String nameA = matchLogPart(matchLog[i], 1);
269         String nameB = matchLogPart(matchLog[i], 2);
270         String score = matchLogPart(matchLog[i], 3);
271         String names = nameA + " VS " + nameB;
272         String scoreLine = "SCORE " + score;
273
274         tft.fillRect(row.x, row.y, row.w, row.h, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
275         tft.drawRoundRect(row.x, row.y, row.w, row.h, 4, TFT_BORDER);
276         tft.setTextColor(TFT_MUTED, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
277         tft.setTextSize(1);
278         tft.setCursor(row.x + 12, row.y + 8);
279         tft.print(date);
280         tft.setCursor(row.x + 152, row.y + 8);
281         tft.print(names);
282         tft.setTextColor(TFT_TEXT, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
283         tft.setTextSize(2);
284         tft.setCursor(row.x + 560, row.y + 12);
285         tft.print(scoreLine);
286     }
287 }
288
289 void drawClockSettings() {
290     tft.fillScreen(TFT_BACKGROUND);
291
292     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
293     tft.setTextSize(3);
294     tft.setCursor(248, 28);
295     tft.print("CLOCK SET");
296
297     tft.fillRect(CLOCK_BACK.x, CLOCK_BACK.y, CLOCK_BACK.w, CLOCK_BACK.h, TFT_PANEL);
298     tft.drawRoundRect(CLOCK_BACK.x, CLOCK_BACK.y, CLOCK_BACK.w, CLOCK_BACK.h, 6, TFT_BORDER);
299     drawCenteredInRect("BACK", CLOCK_BACK, 2, TFT_TEXT, TFT_PANEL);
300
301     const Rect rows[] = {
302         {52, 82, 696, 48},
303         {52, 140, 696, 48},
304         {52, 198, 696, 48},
305         {52, 256, 696, 48},
306         {52, 314, 696, 48}
307     };
308     const Rect minusButtons[] = {
309         CLOCK_YEAR_MINUS,
310         CLOCK_MONTH_MINUS,
311         CLOCK_DAY_MINUS,
312         CLOCK_HOUR_MINUS,
313         CLOCK_MINUTE_MINUS
314     };
315     const Rect plusButtons[] = {
316         CLOCK_YEAR_PLUS,
317         CLOCK_MONTH_PLUS,
318         CLOCK_DAY_PLUS,

```

```

319     CLOCK_HOUR_PLUS,
320     CLOCK_MINUTE_PLUS
321 };
322 const char *labels[] = {"YEAR", "MONTH", "DAY", "HOUR", "MINUTE"};
323 uint16_t values[] = {editingRtcYear, editingRtcMonth, editingRtcDay, editingRtcHour, editingRtcMinute};
324
325 for (uint8_t i = 0; i < 5; i++) {
326     tft.fillRect(rows[i].x, rows[i].y, rows[i].w, rows[i].h, TFT_PANEL);
327     tft.drawRoundRect(rows[i].x, rows[i].y, rows[i].w, rows[i].h, 6, TFT_BORDER);
328     tft.setTextColor(TFT_TEXT, TFT_PANEL);
329     tft.setTextSize(2);
330     tft.setCursor(76, rows[i].y + 16);
331     tft.print(labels[i]);
332     tft.setCursor(i == 0 ? 514 : 536, rows[i].y + 16);
333     if (i == 0) {
334         tft.printf("%u", values[i]);
335     } else {
336         tft.printf("%02u", values[i]);
337     }
338
339     tft.fillRect(minusButtons[i].x, minusButtons[i].y, minusButtons[i].w, minusButtons[i].h, TFT_PANEL_ALT);
340     tft.drawRoundRect(minusButtons[i].x, minusButtons[i].y, minusButtons[i].w, minusButtons[i].h, 6, TFT_BORDER);
341     drawCenteredInRect("-", minusButtons[i], 3, TFT_TEXT, TFT_PANEL_ALT);
342     tft.fillRect(plusButtons[i].x, plusButtons[i].y, plusButtons[i].w, plusButtons[i].h, TFT_PANEL_ALT);
343     tft.drawRoundRect(plusButtons[i].x, plusButtons[i].y, plusButtons[i].w, plusButtons[i].h, 6, TFT_BORDER);
344     drawCenteredInRect("+", plusButtons[i], 3, TFT_TEXT, TFT_PANEL_ALT);
345 }
346
347 tft.setTextColor(ds3231Found ? TFT_MUTED : TFT_DANGER, TFT_BACKGROUND);
348 tft.setTextSize(1);
349 tft.setCursor(72, 380);
350 tft.print(ds3231Found ? "SAVE writes this date/time to the DS3231 RTC." : "DS3231 RTC not detected on I2C address 0x68.");
351
352 tft.fillRect(CLOCK_SAVE.x, CLOCK_SAVE.y, CLOCK_SAVE.w, CLOCK_SAVE.h, ds3231Found ? TFT_OK : TFT_PANEL_ALT);

```

5.9 9. Name editor, checker-color selection, timer panels, and player panels

What this segment does. Draws and handles the name editor, color swatches, timer panels, score panels, status bar, and active player visual treatment.

Why this structure is used. This is where the V 2.0 visual identity lives. Player color selection gives each side a checker-color identity, while active timer inversion is a separate readability cue for the player currently on turn.

Source lines 2490–2944.

```

1     tft.drawRoundRect(CLOCK_SAVE.x, CLOCK_SAVE.y, CLOCK_SAVE.w, CLOCK_SAVE.h, 6, TFT_BORDER);
2     drawCenteredInRect("SAVE", CLOCK_SAVE, 2, ds3231Found ? ILI9341_BLACK : TFT_MUTED, ds3231Found ? TFT_OK : TFT_PANEL_ALT);
3 }
4
5 void drawNameEditor() {
6     const Rect title = {70, 24, 660, 42};
7     const Rect input = {92, 78, 616, 64};
8     const Rect cancel = {72, 420, 168, 44};
9     const Rect clearKey = {258, 420, 168, 44};
10    const Rect ok = {560, 420, 168, 44};
11    const char *rows[] = {"ABCDEFGHJI", "KLMNOPQRST", "UVWXYZ"};
12    const int16_t rowY[] = {148, 208, 268};
13    const int16_t keyW = 66;
14    const int16_t keyH = 44;
15    const int16_t gap = 8;
16    const int16_t swatchY = 326;
17    const int16_t swatchSize = 36;
18    const int16_t swatchGap = 14;
19
20    tft.fillScreen(TFT_BACKGROUND);
21    tft.fillRect(48, 14, 704, 452, TFT_PANEL);
22    tft.drawRoundRect(48, 14, 704, 452, 8, TFT_BORDER);
23    drawCenteredInRect(editingNameFor == PLAYER_A ? "PLAYER A NAME" : "PLAYER B NAME", title, 3, TFT_TEXT, TFT_PANEL);
24
25    tft.fillRect(input.x, input.y, input.w, input.h, TFT_BACKGROUND);
26    tft.drawRoundRect(input.x, input.y, input.w, input.h, 6, TFT_OK);
27    drawCenteredInRect(editingName.length() == 0 ? " " : editingName, input, 4, TFT_TEXT, TFT_BACKGROUND);
28
29    for (uint8_t r = 0; r < 3; r++) {
30        uint8_t count = strlen(rows[r]);
31        int16_t startX = (PANEL_WIDTH - ((count * keyW) + ((count - 1) * gap))) / 2;
32        for (uint8_t i = 0; i < count; i++) {
33            Rect key = {startX + (int16_t)i * (keyW + gap), rowY[r], keyW, keyH};
34            tft.fillRect(key.x, key.y, key.w, key.h, TFT_PANEL_ALT);
35            tft.drawRoundRect(key.x, key.y, key.w, key.h, 6, TFT_BORDER);

```

```

36     String label;
37     label += rows[r][i];
38     drawCenteredInRect(label, key, 3, TFT_TEXT, TFT_PANEL_ALT);
39 }
40 }
41
42 for (uint8_t i = 0; i < playerColorOptionCount(); i++) {
43     uint16_t startX = (PANEL_WIDTH - (playerColorOptionCount() * swatchSize + (playerColorOptionCount() - 1) * swatchGap)) / 2;
44     Rect swatch = {startX + (uint16_t)i * (swatchSize + swatchGap), swatchY, swatchSize, swatchSize};
45     uint16_t color = playerColorOption(i);
46     tft.fillRect(swatch.x, swatch.y, swatch.w, swatch.h, color);
47     tft.drawRoundRect(swatch.x, swatch.y, swatch.w, swatch.h, 5, color == editingPlayerColor ? TFT_OK : TFT_BORDER);
48     if (color == PLAYER_COLOR_WHITE) {
49         tft.drawRoundRect(swatch.x + 3, swatch.y + 3, swatch.w - 6, swatch.h - 6, 4, TFT_MUTED);
50     }
51 }
52
53 const Rect spaceKey = {166, 372, 244, 38};
54 const Rect delKey = {430, 372, 204, 38};
55 tft.fillRect(spaceKey.x, spaceKey.y, spaceKey.w, spaceKey.h, TFT_PANEL_ALT);
56 tft.drawRoundRect(spaceKey.x, spaceKey.y, spaceKey.w, spaceKey.h, 6, TFT_BORDER);
57 drawCenteredInRect("SPACE", spaceKey, 2, TFT_TEXT, TFT_PANEL_ALT);
58 tft.fillRect(delKey.x, delKey.y, delKey.w, delKey.h, TFT_PANEL_ALT);
59 tft.drawRoundRect(delKey.x, delKey.y, delKey.w, delKey.h, 6, TFT_BORDER);
60 drawCenteredInRect("DEL", delKey, 2, TFT_TEXT, TFT_PANEL_ALT);
61
62 tft.fillRect(cancel.x, cancel.y, cancel.w, cancel.h, TFT_DANGER);
63 tft.drawRoundRect(cancel.x, cancel.y, cancel.w, cancel.h, 6, TFT_BORDER);
64 drawCenteredInRect("CANCEL", cancel, 2, TFT_TEXT, TFT_DANGER);
65 tft.fillRect(clearKey.x, clearKey.y, clearKey.w, clearKey.h, TFT_PANEL_ALT);
66 tft.drawRoundRect(clearKey.x, clearKey.y, clearKey.w, clearKey.h, 6, TFT_BORDER);
67 drawCenteredInRect("CLEAR", clearKey, 2, TFT_TEXT, TFT_PANEL_ALT);
68 tft.fillRect(ok.x, ok.y, ok.w, ok.h, TFT_OK);
69 tft.drawRoundRect(ok.x, ok.y, ok.w, ok.h, 6, TFT_BORDER);
70 drawCenteredInRect("OK", ok, 2, ILI9341_BLACK, TFT_OK);
71 }
72
73 void openNameEditor(Player player) {
74     if (player != PLAYER_A && player != PLAYER_B) {
75         return;
76     }
77
78     clock_running = false;
79     active_player = PLAYER_NONE;
80     editingNameFor = player;
81     editingName = (player == PLAYER_A) ? playerA_name : playerB_name;
82     editingPlayerColor = colorForPlayer(player);
83     mode = MODE_NAME_EDIT;
84     markDisplayDirty();
85 }
86
87 void openClockSettings() {
88     uint8_t second = 0;
89     uint8_t minute = 0;
90     uint8_t hour = 0;
91     uint8_t day = 0;
92     uint8_t month = 0;
93     uint16_t year = 0;
94     if (ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) && rtcDateLooksSet(day, month, year)) {
95         editingRtcYear = year;
96         editingRtcMonth = month;
97         editingRtcDay = day;
98         editingRtcHour = hour;
99         editingRtcMinute = minute;
100     } else {
101         editingRtcYear = 2026;
102         editingRtcMonth = 1;
103         editingRtcDay = 1;
104         editingRtcHour = 12;
105         editingRtcMinute = 0;
106     }
107     mode = MODE_CLOCK_SET;
108     markDisplayDirty();
109 }
110
111 void handleClockSettingsTap(uint16_t x, uint16_t y) {
112     if (CLOCK_BACK.containsPadded(x, y, 18, 14)) {
113         mode = MODE_MENU;
114         markDisplayDirty();
115         return;
116     }
117
118     const Rect minusButtons[] = {
119         CLOCK_YEAR_MINUS,
120         CLOCK_MONTH_MINUS,
121         CLOCK_DAY_MINUS,

```

```

122     CLOCK_HOUR_MINUS,
123     CLOCK_MINUTE_MINUS
124 };
125 const Rect plusButtons[] = {
126     CLOCK_YEAR_PLUS,
127     CLOCK_MONTH_PLUS,
128     CLOCK_DAY_PLUS,
129     CLOCK_HOUR_PLUS,
130     CLOCK_MINUTE_PLUS
131 };
132
133 for (uint8_t i = 0; i < 5; i++) {
134     if (minusButtons[i].containsPadded(x, y, 36, 12)) {
135         adjustRtcField(i, -1);
136         return;
137     }
138     if (plusButtons[i].containsPadded(x, y, 36, 12)) {
139         adjustRtcField(i, +1);
140         return;
141     }
142 }
143
144 if (CLOCK_SAVE.containsPadded(x, y, 18, 14) && ds3231Found) {
145     if (ds3231SetTime(0, editingRtcMinute, editingRtcHour, editingRtcDay, editingRtcMonth, editingRtcYear)) {
146         logPrintf("DS3231 RTC set to %04u-%02u-%02u %02u:%02u:00\n",
147             editingRtcYear, editingRtcMonth, editingRtcDay, editingRtcHour, editingRtcMinute);
148         updateRtcStatusCache(true);
149         mode = MODE_MENU;
150     } else {
151         logLine("DS3231 RTC save failed.");
152     }
153     markDisplayDirty();
154     return;
155 }
156 }
157
158 void handleEndMatchConfirmTap(uint16_t x, uint16_t y) {
159     if (CONFIRM_NO.containsPadded(x, y, 18, 14)) {
160         mode = MODE_MATCH;
161         markDisplayDirty();
162         return;
163     }
164
165     if (CONFIRM_YES.containsPadded(x, y, 18, 14)) {
166         forceEndMatch();
167         return;
168     }
169 }
170
171 void adjustRtcField(uint8_t field, int delta) {
172     switch (field) {
173     case 0:
174         editingRtcYear = constrain((int)editingRtcYear + delta, 2024, 2099);
175         break;
176     case 1:
177         editingRtcMonth = constrain((int)editingRtcMonth + delta, 1, 12);
178         break;
179     case 2:
180         editingRtcDay = constrain((int)editingRtcDay + delta, 1, (int)daysInMonth(editingRtcMonth, editingRtcYear));
181         break;
182     case 3:
183         editingRtcHour = (uint8_t)((((int)editingRtcHour + delta + 24) % 24);
184         break;
185     case 4:
186         editingRtcMinute = (uint8_t)((((int)editingRtcMinute + delta + 60) % 60);
187         break;
188     default:
189         break;
190     }
191
192     uint8_t maxDay = daysInMonth(editingRtcMonth, editingRtcYear);
193     if (editingRtcDay > maxDay) {
194         editingRtcDay = maxDay;
195     }
196     markDisplayDirty();
197 }
198
199 void handleNameEditorTap(uint16_t x, uint16_t y) {
200     const Rect cancel = {72, 420, 168, 44};
201     const Rect clearKey = {258, 420, 168, 44};
202     const Rect ok = {560, 420, 168, 44};
203     const Rect spaceKey = {166, 372, 244, 38};
204     const Rect delKey = {430, 372, 204, 38};
205     const char *rows[] = {"ABCDEFGH IJ", "KLMNOPQRST", "UVWXYZ"};
206     const int16_t rowY[] = {148, 208, 268};
207     const int16_t keyW = 66;

```

```

208 const int16_t keyH = 44;
209 const int16_t gap = 8;
210 const int16_t swatchY = 326;
211 const int16_t swatchSize = 36;
212 const int16_t swatchGap = 14;
213
214 if (cancel.contains(x, y)) {
215     editingNameFor = PLAYER_NONE;
216     mode = MODE_MATCH;
217     markDisplayDirty();
218     return;
219 }
220
221 if (ok.contains(x, y)) {
222     if (editingName.length() == 0) {
223         editingName = (editingNameFor == PLAYER_A) ? "PLAYER A" : "PLAYER B";
224     }
225     if (editingNameFor == PLAYER_A) {
226         playerA_name = editingName;
227         playerA_color = editingPlayerColor;
228     } else if (editingNameFor == PLAYER_B) {
229         playerB_name = editingName;
230         playerB_color = editingPlayerColor;
231     }
232     savePlayerNames();
233     editingNameFor = PLAYER_NONE;
234     mode = MODE_MATCH;
235     markDisplayDirty();
236     return;
237 }
238
239 if (clearKey.contains(x, y)) {
240     editingName = "";
241     markDisplayDirty();
242     return;
243 }
244
245 if (delKey.contains(x, y)) {
246     if (editingName.length() > 0) {
247         editingName.remove(editingName.length() - 1);
248     }
249     markDisplayDirty();
250     return;
251 }
252
253 if (spaceKey.contains(x, y)) {
254     if (editingName.length() < PLAYER_NAME_MAX_LEN && editingName.length() > 0) {
255         editingName += ' ';
256     }
257     markDisplayDirty();
258     return;
259 }
260
261 int16_t swatchStartX = (PANEL_WIDTH - (playerColorOptionCount() * swatchSize + (playerColorOptionCount() - 1) * swatchGap)) / 2;
262 for (uint8_t i = 0; i < playerColorOptionCount(); i++) {
263     Rect swatch = {swatchStartX + (int16_t)i * (swatchSize + swatchGap), swatchY, swatchSize, swatchSize};
264     if (swatch.contains(x, y)) {
265         editingPlayerColor = playerColorOption(i);
266         markDisplayDirty();
267         return;
268     }
269 }
270
271 for (uint8_t r = 0; r < 3; r++) {
272     uint8_t count = strlen(rows[r]);
273     int16_t startX = (PANEL_WIDTH - ((count * keyW) + ((count - 1) * gap))) / 2;
274     for (uint8_t i = 0; i < count; i++) {
275         Rect key = {startX + (int16_t)i * (keyW + gap), rowY[r], keyW, keyH};
276         if (key.contains(x, y) && editingName.length() < PLAYER_NAME_MAX_LEN) {
277             editingName += rows[r][i];
278             markDisplayDirty();
279             return;
280         }
281     }
282 }
283 }
284
285 uint16_t colorForPlayer(Player player) {
286     if (player == PLAYER_A) {
287         return playerA_color;
288     }
289     if (player == PLAYER_B) {
290         return playerB_color;
291     }
292     return TFT_PANEL;
293 }

```

```

294
295 uint16_t textColorForPlayerColor(uint16_t color) {
296     return (color == PLAYER_COLOR_WHITE || color == PLAYER_COLOR_YELLOW) ? TFT_TEXT_DARK : TFT_TEXT;
297 }
298
299 uint16_t playerColorOption(uint8_t index) {
300     switch (index) {
301         case 0:
302             return PLAYER_COLOR_RED;
303         case 1:
304             return PLAYER_COLOR_BLUE;
305         case 2:
306             return PLAYER_COLOR_GREEN;
307         case 3:
308             return PLAYER_COLOR_YELLOW;
309         case 4:
310             return PLAYER_COLOR_GRAY;
311         case 5:
312             return PLAYER_COLOR_WHITE;
313         default:
314             return PLAYER_COLOR_BLUE;
315     }
316 }
317
318 uint8_t playerColorOptionCount() {
319     return 6;
320 }
321
322 void drawTimerPanels() {
323     if (mode != MODE_MATCH) {
324         return;
325     }
326
327     drawStatusBar();
328     drawTimerPanel(PLAYER_A, REGION_TIMER_A);
329     drawTimerPanel(PLAYER_B, REGION_TIMER_B);
330 }
331
332 void drawTimerPanel(Player player, const Rect &region) {
333     bool isA = (player == PLAYER_A);
334     bool active = (active_player == player && clock_running);
335     uint16_t accent = isA ? TFT_ACCENT_A : TFT_ACCENT_B;
336     uint16_t panelColor = active ? ILI9341_WHITE : TFT_PANEL;
337     uint32_t remaining = visibleReserveMs(player, millis());
338     String value = formatTimeMs(remaining);
339     uint16_t valueColor = active ? TFT_ACTIVE_TEXT : TFT_TEXT;
340     bool showingDelay = false;
341
342     if (active && timingMode == SIMPLE_DELAY && delay_seconds > 0) {
343         uint32_t elapsedMs = millis() - turnStartMs;
344         uint32_t delayMs = (uint32_t)delay_seconds * 1000UL;
345         if (elapsedMs < delayMs) {
346             uint32_t delayRemaining = (delayMs - elapsedMs + 999UL) / 1000UL;
347             value = String(delayRemaining);
348             valueColor = active ? TFT_ACTIVE_TEXT : TFT_WARN;
349             showingDelay = true;
350         }
351     }
352
353     tft.fillRect(region.x, region.y, region.w, region.h, panelColor);
354     tft.drawRoundRect(region.x, region.y, region.w, region.h, 6, accent);
355
356     #if BGMCMC_HAS_CLOCK_FONT
357     tft.setFont(&FreeMonoBold24pt7b);
358     tft.setTextSize(1);
359     tft.setTextColor(valueColor, panelColor);
360     int16_t x1;
361     int16_t y1;
362     uint16_t w;
363     uint16_t h;
364     tft.getTextBounds(value, 0, 0, &x1, &y1, &w, &h);
365     int16_t valueX = region.x + ((region.w - (int16_t)w) / 2) - x1;
366     int16_t valueY = region.y + ((region.h - (int16_t)h) / 2) - y1;
367     tft.setCursor(valueX, valueY);
368     tft.print(value);
369     tft.setFont();
370 #else
371     tft.setTextSize(showingDelay ? 6 : 4);
372     tft.setTextColor(valueColor, panelColor);
373     int16_t valueX = showingDelay ? (region.x + 104) : (region.x + 48);
374     if (!showingDelay && remaining >= 10UL * 60UL * 1000UL) {
375         valueX = region.x + 36;
376     }
377     tft.setCursor(valueX, showingDelay ? (region.y + 30) : (region.y + 38));
378     tft.print(value);
379 #endif

```

```

380 }
381
382 void drawPlayerPanel(Player player, int16_t x, int16_t y, int16_t w, int16_t h) {
383     bool isA = (player == PLAYER_A);
384     bool active = (active_player == player && clock_running);
385     uint16_t accent = isA ? TFT_ACCENT_A : TFT_ACCENT_B;
386     uint16_t border = active ? TFT_OK : accent;
387     uint32_t remaining = isA ? playerA_time_remaining : playerB_time_remaining;
388     int score = isA ? playerA_score : playerB_score;
389
390     tft.fillRect(x, y, w, h, TFT_PANEL);
391     tft.drawRoundRect(x, y, w, h, 6, border);
392     tft.drawRoundRect(x + 3, y + 3, w - 6, h - 6, 6, TFT_PANEL_ALT);
393     if (active) {
394         tft.fillRect(x + 8, y + 8, w - 16, 10, TFT_OK);
395     }
396
397     tft.setTextColor(accent, TFT_PANEL);
398     tft.setTextSize(3);
399     tft.setCursor(x + 24, y + 34);
400     tft.print(isA ? "PLAYER A" : "PLAYER B");
401
402     tft.setTextColor((remaining <= LOW_TIME_WARNING_SECONDS) ? TFT_DANGER : TFT_TEXT, TFT_PANEL);
403     tft.setTextSize(6);
404     tft.setCursor(x + 24, y + 98);
405     tft.print(formatTimeMs(remaining));
406
407     tft.fillRect(x + 24, y + 188, 150, 34, active ? TFT_OK : TFT_PANEL_ALT);
408     tft.setTextColor(active ? ILI9341_BLACK : TFT_TEXT, active ? TFT_OK : TFT_PANEL_ALT);
409     tft.setTextSize(2);
410     tft.setCursor(x + 40, y + 198);
411     tft.printf("SCORE %d", score);
412
413     tft.setTextColor(TFT_MUTED, TFT_PANEL);
414     tft.setTextSize(1);
415     tft.setCursor(x + 212, y + 204);
416     tft.print(active ? "RUNNING" : "WAITING");
417 }
418
419 void drawStatusBar() {
420     tft.fillRect(REGION_STATUS.x, REGION_STATUS.y, REGION_STATUS.w, REGION_STATUS.h, TFT_PANEL);
421     tft.fillRect(0, REGION_STATUS.h - 1, tft.width(), 1, TFT_BORDER);
422     tft.setTextColor(TFT_TEXT, TFT_PANEL);
423     tft.setTextSize(2);
424     tft.setCursor(18, 10);
425     tft.print("BATTERY");
426     tft.setCursor(114, 10);
427     int pct = batteryPercent();
428     if (pct < 0) {
429         tft.print("--%");
430     } else {
431         tft.printf("%d%%", pct);
432     }
433     if (chargingIndicatorOn()) {
434         drawChargeBolt(168, 6, TFT_WARN, TFT_PANEL);
435     }
436
437     tft.setTextColor(TFT_MUTED, TFT_PANEL);
438     tft.setCursor(300, 10);
439     tft.print("ELAPSED");
440     tft.setTextColor(TFT_TEXT, TFT_PANEL);
441     tft.setCursor(410, 10);
442     tft.print(formatTime(match_elapsed_seconds));
443
444     tft.setTextColor(TFT_TEXT, TFT_PANEL);
445     tft.setTextSize(2);
446     tft.setCursor(tft.width() - 168, 10);
447     tft.print(cachedRtcStatus);
448 }
449
450 void drawChargeBolt(int16_t x, int16_t y, uint16_t color, uint16_t background) {
451     tft.fillRect(x, y, 20, 24, background);
452     fillTriangleRows(x + 11, y + 3, x + 5, y + 12, x + 11, y + 12, color);
453     fillTriangleRows(x + 9, y + 10, x + 16, y + 10, x + 6, y + 21, color);
454 }

```

5.10 10. Splash screen and generated graphic helpers

What this segment does. Draws either a compiled splash image or a generated splash fallback using row-based triangle/circle fills.

Why this structure is used. The compiled image gives the product a polished startup screen. The generated fallback keeps the sketch usable even if the image header is missing or disabled.

Source lines 2945–3069.

```

1 void drawStartupTest() {
2     logLine("Drawing startup splash image");
3
4     setBrightness(255);
5     drawSplashImage();
6     delay(4000);
7 }
8
9 void drawSplashImage() {
10    if (SPLASH_IMAGE_SCALE == 2) {
11        tft.drawRGB565ImageScaled2x(0, 0, SPLASH_IMAGE_WIDTH, SPLASH_IMAGE_HEIGHT, SPLASH_IMAGE_RGB565);
12    } else {
13        tft.drawRGB565Image(0, 0, SPLASH_IMAGE_WIDTH, SPLASH_IMAGE_HEIGHT, SPLASH_IMAGE_RGB565);
14    }
15 }
16
17 void drawGeneratedSplashImage() {
18     const uint16_t board = 0x0841;
19     const uint16_t wood = 0x3922;
20     const uint16_t woodLite = 0x7A84;
21     const uint16_t gold = 0xDDA6;
22     const uint16_t amber = 0xB3A0;
23     const uint16_t cream = 0xDEB2;
24     const uint16_t orange = 0x8A80;
25     const uint16_t blackChecker = 0x10A2;
26     const uint16_t whiteChecker = 0xF6D8;
27     const uint16_t clockBg = 0x0000;
28     const uint16_t silver = 0xD6BA;
29
30     tft.fillScreen(board);
31     tft.fillRect(0, 0, 58, PANEL_HEIGHT, wood);
32     tft.fillRect(742, 0, 58, PANEL_HEIGHT, wood);
33     for (int16_t x = 8; x < 58; x += 12) {
34         tft.fillRect(x, 0, 2, PANEL_HEIGHT, woodLite);
35         tft.fillRect(742 + x, 0, 2, PANEL_HEIGHT, woodLite);
36     }
37
38     const int16_t pointTop = 0;
39     const int16_t pointBottom = PANEL_HEIGHT - 1;
40     const int16_t pointH = 220;
41     const int16_t leftPoints[] = {88, 140, 190};
42     const int16_t rightPoints[] = {610, 662, 714};
43     for (uint8_t i = 0; i < 3; i++) {
44         uint16_t color = (i == 1) ? orange : cream;
45         fillTriangleRows(leftPoints[i], pointTop, leftPoints[i] + 44, pointTop, leftPoints[i] + 22, pointTop + pointH, color);
46         fillTriangleRows(rightPoints[i], pointTop, rightPoints[i] + 44, pointTop, rightPoints[i] + 22, pointTop + pointH, color);
47         fillTriangleRows(leftPoints[i], pointBottom, leftPoints[i] + 44, pointBottom, leftPoints[i] + 22, pointBottom - pointH, color);
48         fillTriangleRows(rightPoints[i], pointBottom, rightPoints[i] + 44, pointBottom, rightPoints[i] + 22, pointBottom - pointH, color);
49     }
50
51     fillCircleRows(372, 120, 32, whiteChecker);
52     fillCircleRows(418, 128, 28, blackChecker);
53     fillCircleRows(372, 120, 22, 0xE5B1);
54     fillCircleRows(418, 128, 19, 0x2945);
55     tft.fillRect(296, 138, 78, 2, gold);
56     tft.fillRect(450, 138, 106, 2, gold);
57     tft.fillRect(314, 122, 36, 2, gold);
58     tft.fillRect(318, 110, 24, 2, gold);
59
60     drawCenteredInRect("BACKGAMMON", {110, 166, 580, 60}, 6, TFT_TEXT, board);
61     tft.fillRect(250, 236, 58, 2, gold);
62     tft.fillRect(492, 236, 58, 2, gold);
63     drawCenteredInRect("CLOCK", {298, 220, 204, 48}, 4, amber, board);
64
65     tft.fillRect(224, 288, 352, 78, clockBg);
66     tft.drawRoundRect(224, 288, 352, 78, 8, gold);
67     drawCenteredInRect("15:00 : 15:00", {242, 300, 316, 52}, 4, silver, clockBg);
68     tft.setTextColor(amber, board);
69     tft.setTextSize(2);
70     tft.setCursor(254, 390);
71     tft.print("FOCUS. STRATEGY. EVERY MOVE COUNTS.");
72     tft.setCursor(286, 434);
73     tft.print("PLAY FAIR. PLAY STRONG.");
74 }
75
76 void fillTriangleRows(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color) {
77     if (y1 < y0) {
78         int16_t tx = x0; x0 = x1; x1 = tx;
79         int16_t ty = y0; y0 = y1; y1 = ty;
80     }

```

```

81  if (y2 < y0) {
82      int16_t tx = x0; x0 = x2; x2 = tx;
83      int16_t ty = y0; y0 = y2; y2 = ty;
84  }
85  if (y2 < y1) {
86      int16_t tx = x1; x1 = x2; x2 = tx;
87      int16_t ty = y1; y1 = y2; y2 = ty;
88  }
89
90  int16_t totalHeight = y2 - y0;
91  if (totalHeight == 0) {
92      return;
93  }
94
95  for (int16_t y = y0; y <= y2; y++) {
96      bool secondHalf = y > y1 || y1 == y0;
97      int16_t segmentHeight = secondHalf ? (y2 - y1) : (y1 - y0);
98      if (segmentHeight == 0) {
99          continue;
100      }
101      float alpha = (float)(y - y0) / (float)totalHeight;
102      float beta = secondHalf ? (float)(y - y1) / (float)segmentHeight : (float)(y - y0) / (float)segmentHeight;
103      int16_t ax = x0 + (int16_t)((x2 - x0) * alpha);
104      int16_t bx = secondHalf ? x1 + (int16_t)((x2 - x1) * beta) : x0 + (int16_t)((x1 - x0) * beta);
105      if (ax > bx) {
106          int16_t tmp = ax;
107          ax = bx;
108          bx = tmp;
109      }
110      tft.fillRect(ax, y, bx - ax + 1, 1, color);
111  }
112 }
113
114 void fillCircleRows(int16_t cx, int16_t cy, int16_t radius, uint16_t color) {
115     int16_t r2 = radius * radius;
116     for (int16_t y = -radius; y <= radius; y++) {
117         int16_t x = 0;
118         while ((x + 1) * (x + 1) + y * y <= r2) {
119             x++;
120         }
121         tft.fillRect(cx - x, cy + y, (x * 2) + 1, 1, color);
122     }
123     tft.drawRoundRect(cx - radius, cy - radius, radius * 2, radius * 2, radius, TFT_BORDER);
124 }

```

5.11 11. I2C devices: expanders, fuel gauge, RTC, and touch controller

What this segment does. Initializes the CrowPanel PCA9557, optional PCF8575 button expander, MAX17048 fuel gauge, DS3231 RTC, I2C scanner, GT911 reset/init, and GT911 reads/writes.

Why this structure is used. This segment is intentionally tolerant. Missing accessories are logged but not fatal, so the same firmware can run on a minimal screen-only build or a fully wired build.

Source lines 3070–3510.

```

1  void initCrowPanelExpander() {
2      Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
3      Wire.setClock(400000);
4      scanI2C("before PCA9557 init");
5
6      uint8_t pcaAddress = 0;
7      for (uint8_t address = 0x18; address <= 0x1F; address++) {
8          Wire.beginTransmission(address);
9          if (Wire.endTransmission() == 0) {
10             pcaAddress = address;
11             break;
12         }
13     }
14
15     if (pcaAddress == 0) {
16         logLine("PCA9557 not found on I2C; continuing without expander init");
17         return;
18     }
19
20     logPrintf("PCA9557 found at 0x%02X\n", pcaAddress);
21     pca9557Address = pcaAddress;
22
23     // Elecrow V3 sequence: I00/I01 low, short delay, I00 high, I01 input.
24     // Registers: 1=output port, 2=polarity, 3=configuration (1=input, 0=output).
25     bool ok = true;

```

```

26 ok &= pca9557Write(pcaAddress, 0x02, 0x00);
27 ok &= pca9557Write(pcaAddress, 0x03, 0x00);
28 ok &= pca9557Write(pcaAddress, 0x01, 0x00);
29 delay(20);
30 ok &= pca9557Write(pcaAddress, 0x01, 0x01);
31 delay(100);
32 ok &= pca9557Write(pcaAddress, 0x03, 0x02);
33
34 logPrintf("PCA9557 init: %s\n", ok ? "OK" : "FAILED");
35 scanI2C("after PCA9557 init");
36 }
37
38 bool pca9557Write(uint8_t address, uint8_t reg, uint8_t value) {
39     Wire.beginTransmission(address);
40     Wire.write(reg);
41     Wire.write(value);
42     return Wire.endTransmission() == 0;
43 }
44
45 uint8_t pca9557Read(uint8_t address, uint8_t reg) {
46     Wire.beginTransmission(address);
47     Wire.write(reg);
48     if (Wire.endTransmission(false) != 0) {
49         return 0xFF;
50     }
51     if (Wire.requestFrom((int)address, 1) != 1) {
52         return 0xFF;
53     }
54     return Wire.read();
55 }
56
57 void initButtonExpander() {
58     uint8_t foundAddress = 0;
59     for (uint8_t address = 0x20; address <= 0x27; address++) {
60         Wire.beginTransmission(address);
61         if (Wire.endTransmission() == 0) {
62             foundAddress = address;
63             break;
64         }
65     }
66
67     if (foundAddress == 0) {
68         logLine("PCF8575 button expander not found; physical I2C buttons disabled.");
69         return;
70     }
71
72     pcf8575Address = foundAddress;
73     bool ok = pcf8575Write(pcf8575Address, 0xFFFF);
74     logPrintf("PCF8575 button expander found at 0x%02X: %s\n", pcf8575Address, ok ? "OK" : "FAILED");
75     logPrintf("PCF8575 buttons: Play/Pause=P%02u Player A=P%02u Player B=P%02u\n",
76             PCF8575_PLAY_PAUSE_PIN, PCF8575_PLAYER_A_PIN, PCF8575_PLAYER_B_PIN);
77     scanI2C("after PCF8575 init");
78 }
79
80 bool pcf8575Write(uint8_t address, uint16_t value) {
81     Wire.beginTransmission(address);
82     Wire.write(value & 0xFF);
83     Wire.write((value >> 8) & 0xFF);
84     return Wire.endTransmission() == 0;
85 }
86
87 uint16_t pcf8575Read(uint8_t address) {
88     if (Wire.requestFrom((int)address, 2) != 2) {
89         return 0xFFFF;
90     }
91     uint8_t low = Wire.read();
92     uint8_t high = Wire.read();
93     return (uint16_t)low | ((uint16_t)high << 8);
94 }
95
96 void initFuelGauge() {
97     Wire.beginTransmission(MAX17048_I2C_ADDRESS);
98     if (Wire.endTransmission() != 0) {
99         max17048Found = false;
100         logLine("MAX17048 fuel gauge not found; battery display will use fallback.");
101         return;
102     }
103
104     uint16_t version = 0;
105     max17048Found = true;
106     max17048Read16(0x08, &version);
107     logPrintf("MAX17048 fuel gauge found at 0x%02X version=0x%04X\n", MAX17048_I2C_ADDRESS, version);
108     logPrintf("MAX17048 battery: %.2fV %.1f%%\n", max17048CellVoltage(), max17048StateOfCharge());
109     scanI2C("after MAX17048 init");
110 }
111

```

```

112 bool max17048Read16(uint8_t reg, uint16_t *value) {
113     if (value == nullptr) {
114         return false;
115     }
116
117     Wire.beginTransmission(MAX17048_I2C_ADDRESS);
118     Wire.write(reg);
119     if (Wire.endTransmission(false) != 0) {
120         return false;
121     }
122
123     if (Wire.requestFrom((int)MAX17048_I2C_ADDRESS, 2) != 2) {
124         return false;
125     }
126
127     uint8_t msb = Wire.read();
128     uint8_t lsb = Wire.read();
129     *value = ((uint16_t)msb << 8) | lsb;
130     return true;
131 }
132
133 float max17048CellVoltage() {
134     uint16_t raw = 0;
135     if (!max17048Found || !max17048Read16(0x02, &raw)) {
136         return 0.0f;
137     }
138     return (float)(raw >> 4) * 0.00125f;
139 }
140
141 float max17048StateOfCharge() {
142     uint16_t raw = 0;
143     if (!max17048Found || !max17048Read16(0x04, &raw)) {
144         return 0.0f;
145     }
146     float pct = (float)(raw >> 8) + ((float)(raw & 0xFF) / 256.0f);
147     if (pct < 0.0f) {
148         pct = 0.0f;
149     }
150     if (pct > 100.0f) {
151         pct = 100.0f;
152     }
153     return pct;
154 }
155
156 void initRtcClock() {
157     Wire.beginTransmission(DS3231_I2C_ADDRESS);
158     if (Wire.endTransmission() != 0) {
159         ds3231Found = false;
160         logLine("DS3231 RTC not found; match log dates will be DATE UNSET.");
161         return;
162     }
163
164     ds3231Found = true;
165     uint8_t second = 0;
166     uint8_t minute = 0;
167     uint8_t hour = 0;
168     uint8_t day = 0;
169     uint8_t month = 0;
170     uint16_t year = 0;
171     if (ds3231ReadTime(&second, &minute, &hour, &day, &month, &year)) {
172         if (rtcDateLooksSet(day, month, year)) {
173             logPrintf("DS3231 RTC found at 0x%02X date=%04u-%02u-%02u time=%02u:%02u:%02u\n",
174                 DS3231_I2C_ADDRESS, year, month, day, hour, minute, second);
175         } else {
176             logPrintf("DS3231 RTC found at 0x%02X but date looks unset: %04u-%02u-%02u\n",
177                 DS3231_I2C_ADDRESS, year, month, day);
178         }
179     } else {
180         logPrintf("DS3231 RTC found at 0x%02X but time read failed.\n", DS3231_I2C_ADDRESS);
181     }
182     scanI2C("after DS3231 init");
183 }
184
185 bool ds3231ReadTime(uint8_t *second, uint8_t *minute, uint8_t *hour, uint8_t *day, uint8_t *month, uint16_t *year) {
186     if (second == nullptr || minute == nullptr || hour == nullptr || day == nullptr || month == nullptr || year == nullptr) {
187         return false;
188     }
189
190     Wire.beginTransmission(DS3231_I2C_ADDRESS);
191     Wire.write(0x00);
192     if (Wire.endTransmission(false) != 0) {
193         return false;
194     }
195
196     if (Wire.requestFrom((int)DS3231_I2C_ADDRESS, 7) != 7) {
197         return false;

```

```

198 }
199
200 uint8_t rawSecond = Wire.read();
201 uint8_t rawMinute = Wire.read();
202 uint8_t rawHour = Wire.read();
203 Wire.read(); // Day of week, unused for match logs.
204 uint8_t rawDay = Wire.read();
205 uint8_t rawMonth = Wire.read();
206 uint8_t rawYear = Wire.read();
207
208 *second = bcdToDec(rawSecond & 0x7F);
209 *minute = bcdToDec(rawMinute & 0x7F);
210 if ((rawHour & 0x40) != 0) {
211     uint8_t hour12 = bcdToDec(rawHour & 0x1F);
212     bool pm = (rawHour & 0x20) != 0;
213     *hour = (hour12 % 12) + (pm ? 12 : 0);
214 } else {
215     *hour = bcdToDec(rawHour & 0x3F);
216 }
217 *day = bcdToDec(rawDay & 0x3F);
218 *month = bcdToDec(rawMonth & 0x1F);
219 *year = 2000 + bcdToDec(rawYear);
220 return true;
221 }
222
223 bool ds3231SetTime(uint8_t second, uint8_t minute, uint8_t hour, uint8_t day, uint8_t month, uint16_t year) {
224     if (!ds3231Found || !rtcDateLooksSet(day, month, year) || hour > 23 || minute > 59 || second > 59) {
225         return false;
226     }
227
228     Wire.beginTransaction(DS3231_I2C_ADDRESS);
229     Wire.write(0x00);
230     Wire.write(decToBcd(second));
231     Wire.write(decToBcd(minute));
232     Wire.write(decToBcd(hour)); // 24-hour mode.
233     Wire.write(decToBcd(1)); // Day of week is not used by this firmware.
234     Wire.write(decToBcd(day));
235     Wire.write(decToBcd(month));
236     Wire.write(decToBcd((uint8_t)(year - 2000)));
237     return Wire.endTransmission() == 0;
238 }
239
240 bool rtcDateLooksSet(uint8_t day, uint8_t month, uint16_t year) {
241     return year >= 2024 && year <= 2099 && month >= 1 && month <= 12 && day >= 1 && day <= 31;
242 }
243
244 uint8_t daysInMonth(uint8_t month, uint16_t year) {
245     switch (month) {
246     case 4:
247     case 6:
248     case 9:
249     case 11:
250         return 30;
251     case 2: {
252         bool leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
253         return leap ? 29 : 28;
254     }
255     default:
256         return 31;
257     }
258 }
259
260 uint8_t bcdToDec(uint8_t value) {
261     return ((value >> 4) * 10) + (value & 0x0F);
262 }
263
264 uint8_t decToBcd(uint8_t value) {
265     return ((value / 10) << 4) | (value % 10);
266 }
267
268 String rtcStatusText() {
269     uint8_t second = 0;
270     uint8_t minute = 0;
271     uint8_t hour = 0;
272     uint8_t day = 0;
273     uint8_t month = 0;
274     uint16_t year = 0;
275     if (!ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) || !rtcDateLooksSet(day, month, year)) {
276         return "DATE UNSET";
277     }
278
279     char buffer[18];
280     snprintf(buffer, sizeof(buffer), "%02u/%02u %02u:%02u", month, day, hour, minute);
281     return String(buffer);
282 }
283

```

```

284 bool updateRtcStatusCache(bool force) {
285     uint32_t nowMs = millis();
286     if (!force && nowMs - lastRtcStatusMs < 1000UL) {
287         return false;
288     }
289
290     String next = rtcStatusText();
291     bool changed = force || next != cachedRtcStatus;
292     cachedRtcStatus = next;
293     lastRtcStatusMs = nowMs;
294     return changed;
295 }
296
297 void scanI2C(const char *label) {
298     logPrintf("I2C scan %s:", label);
299     bool any = false;
300     for (uint8_t address = 1; address < 127; address++) {
301         Wire.beginTransmission(address);
302         if (Wire.endTransmission() == 0) {
303             logPrintf(" 0x%02X", address);
304             any = true;
305         }
306     }
307     logLine(any ? "" : " none");
308 }
309
310 void resetTouchController(uint8_t outputValue, uint8_t configValue) {
311     if (pca9557Address == 0) {
312         return;
313     }
314     pca9557Write(pca9557Address, 0x03, 0x00);
315     pca9557Write(pca9557Address, 0x01, outputValue);
316     delay(20);
317     pca9557Write(pca9557Address, 0x03, configValue);
318     delay(80);
319 }
320
321 void initTouch() {
322     scanI2C("before GT911 init");
323
324     const uint8_t addresses[] = {0x5D, 0x14};
325     for (uint8_t i = 0; i < sizeof(addresses); i++) {
326         Wire.beginTransmission(addresses[i]);
327         if (Wire.endTransmission() == 0) {
328             gt911Address = addresses[i];
329             break;
330         }
331     }
332
333     if (gt911Address == 0) {
334         const uint8_t resetOutputs[] = {0x01, 0x00, 0x03, 0x02};
335         const uint8_t resetConfigs[] = {0x02, 0x02, 0x00, 0x00};
336         for (uint8_t attempt = 0; attempt < 4 && gt911Address == 0; attempt++) {
337             resetTouchController(resetOutputs[attempt], resetConfigs[attempt]);
338             scanI2C("after touch reset attempt");
339             for (uint8_t i = 0; i < sizeof(addresses); i++) {
340                 Wire.beginTransmission(addresses[i]);
341                 if (Wire.endTransmission() == 0) {
342                     gt911Address = addresses[i];
343                     break;
344                 }
345             }
346         }
347     }
348
349     if (gt911Address == 0) {
350         logLine("GT911 touch controller not found. Touch disabled.");
351         return;
352     }
353
354     uint8_t productId[4] = {};
355     gt911Read(0x8140, productId, sizeof(productId));
356     logPrintf("GT911 found at 0x%02X id=%c%c%c%c\n", gt911Address,
357             productId[0], productId[1], productId[2], productId[3]);
358     gt911Write(0x814E, 0x00);
359 }
360
361 TouchPoint readTouch() {
362     TouchPoint point = {false, 0, 0};
363     if (gt911Address == 0) {
364         return point;
365     }
366
367     uint8_t status = 0;
368     if (!gt911Read(0x814E, &status, 1)) {
369         return point;

```

```

370 }
371
372 uint8_t count = status & 0x0F;
373 if (count == 0) {
374     if (status & 0x80) {
375         gt911Write(0x814E, 0x00);
376     }
377     return point;
378 }
379 if (count > 5) {
380     gt911Write(0x814E, 0x00);
381     return point;
382 }
383
384 uint8_t data[7] = {};
385 if (gt911Read(0x8150, data, sizeof(data))) {
386     uint16_t xLowHigh = ((uint16_t)data[1] << 8) | data[0];
387     uint16_t yLowHigh = ((uint16_t)data[3] << 8) | data[2];
388     uint16_t xHighLow = ((uint16_t)data[0] << 8) | data[1];
389     uint16_t yHighLow = ((uint16_t)data[2] << 8) | data[3];
390     bool lowHighValid = xLowHigh < PANEL_WIDTH && yLowHigh < PANEL_HEIGHT;
391     bool highLowValid = xHighLow < PANEL_WIDTH && yHighLow < PANEL_HEIGHT;
392
393     if (lowHighValid || highLowValid) {
394         point.touched = true;
395         point.x = lowHighValid ? xLowHigh : xHighLow;
396         point.y = lowHighValid ? yLowHigh : yHighLow;
397     } else {
398         uint16_t shiftedXLowHigh = ((uint16_t)data[2] << 8) | data[1];
399         uint16_t shiftedYLowHigh = ((uint16_t)data[4] << 8) | data[3];
400         uint16_t shiftedXHighLow = ((uint16_t)data[1] << 8) | data[2];
401         uint16_t shiftedYHighLow = ((uint16_t)data[3] << 8) | data[4];
402         bool shiftedLowHighValid = shiftedXLowHigh < PANEL_WIDTH && shiftedYLowHigh < PANEL_HEIGHT;
403         bool shiftedHighLowValid = shiftedXHighLow < PANEL_WIDTH && shiftedYHighLow < PANEL_HEIGHT;
404
405         if (shiftedLowHighValid || shiftedHighLowValid) {
406             point.touched = true;
407             point.x = shiftedLowHighValid ? shiftedXLowHigh : shiftedXHighLow;
408             point.y = shiftedLowHighValid ? shiftedYLowHigh : shiftedYHighLow;
409         }
410     }
411 }
412
413 gt911Write(0x814E, 0x00);
414 return point;
415 }
416
417 bool gt911Read(uint16_t reg, uint8_t *data, uint8_t len) {
418     Wire.beginTransmission(gt911Address);
419     Wire.write(reg >> 8);
420     Wire.write(reg & 0xFF);
421     if (Wire.endTransmission(false) != 0) {
422         return false;
423     }
424     uint8_t received = Wire.requestFrom((int)gt911Address, (int)len);
425     if (received != len) {
426         return false;
427     }
428
429     for (uint8_t i = 0; i < len; i++) {
430         data[i] = Wire.read();
431     }
432     return true;
433 }
434
435 bool gt911Write(uint16_t reg, uint8_t value) {
436     Wire.beginTransmission(gt911Address);
437     Wire.write(reg >> 8);
438     Wire.write(reg & 0xFF);
439     Wire.write(value);
440     return Wire.endTransmission() == 0;
441 }

```

5.12 12. Formatting, custom font drawing, battery display, logs, settings, sleep, and sound

What this segment does. Includes debug logging, centered text helpers, score/match font renderers, GFXfont scaling, time formatting, battery/charging indicators, match log persistence, saved settings, brightness, sleep, and buzzer hooks.

Why this structure is used. The final helper layer is where many reusable support functions live. It also contains the font replacement points: timer font path, UI text path, score font path, and match target font path.

Source lines 3511–4104.

```

1 void logLine(const char *message) {
2     Serial.println(message);
3     Serial0.println(message);
4 }
5
6 void logPrintf(const char *format, ...) {
7     char buffer[160];
8     va_list args;
9     va_start(args, format);
10    vsnprintf(buffer, sizeof(buffer), format, args);
11    va_end(args);
12
13    Serial.print(buffer);
14    Serial0.print(buffer);
15 }
16
17 void drawCentered(const String &text, int16_t y, uint8_t textSize, uint16_t color) {
18     tft.setTextSize(textSize);
19     tft.setTextColor(color, TFT_BACKGROUND);
20     int16_t x1;
21     int16_t y1;
22     uint16_t w;
23     uint16_t h;
24     tft.getTextBounds(text, 0, y, &x1, &y1, &w, &h);
25     tft.setCursor((tft.width() - w) / 2, y);
26     tft.print(text);
27 }
28
29 void drawCenteredInRect(const String &text, const Rect &region, uint8_t textSize, uint16_t color, uint16_t background) {
30     #if BGMC_HAS_SMALL_UI_FONT
31     bool useUIFont = (textSize >= 2 && textSize <= 3);
32     if (useUIFont) {
33         tft.setFont(&FreeSans12pt7b);
34         tft.setTextSize(1);
35     } else {
36         tft.setFont();
37         tft.setTextSize(textSize);
38     }
39     #elif BGMC_HAS_UI_FONT
40     bool useUIFont = (textSize >= 2 && textSize <= 3);
41     if (useUIFont) {
42         tft.setFont(&FreeSans18pt7b);
43         tft.setTextSize(1);
44     } else {
45         tft.setFont();
46         tft.setTextSize(textSize);
47     }
48     #else
49     tft.setFont();
50     tft.setTextSize(textSize);
51     #endif
52     tft.setTextColor(color, background);
53     int16_t x1;
54     int16_t y1;
55     uint16_t w;
56     uint16_t h;
57     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
58     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
59     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
60     tft.setCursor(x, y);
61     tft.print(text);
62     tft.setFont();
63 }
64
65 void drawScoreInRect(int score, const Rect &region, uint16_t color, uint16_t background) {
66     String text = String(score);
67     #if BGMC_HAS_COLLEGE_SCORE_FONT
68     tft.setFont(&collegeb96pt7b);
69     tft.setTextSize(1);
70     #elif BGMC_HAS_SCORE_96_FONT
71     tft.setFont(&FreeSansBold96pt7b);
72     tft.setTextSize(1);
73     #elif BGMC_HAS_SCORE_FONT
74     tft.setFont(&FreeSansBold24pt7b);
75     tft.setTextSize(4);
76     #elif BGMC_HAS_UI_FONT
77     tft.setFont(&FreeSans18pt7b);
78     tft.setTextSize(3);
79     #else
80 
```

```

81     tft.setFont();
82     tft.setTextSize(10);
83 #endif
84     tft.setTextColor(color, background);
85     int16_t x1;
86     int16_t y1;
87     uint16_t w;
88     uint16_t h;
89     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
90 #if BGMC_HAS_COLLEGE_SCORE_FONT || BGMC_HAS_SCORE_96_FONT
91     if (w > region.w - 28 || h > region.h - 28) {
92 #if BGMC_HAS_SCORE_FONT
93         tft.setFont(&FreeSansBold24pt7b);
94         tft.setTextSize(4);
95 #else
96         tft.setFont();
97         tft.setTextSize(10);
98 #endif
99         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
100     }
101 #endif
102 #if BGMC_HAS_SCORE_FONT
103     uint8_t scoreSize = 4;
104     while (scoreSize > 1 && (w > region.w - 28 || h > region.h - 28)) {
105         scoreSize--;
106         tft.setTextSize(scoreSize);
107         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
108     }
109 #endif
110     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
111     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
112     tft.setCursor(x, y);
113     tft.print(text);
114     tft.setFont();
115 }
116
117 void drawMatchTargetInRect(int target, const Rect &region, uint16_t color, uint16_t background) {
118     String text = String(target);
119 #if BGMC_HAS_MATCH_TARGET_48_FONT
120     {
121         tft.setFont(&FreeSansBold48pt7b);
122         tft.setTextSize(1);
123         tft.setTextColor(color, background);
124         int16_t x1;
125         int16_t y1;
126         uint16_t w;
127         uint16_t h;
128         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
129         int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
130         int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
131         tft.setCursor(x, y);
132         tft.print(text);
133         tft.setFont();
134     }
135     return;
136 #elif BGMC_HAS_SCORE_FONT
137     (void)background;
138     drawGfxTextScaledInRect(&FreeSansBold24pt7b, text, region, color, 3, 2);
139     return;
140 #elif BGMC_HAS_SMALL_UI_FONT
141     tft.setFont(&FreeSans12pt7b);
142     tft.setTextSize(3);
143 #elif BGMC_HAS_UI_FONT
144     tft.setFont(&FreeSans18pt7b);
145     tft.setTextSize(2);
146 #else
147     tft.setFont();
148     tft.setTextSize(7);
149 #endif
150     tft.setTextColor(color, background);
151     int16_t x1;
152     int16_t y1;
153     uint16_t w;
154     uint16_t h;
155     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
156 #if BGMC_HAS_SMALL_UI_FONT
157     if (w > region.w - 24 || h > region.h - 8) {
158         tft.setTextSize(2);
159         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
160     }
161 #endif
162     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
163     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
164     tft.setCursor(x, y);
165     tft.print(text);
166     tft.setFont();

```

```

167 }
168
169 void drawGfxTextScaledInRect(const GFXfont *font, const String &text, const Rect &region, uint16_t color,
170                             uint8_t scaleNum, uint8_t scaleDen) {
171     int16_t x1;
172     int16_t y1;
173     uint16_t w;
174     uint16_t h;
175     getGfxTextBoundsScaled(font, text, scaleNum, scaleDen, &x1, &y1, &w, &h);
176
177     if (w > region.w - 24 || h > region.h - 8) {
178         scaleNum = 1;
179         scaleDen = 1;
180         getGfxTextBoundsScaled(font, text, scaleNum, scaleDen, &x1, &y1, &w, &h);
181     }
182
183     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
184     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
185     drawGfxTextScaled(font, text, x, y, color, scaleNum, scaleDen);
186 }
187
188 void getGfxTextBoundsScaled(const GFXfont *font, const String &text, uint8_t scaleNum, uint8_t scaleDen,
189                             int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
190     int16_t minX = 32767;
191     int16_t minY = 32767;
192     int16_t maxX = -32768;
193     int16_t maxY = -32768;
194     int16_t cursor = 0;
195     uint16_t first = pgm_read_word(&font->first);
196     uint16_t last = pgm_read_word(&font->last);
197     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&font->glyph);
198
199     for (uint16_t i = 0; i < text.length(); i++) {
200         char c = text.charAt(i);
201         if (c < first || c > last) {
202             continue;
203         }
204         GFXglyph glyph;
205         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
206         if (glyph.width > 0 && glyph.height > 0) {
207             int16_t gx1 = cursor + ((int32_t)glyph.xOffset * scaleNum) / scaleDen;
208             int16_t gy1 = ((int32_t)glyph.yOffset * scaleNum) / scaleDen;
209             int16_t gx2 = cursor + ((int32_t)(glyph.xOffset + glyph.width) * scaleNum + scaleDen - 1) / scaleDen - 1;
210             int16_t gy2 = ((int32_t)(glyph.yOffset + glyph.height) * scaleNum + scaleDen - 1) / scaleDen - 1;
211             if (gx1 < minX) minX = gx1;
212             if (gy1 < minY) minY = gy1;
213             if (gx2 > maxX) maxX = gx2;
214             if (gy2 > maxY) maxY = gy2;
215         }
216         cursor += ((int32_t)glyph.xAdvance * scaleNum + scaleDen - 1) / scaleDen;
217     }
218
219     if (maxX < minX || maxY < minY) {
220         *x1 = 0;
221         *y1 = 0;
222         *w = 0;
223         *h = 0;
224         return;
225     }
226     *x1 = minX;
227     *y1 = minY;
228     *w = maxX - minX + 1;
229     *h = maxY - minY + 1;
230 }
231
232 void drawGfxTextScaled(const GFXfont *font, const String &text, int16_t x, int16_t y, uint16_t color,
233                       uint8_t scaleNum, uint8_t scaleDen) {
234     int16_t cursor = x;
235     uint16_t first = pgm_read_word(&font->first);
236     uint16_t last = pgm_read_word(&font->last);
237     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&font->glyph);
238     const uint8_t *bitmap = (const uint8_t *)pgm_read_ptr(&font->bitmap);
239
240     for (uint16_t i = 0; i < text.length(); i++) {
241         char c = text.charAt(i);
242         if (c < first || c > last) {
243             continue;
244         }
245
246         GFXglyph glyph;
247         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
248         uint16_t bitIndex = 0;
249         uint16_t bitmapOffset = glyph.bitmapOffset;
250         uint8_t bits = 0;
251
252         for (uint8_t yy = 0; yy < glyph.height; yy++) {

```

```

253     int16_t rowTop = y + ((int32_t)(glyph.yOffset + yy) * scaleNum) / scaleDen;
254     int16_t rowBottom = y + ((int32_t)(glyph.yOffset + yy + 1) * scaleNum + scaleDen - 1) / scaleDen;
255     int16_t rowH = max((int16_t)1, (int16_t)(rowBottom - rowTop));
256
257     for (uint8_t xx = 0; xx < glyph.width; xx++) {
258         if ((bitIndex & 7) == 0) {
259             bits = pgm_read_byte(&bitmap[bitmapOffset++]);
260         }
261         if (bits & 0x80) {
262             int16_t colLeft = cursor + ((int32_t)(glyph.xOffset + xx) * scaleNum) / scaleDen;
263             int16_t colRight = cursor + ((int32_t)(glyph.xOffset + xx + 1) * scaleNum + scaleDen - 1) / scaleDen;
264             int16_t colW = max((int16_t)1, (int16_t)(colRight - colLeft));
265             tft.fillRect(colLeft, rowTop, colW, rowH, color);
266         }
267         bits <<= 1;
268         bitIndex++;
269     }
270     cursor += ((int32_t)glyph.xAdvance * scaleNum + scaleDen - 1) / scaleDen;
271 }
272 }
273 }
274
275 String formatTime(uint32_t seconds) {
276     uint32_t minutes = seconds / 60;
277     uint32_t secs = seconds % 60;
278     char buffer[12];
279     snprintf(buffer, sizeof(buffer), "%02lu:%02lu", (unsigned long)minutes, (unsigned long)secs);
280     return String(buffer);
281 }
282
283 String formatTimeMs(uint32_t milliseconds) {
284     uint32_t seconds = (milliseconds + 999UL) / 1000UL;
285     return formatTime(seconds);
286 }
287
288 String timingModeLabel() {
289     switch (timingMode) {
290         case BRONSTEIN_DELAY:
291             return "Bronstein";
292         case FISCHER_INCREMENT:
293             return "Fischer";
294         case SIMPLE_DELAY:
295             default:
296                 return "Simple";
297     }
298 }
299
300 String playPauseLabel() {
301     if (clock_running) {
302         return "Pause Game";
303     }
304     if (game_number == 0 && !game_in_progress) {
305         return "Start Match";
306     }
307     return "Play Game";
308 }
309
310 String matchStateLabel() {
311     if (crawford_active) {
312         return "CRAWFORD GAME - CUBE OFF";
313     }
314     if (post_crawford) {
315         return "POST-CRAWFORD - CUBE ON";
316     }
317     return "NORMAL";
318 }
319
320 int batteryPercent() {
321     if (max17048Found) {
322         return (int)(max17048StateOfCharge() + 0.5f);
323     }
324
325     if (PIN_BATTERY_ADC == 255) {
326         return -1;
327     }
328
329     int raw = analogRead(PIN_BATTERY_ADC);
330     float adcVolts = ((float)raw / 4095.0f) * BATTERY_ADC_REF_VOLTS;
331     float batteryVolts = adcVolts * BATTERY_DIVIDER_RATIO;
332     float pct = (batteryVolts - BATTERY_EMPTY_VOLTS) * 100.0f /
333                (BATTERY_FULL_VOLTS - BATTERY_EMPTY_VOLTS);
334     if (pct < 0.0f) {
335         pct = 0.0f;
336     }
337     if (pct > 100.0f) {
338         pct = 100.0f;
339     }

```

```

339 }
340 return (int)(pct + 0.5f);
341 }
342
343 bool chargingIndicatorOn() {
344     if (PIN_CHARGE_DETECT == 255) {
345         if (!max17048Found) {
346             return false;
347         }
348
349         uint32_t nowMs = millis();
350         if (lastChargeCheckMs == 0 || nowMs - lastChargeCheckMs >= 30000UL) {
351             float voltage = max17048CellVoltage();
352             float percent = max17048StateOfCharge();
353             if (lastChargeCheckMs != 0) {
354                 chargeTrendRising = (voltage > lastChargeCheckVoltage + 0.006f) ||
355                                     (percent > lastChargeCheckPercent + 0.08f);
356                 if (chargeTrendRising) {
357                     chargeIndicatorUntilMs = nowMs + 120000UL;
358                 }
359             }
360             lastChargeCheckVoltage = voltage;
361             lastChargeCheckPercent = percent;
362             lastChargeCheckMs = nowMs;
363         }
364
365         return chargeTrendRising || nowMs < chargeIndicatorUntilMs;
366     }
367     return digitalRead(PIN_CHARGE_DETECT) == LOW;
368 }
369
370 void recordMatchLog() {
371     String entry = matchLogDate();
372     entry += "|";
373     entry += playerA_name;
374     entry += "|";
375     entry += playerB_name;
376     entry += "|";
377     entry += String(playerA_score);
378     entry += "-";
379     entry += String(playerB_score);
380
381     for (int8_t i = MATCH_LOG_MAX_ENTRIES - 1; i > 0; i--) {
382         matchLog[i] = matchLog[i - 1];
383     }
384     matchLog[0] = entry;
385     if (matchLogCount < MATCH_LOG_MAX_ENTRIES) {
386         matchLogCount++;
387     }
388
389     prefs.putUChar("logCount", matchLogCount);
390     for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
391         char key[8];
392         snprintf(key, sizeof(key), "log%u", i);
393         prefs.putString(key, matchLog[i]);
394     }
395 }
396
397 void clearMatchLog() {
398     matchLogCount = 0;
399     prefs.putUChar("logCount", matchLogCount);
400     for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
401         matchLog[i] = "";
402         char key[8];
403         snprintf(key, sizeof(key), "log%u", i);
404         prefs.putString(key, "");
405     }
406 }
407
408 String matchLogDate() {
409     uint8_t second = 0;
410     uint8_t minute = 0;
411     uint8_t hour = 0;
412     uint8_t day = 0;
413     uint8_t month = 0;
414     uint16_t year = 0;
415     if (!ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) || !rtcDateLooksSet(day, month, year)) {
416         return "DATE UNSET";
417     }
418
419     char buffer[12];
420     snprintf(buffer, sizeof(buffer), "%04u-%02u-%02u", year, month, day);
421     return String(buffer);
422 }
423
424 String matchLogPart(const String &entry, uint8_t part) {

```

```

425 int start = 0;
426 for (uint8_t i = 0; i < part; i++) {
427     int separator = entry.indexOf('|', start);
428     if (separator < 0) {
429         return "";
430     }
431     start = separator + 1;
432 }
433
434 int end = entry.indexOf('|', start);
435 if (end < 0) {
436     return entry.substring(start);
437 }
438 return entry.substring(start, end);
439 }
440
441 // -----
442 // Settings and power management
443 // -----
444 void loadSettings() {
445     prefs.begin("bgclock", false);
446     match_target = prefs.getInt("target", 9);
447     initial_time_seconds = prefs.getULong("time", DEFAULT_INITIAL_TIME_SECONDS);
448     delay_seconds = prefs.getUChar("delay", DEFAULT_DELAY_SECONDS);
449     if (delay_seconds == 7) {
450         delay_seconds = DEFAULT_DELAY_SECONDS;
451         prefs.putUChar("delay", delay_seconds);
452     }
453     increment_seconds = prefs.getUChar("inc", DEFAULT_INCREMENT_SECONDS);
454     brightness = prefs.getUChar("bright", DEFAULT_BRIGHTNESS);
455     timingMode = (TimingMode)prefs.getUChar("timing", (uint8_t)SIMPLE_DELAY);
456     if (timingMode > FISCHER_INCREMENT) {
457         timingMode = SIMPLE_DELAY;
458     }
459     timerScope = (TimerScope)prefs.getUChar("scope", (uint8_t)TIMER_BY_MATCH);
460     if (timerScope > TIMER_BY_MATCH) {
461         timerScope = TIMER_BY_MATCH;
462     }
463     playerA_name = prefs.getString("nameA", "PLAYER A");
464     playerB_name = prefs.getString("nameB", "PLAYER B");
465     playerA_color = prefs.getUShort("colorA", PLAYER_COLOR_BLUE);
466     playerB_color = prefs.getUShort("colorB", PLAYER_COLOR_RED);
467     if (playerA_name.length() == 0 || playerA_name.length() > PLAYER_NAME_MAX_LEN) {
468         playerA_name = "PLAYER A";
469     }
470     if (playerB_name.length() == 0 || playerB_name.length() > PLAYER_NAME_MAX_LEN) {
471         playerB_name = "PLAYER B";
472     }
473
474     matchLogCount = prefs.getUChar("logCount", 0);
475     if (matchLogCount > MATCH_LOG_MAX_ENTRIES) {
476         matchLogCount = MATCH_LOG_MAX_ENTRIES;
477     }
478     for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
479         char key[8];
480         snprintf(key, sizeof(key), "log%u", i);
481         matchLog[i] = prefs.getString(key, "");
482     }
483
484     bool validTarget = false;
485     for (uint8_t i = 0; i < MATCH_TARGET_COUNT; i++) {
486         if (MATCH_TARGETS[i] == match_target) {
487             validTarget = true;
488         }
489     }
490     if (!validTarget) {
491         match_target = 9;
492     }
493 }
494
495 void saveSettingsIfNeeded() {
496     if (!settingsDirty) {
497         return;
498     }
499
500     uint32_t nowMs = millis();
501     if (nowMs - settingsDirtyMs < SETTINGS_SAVE_DELAY_MS) {
502         return;
503     }
504
505     prefs.putInt("target", match_target);
506     prefs.putULong("time", initial_time_seconds);
507     prefs.putUChar("delay", delay_seconds);
508     prefs.putUChar("inc", increment_seconds);
509     prefs.putUChar("bright", brightness);
510     prefs.putUChar("timing", (uint8_t)timingMode);

```

```

511   prefs.putUChar("scope", (uint8_t)timerScope);
512   settingsDirty = false;
513 }
514
515 void savePlayerNames() {
516   prefs.putString("nameA", playerA_name);
517   prefs.putString("nameB", playerB_name);
518   prefs.putUShort("colorA", playerA_color);
519   prefs.putUShort("colorB", playerB_color);
520 }
521
522 void markSettingsDirty() {
523   settingsDirty = true;
524   settingsDirtyMs = millis();
525   markDisplayDirty();
526 }
527
528 void markDisplayDirty() {
529   displayDirty = true;
530 }
531
532 void setBrightness(uint8_t duty) {
533   if (duty == 0) {
534     digitalWrite(PIN_TFT_BACKLIGHT, TFT_BACKLIGHT_ACTIVE_HIGH ? LOW : HIGH);
535     return;
536   }
537
538   digitalWrite(PIN_TFT_BACKLIGHT, TFT_BACKLIGHT_ACTIVE_HIGH ? HIGH : LOW);
539 }
540
541 void wakeActivity() {
542   lastActivityMs = millis();
543 }
544
545 void maybeSleep() {
546   if (clock_running) {
547     return;
548   }
549
550   if (PIN_PLAYER_A_BUTTON == 255 || PIN_PLAYER_B_BUTTON == 255 || PIN_MENU_SELECT == 255) {
551     return;
552   }
553
554   uint32_t nowMs = millis();
555   if (nowMs - lastActivityMs < SLEEP_AFTER_IDLE_MS) {
556     return;
557   }
558
559   tft.fillScreen(ILI9341_BLACK);
560   setBrightness(0);
561   delay(20); // Tiny hardware settle before deep sleep.
562
563   // Wake on the two large player buttons and menu button. Add more pins if
564   // your board supports the required EXT1 wake bitmask for those GPIOs.
565   uint64_t wakeMask = (1ULL << PIN_PLAYER_A_BUTTON) |
566                       (1ULL << PIN_PLAYER_B_BUTTON) |
567                       (1ULL << PIN_MENU_SELECT);
568   esp_sleep_enable_ext1_wakeup(wakeMask, ESP_EXT1_WAKEUP_ANY_LOW);
569   esp_deep_sleep_start();
570 }
571
572 // -----
573 // Buzzer feedback
574 // -----
575 void beep(uint16_t frequency, uint16_t durationMs) {
576   if (PIN_BUZZER == 255) {
577     return;
578   }
579
580   tone(PIN_BUZZER, frequency, durationMs);
581 }
582
583 void playButtonBeep() {
584   beep(1800, 20);
585 }
586
587 void playTimeoutBeep() {
588   beep(900, 300);
589 }
590
591 void playMatchOverBeep() {
592   beep(1600, 120);
593   // tone() is non-blocking on most Arduino cores, so keep this simple.
594 }

```

6 Top-Level Function Inventory

The table below lists top-level functions detected in the sketch, where they appear, and their role in the firmware. Small class methods inside local helper classes are intentionally summarized with the surrounding segment when appropriate.

Function	Lines	Segment	Purpose
begin	284–296	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
update	298–316	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
wasPressed	318–320	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
begin	331–338	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
update	340–358	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
wasPressed	360–362	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
contains	371–373	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
containsPadded	375–377	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
begin	394–460	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
setRotation	462–466	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
fillScreen	468–470	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
setTextSize	472–474	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
setTextColor	476–479	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
setFont	481–483	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
setCursor	485–488	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
print	490–492	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.

Function	Lines	Segment	Purpose
<code>print</code>	494–501	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>print</code>	503–507	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>print</code>	509–513	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>print</code>	515–519	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>printf</code>	521–528	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>fillRect</code>	533–566	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>drawRGB565Image</code>	568–592	Core enums, button structures, Rects, and display wrapper	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawRGB565ImageScaled</code>	594–618	Core enums, button structures, Rects, and display wrapper	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawRoundRect</code>	620–626	Core enums, button structures, Rects, and display wrapper	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>getTextBounds</code>	628–637	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>writeCommand</code>	640–642	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>writeData</code>	644–646	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>writeData16</code>	648–650	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>setAddressWindow</code>	652–660	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>drawPixel</code>	662–667	Core enums, button structures, Rects, and display wrapper	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>getGfxTextBounds</code>	669–711	Core enums, button structures, Rects, and display wrapper	Helper function used by the firmware state machine.
<code>drawChar</code>	770–791	Core enums, button structures, Rects, and display wrapper	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawGfxChar</code>	793–822	Core enums, button structures, Rects, and display wrapper	Draws a screen, UI region, glyph, indicator, or other visual element.

Function	Lines	Segment	Purpose
setup	1108–1173	Arduino startup and run-time loop	Initializes display, splash, settings, I2C devices, touch, buttons, timers, and the first UI screen.
loop	1175–1203	Arduino startup and run-time loop	Main Arduino loop: handles input, updates timers, saves dirty settings, redraws the UI, and checks sleep.
handleButtons	1208–1321	Button and touch input handling	Reads direct GPIO and PCF8575 buttons and converts presses into game actions.
handleTouch	1323–1392	Button and touch input handling	Reads GT911 touch state and converts press/release movement into taps or swipes.
handleTouchTap	1394–1596	Button and touch input handling	Dispatches taps by screen mode and hit-test region.
handleTouchSwipe	1598–1613	Button and touch input handling	Handles user input or a screen-specific interaction.
pauseClock	1615–1625	Clock engine and timing rules	Helper function used by the firmware state machine.
resumeClock	1627–1637	Clock engine and timing rules	Helper function used by the firmware state machine.
togglePlayPause	1639–1651	Clock engine and timing rules	Helper function used by the firmware state machine.
switchClockTo	1653–1683	Clock engine and timing rules	Helper function used by the firmware state machine.
updateClock	1688–1731	Clock engine and timing rules	Helper function used by the firmware state machine.
addIncrementTo	1733–1744	Clock engine and timing rules	Helper function used by the firmware state machine.
applyTurnElapsed	1746–1782	Clock engine and timing rules	Helper function used by the firmware state machine.
visibleReserveMs	1784–1804	Clock engine and timing rules	Helper function used by the firmware state machine.
matchClockSeconds	1816–1818	Clock engine and timing rules	Helper function used by the firmware state machine.
startingClockMs	1820–1823	Clock engine and timing rules	Helper function used by the firmware state machine.
adjustScore	1828–1869	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
checkMatchState	1871–1900	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
startNewGame	1902–1937	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
endCurrentGame	1939–1950	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
markCrawfordUsed	1952–1958	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
forceEndMatch	1960–1972	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
resetMatch	1974–1994	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.

Function	Lines	Segment	Purpose
<code>enterMenu</code>	1999–2004	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
<code>handleMenu</code>	2006–2034	Scoring, Crawford logic, match lifecycle, and menu changes	Handles user input or a screen-specific interaction.
<code>changeMenuValue</code>	2036–2113	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
<code>nextMenuItem</code>	2115–2118	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
<code>previousMenuItem</code>	2120–2124	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
<code>findCurrentTargetIndex</code>	2126–2133	Scoring, Crawford logic, match lifecycle, and menu changes	Helper function used by the firmware state machine.
<code>drawDisplay</code>	2138–2335	Display renderer and main screens	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawMatchWinnerBanner</code>	2337–2347	Display renderer and main screens	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawTimeoutBanner</code>	2349–2359	Display renderer and main screens	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawEndMatchConfirm</code>	2361–2378	Display renderer and main screens	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawMatchLog</code>	2380–2424	Display renderer and main screens	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawClockSettings</code>	2426–2492	Display renderer and main screens	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawNameEditor</code>	2494–2560	Name editor, checker-color selection, timer panels, and player panels	Draws player-name editor with on-screen keyboard and checker-color swatches.
<code>openNameEditor</code>	2562–2574	Name editor, checker-color selection, timer panels, and player panels	Helper function used by the firmware state machine.
<code>openClockSettings</code>	2576–2598	Name editor, checker-color selection, timer panels, and player panels	Helper function used by the firmware state machine.
<code>handleClockSettingsTap</code>	2600–2645	Name editor, checker-color selection, timer panels, and player panels	Handles user input or a screen-specific interaction.
<code>handleEndMatchConfirmTap</code>	2647–2658	Name editor, checker-color selection, timer panels, and player panels	Handles user input or a screen-specific interaction.
<code>adjustRtcField</code>	2660–2686	Name editor, checker-color selection, timer panels, and player panels	Supports RTC date/time conversion, validation, and status display.
<code>handleNameEditorTap</code>	2688–2772	Name editor, checker-color selection, timer panels, and player panels	Handles name keyboard taps, color swatches, cancel/clear/OK, and save/cancel behavior.
<code>colorForPlayer</code>	2774–2782	Name editor, checker-color selection, timer panels, and player panels	Returns the currently selected checker color for a player.

Function	Lines	Segment	Purpose
<code>textColorForPlayerColor</code>	2784–2786	Name editor, checker-color selection, timer panels, and player panels	Chooses readable score text color for a selected checker color.
<code>playerColorOption</code>	2788–2805	Name editor, checker-color selection, timer panels, and player panels	Returns a preset checker color by index.
<code>playerColorOptionColor</code>	2807–2809	Name editor, checker-color selection, timer panels, and player panels	Provides optional audio feedback when a buzzer pin is configured.
<code>drawTimerPanels</code>	2811–2819	Name editor, checker-color selection, timer panels, and player panels	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawTimerPanel</code>	2821–2869	Name editor, checker-color selection, timer panels, and player panels	Draws a player timer panel, including active/inactive visual treatment.
<code>drawPlayerPanel</code>	2871–2906	Name editor, checker-color selection, timer panels, and player panels	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawStatusBar</code>	2908–2937	Name editor, checker-color selection, timer panels, and player panels	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawChargeBolt</code>	2939–2943	Name editor, checker-color selection, timer panels, and player panels	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawStartupTest</code>	2945–2951	Splash screen and generated graphic helpers	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawSplashImage</code>	2953–2959	Splash screen and generated graphic helpers	Draws the compiled splash image when available.
<code>drawGeneratedSplashImage</code>	2961–3018	Splash screen and generated graphic helpers	Draws a generated splash fallback when no image header is used.
<code>fillTriangleRows</code>	3020–3056	Splash screen and generated graphic helpers	Helper function used by the firmware state machine.
<code>fillCircleRows</code>	3058–3068	Splash screen and generated graphic helpers	Helper function used by the firmware state machine.
<code>initCrowPanelExpander</code>	3070–3105	I2C devices: expanders, fuel gauge, RTC, and touch controller	Finds and initializes the PCA9557 on the CrowPanel.
<code>pca9557Write</code>	3107–3112	I2C devices: expanders, fuel gauge, RTC, and touch controller	Performs low-level I2C access for an I/O expander.
<code>pca9557Read</code>	3114–3124	I2C devices: expanders, fuel gauge, RTC, and touch controller	Performs low-level I2C access for an I/O expander.
<code>initButtonExpander</code>	3126–3147	I2C devices: expanders, fuel gauge, RTC, and touch controller	Initializes optional PCF8575 for physical player buttons.
<code>pcf8575Write</code>	3149–3154	I2C devices: expanders, fuel gauge, RTC, and touch controller	Performs low-level I2C access for an I/O expander.
<code>pcf8575Read</code>	3156–3163	I2C devices: expanders, fuel gauge, RTC, and touch controller	Performs low-level I2C access for an I/O expander.
<code>initFuelGauge</code>	3165–3179	I2C devices: expanders, fuel gauge, RTC, and touch controller	Detects MAX17048 and reports battery voltage/SOC.

Function	Lines	Segment	Purpose
<code>max17048Read16</code>	3181–3200	I2C devices: expanders, fuel gauge, RTC, and touch controller	Reads and converts MAX17048 fuel-gauge data over I2C.
<code>max17048CellVoltage</code>	3202–3208	I2C devices: expanders, fuel gauge, RTC, and touch controller	Reads and converts MAX17048 fuel-gauge data over I2C.
<code>max17048StateOfCharge</code>	3210–3223	I2C devices: expanders, fuel gauge, RTC, and touch controller	Reads and converts MAX17048 fuel-gauge data over I2C.
<code>initRtcClock</code>	3225–3252	I2C devices: expanders, fuel gauge, RTC, and touch controller	Detects DS3231 and caches initial RTC status.
<code>ds3231ReadTime</code>	3254–3290	I2C devices: expanders, fuel gauge, RTC, and touch controller	Supports RTC date/time conversion, validation, and status display.
<code>ds3231SetTime</code>	3292–3307	I2C devices: expanders, fuel gauge, RTC, and touch controller	Supports RTC date/time conversion, validation, and status display.
<code>rtcDateLooksSet</code>	3309–3311	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.
<code>daysInMonth</code>	3313–3327	I2C devices: expanders, fuel gauge, RTC, and touch controller	Supports RTC date/time conversion, validation, and status display.
<code>bcdToDec</code>	3329–3331	I2C devices: expanders, fuel gauge, RTC, and touch controller	Supports RTC date/time conversion, validation, and status display.
<code>decToBcd</code>	3333–3335	I2C devices: expanders, fuel gauge, RTC, and touch controller	Supports RTC date/time conversion, validation, and status display.
<code>rtcStatusText</code>	3337–3351	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.
<code>updateRtcStatusCache</code>	3353–3364	I2C devices: expanders, fuel gauge, RTC, and touch controller	Supports RTC date/time conversion, validation, and status display.
<code>scanI2C</code>	3366–3377	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.
<code>resetTouchController</code>	3379–3388	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.
<code>initTouch</code>	3390–3428	I2C devices: expanders, fuel gauge, RTC, and touch controller	Resets and detects GT911 touch controller.
<code>readTouch</code>	3430–3484	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.
<code>gt911Read</code>	3486–3502	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.
<code>gt911Write</code>	3504–3510	I2C devices: expanders, fuel gauge, RTC, and touch controller	Helper function used by the firmware state machine.

Function	Lines	Segment	Purpose
<code>logLine</code>	3512–3515	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>logPrintf</code>	3517–3526	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>drawCentered</code>	3528–3538	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawCenteredInRect</code>	3540–3574	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Draws a screen, UI region, glyph, indicator, or other visual element.
<code>drawScoreInRect</code>	3576–3625	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Draws large score numbers using the score font path or fallback text scaling.
<code>drawMatchTargetInRect</code>	3627–3677	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Draws match target value with match-target font path or fallback.
<code>formatTime</code>	3785–3791	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Formats internal values into display-ready text.
<code>formatTimeMs</code>	3793–3796	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Formats internal values into display-ready text.
<code>timingModeLabel</code>	3798–3808	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>playPauseLabel</code>	3810–3818	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Provides optional audio feedback when a buzzer pin is configured.
<code>matchStateLabel</code>	3820–3828	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>batteryPercent</code>	3830–3851	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>chargingIndicatorOn</code>	3853–3878	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>recordMatchLog</code>	3880–3905	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.

Function	Lines	Segment	Purpose
<code>clearMatchLog</code>	3907–3916	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>matchLogDate</code>	3918–3932	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>matchLogPart</code>	3934–3949	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>loadSettings</code>	3954–4003	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Loads preferences, names, colors, timing settings, brightness, and log data.
<code>saveSettingsIfNeeded</code>	4005–4023	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Delayed write of changed preferences to reduce flash wear.
<code>savePlayerNames</code>	4025–4030	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>markSettingsDirty</code>	4032–4036	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>markDisplayDirty</code>	4038–4040	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>setBrightness</code>	4042–4049	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>wakeActivity</code>	4051–4053	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>maybeSleep</code>	4055–4080	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Helper function used by the firmware state machine.
<code>beep</code>	4085–4091	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Provides optional audio feedback when a buzzer pin is configured.
<code>playButtonBeep</code>	4093–4095	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Provides optional audio feedback when a buzzer pin is configured.
<code>playTimeoutBeep</code>	4097–4099	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Provides optional audio feedback when a buzzer pin is configured.

Function	Lines	Segment	Purpose
playMatchOverBeep	4101–4104	Formatting, custom font drawing, battery display, logs, settings, sleep, and sound	Provides optional audio feedback when a buzzer pin is configured.

7 Command/API Appendix

Command/API family	Why it appears in this sketch
Wire.begin, Wire.beginTransmission, Wire.requestFrom Preferences	Used for I2C communication with the PCF8575, PCA9557, MAX17048, DS3231, and GT911 touch controller. Stores match settings, brightness, names, player colors, and log data in ESP32 non-volatile storage.
esp_lcd_new_rgb_panel and panel ops millis()	Creates and controls the RGB LCD panel driver for the CrowPanel's parallel display. Provides non-blocking timing for clock countdown, debounce windows, idle timeout, and delayed saving.
pinMode, digitalRead, digitalWrite tone	Used for direct buttons, optional charger detect, backlight control, and optional buzzer support.
__has_include	Generates optional beeps when a buzzer pin is configured. Allows optional font headers and splash/font assets to be included when present but skipped safely when missing.
GFXfont / font headers	Support optional custom font rendering for UI, clock, score, and match target text.

A Full V 2.0 Source Code

The complete source code is included below with syntax coloring.

```

1  /*
2   * Portable Backgammon Match Clock and Scoreboard
3   * Target: Elecrow CrowPanel ESP32 HMI 5.0-inch display (DIS07050H)
4   *
5   * Display library:
6   *   This sketch uses the ESP32-S3 RGB LCD peripheral for the built-in 800x480
7   *   RGB-parallel panel.
8   *
9   * Clock model:
10  *   - At the start of each game, no clock is running.
11  *   - Pressing Player A's large button ends A's turn and starts B's clock.
12  *   - Pressing Player B's large button ends B's turn and starts A's clock.
13  *   - Optional delay/increment is supported. Delay is Bronstein/simple delay:
14  *     main time does not tick down until the active turn exceeds delaySeconds.
15  *   - Increment, if enabled, is added to the player whose turn was ended.
16  *
17  * Crawford model:
18  *   - When either player reaches match_target - 1 and Crawford has not been used,
19  *     the following game is marked Crawford and the cube is disabled.
20  *   - After that game ends, all later games are post-Crawford and cube is enabled.
21  */
22
23 #include <Arduino.h>
24 #include <Wire.h>
25 #include <stdarg.h>
26 #include <Preferences.h>
27 #include <esp_sleep.h>
28 #include <esp_heap_caps.h>
29 #include <esp_lcd_panel_ops.h>
30 #include <esp_lcd_panel_rgb.h>
31 #include "splash_image.h"
32
33 #if __has_include(<gfxfont.h>)

```

```

34 #include <gfxfont.h>
35 #else
36 typedef struct {
37     uint16_t bitmapOffset;
38     uint8_t width;
39     uint8_t height;
40     uint8_t xAdvance;
41     int8_t xOffset;
42     int8_t yOffset;
43 } GFXglyph;
44
45 typedef struct {
46     uint8_t *bitmap;
47     GFXglyph *glyph;
48     uint16_t first;
49     uint16_t last;
50     uint16_t yAdvance;
51 } GFXfont;
52 #endif
53
54 #if __has_include("FreeMonoBold24pt7b.h")
55 #include "FreeMonoBold24pt7b.h"
56 #define BGMC_HAS_CLOCK_FONT 1
57 #elif __has_include(<Fonts/FreeMonoBold24pt7b.h>)
58 #include <Fonts/FreeMonoBold24pt7b.h>
59 #define BGMC_HAS_CLOCK_FONT 1
60 #else
61 #define BGMC_HAS_CLOCK_FONT 0
62 #endif
63
64 #if __has_include("FreeSans18pt7b.h")
65 #include "FreeSans18pt7b.h"
66 #define BGMC_HAS_UI_FONT 1
67 #elif __has_include(<Fonts/FreeSans18pt7b.h>)
68 #include <Fonts/FreeSans18pt7b.h>
69 #define BGMC_HAS_UI_FONT 1
70 #else
71 #define BGMC_HAS_UI_FONT 0
72 #endif
73
74 #if __has_include("FreeSans12pt7b.h")
75 #include "FreeSans12pt7b.h"
76 #define BGMC_HAS_SMALL_UI_FONT 1
77 #elif __has_include(<Fonts/FreeSans12pt7b.h>)
78 #include <Fonts/FreeSans12pt7b.h>
79 #define BGMC_HAS_SMALL_UI_FONT 1
80 #else
81 #define BGMC_HAS_SMALL_UI_FONT 0
82 #endif
83
84 #if __has_include("FreeSansBold96pt7b.h")
85 #include "FreeSansBold96pt7b.h"
86 #define BGMC_HAS_SCORE_96_FONT 1
87 #elif __has_include(<Fonts/FreeSansBold96pt7b.h>)
88 #include <Fonts/FreeSansBold96pt7b.h>
89 #define BGMC_HAS_SCORE_96_FONT 1
90 #else
91 #define BGMC_HAS_SCORE_96_FONT 0
92 #endif
93
94 #if __has_include("collegeb96pt7b.h")
95 #include "collegeb96pt7b.h"
96 #define BGMC_HAS_COLLEGE_SCORE_FONT 1
97 #else
98 #define BGMC_HAS_COLLEGE_SCORE_FONT 0
99 #endif
100
101 #if __has_include("FreeSansBold48pt7b.h")
102 #include "FreeSansBold48pt7b.h"
103 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
104 #elif __has_include(<Fonts/FreeSansBold48pt7b.h>)
105 #include <Fonts/FreeSansBold48pt7b.h>
106 #define BGMC_HAS_MATCH_TARGET_48_FONT 1
107 #else
108 #define BGMC_HAS_MATCH_TARGET_48_FONT 0
109 #endif
110
111 #if __has_include("FreeSansBold24pt7b.h")
112 #include "FreeSansBold24pt7b.h"
113 #define BGMC_HAS_SCORE_FONT 1
114 #elif __has_include(<Fonts/FreeSansBold24pt7b.h>)
115 #include <Fonts/FreeSansBold24pt7b.h>
116 #define BGMC_HAS_SCORE_FONT 1
117 #else
118 #define BGMC_HAS_SCORE_FONT 0
119 #endif

```

```

120 // -----
121 // DIS07050H display-first pin map. External buttons can be added later on
122 // available expansion pins; avoid the LCD RGB bus pins below.
123 // -----
124
125 static const uint8_t PIN_PLAYER_A_BUTTON = 38; // GPIO_D IO38: one-button clock toggle to GND.
126 static const uint8_t PIN_PLAYER_B_BUTTON = 255;
127 static const uint8_t PIN_SCORE_A_PLUS = 255;
128 static const uint8_t PIN_SCORE_A_MINUS = 255;
129 static const uint8_t PIN_SCORE_B_PLUS = 255;
130 static const uint8_t PIN_SCORE_B_MINUS = 255;
131 static const uint8_t PIN_MENU_SELECT = 255;
132 static const uint8_t PIN_MENU_UP = 255;
133 static const uint8_t PIN_MENU_DOWN = 255;
134 static const uint8_t PIN_BUZZER = 255;
135 static const uint8_t PIN_TFT_BACKLIGHT = 2;
136 static const uint8_t PIN_BATTERY_ADC = 255;
137 static const uint8_t PIN_CHARGE_DETECT = 255; // Optional charger STAT pin, active-low when wired.
138
139 static const uint16_t PANEL_WIDTH = 800;
140 static const uint16_t PANEL_HEIGHT = 480;
141 static const uint8_t PIN_LCD_D0 = 8;
142 static const uint8_t PIN_LCD_D1 = 3;
143 static const uint8_t PIN_LCD_D2 = 46;
144 static const uint8_t PIN_LCD_D3 = 9;
145 static const uint8_t PIN_LCD_D4 = 1;
146 static const uint8_t PIN_LCD_D5 = 5;
147 static const uint8_t PIN_LCD_D6 = 6;
148 static const uint8_t PIN_LCD_D7 = 7;
149 static const uint8_t PIN_LCD_D8 = 15;
150 static const uint8_t PIN_LCD_D9 = 16;
151 static const uint8_t PIN_LCD_D10 = 4;
152 static const uint8_t PIN_LCD_D11 = 45;
153 static const uint8_t PIN_LCD_D12 = 48;
154 static const uint8_t PIN_LCD_D13 = 47;
155 static const uint8_t PIN_LCD_D14 = 21;
156 static const uint8_t PIN_LCD_D15 = 14;
157 static const uint8_t PIN_LCD_DE = 40;
158 static const uint8_t PIN_LCD_VSYNC = 41;
159 static const uint8_t PIN_LCD_HSYNC = 39;
160 static const uint8_t PIN_LCD_PCLK = 0;
161 static const uint8_t PIN_I2C_SDA = 19;
162 static const uint8_t PIN_I2C_SCL = 20;
163 static const uint8_t PCF8575_PLAY_PAUSE_PIN = 0; // PCF8575 P00, button to GND.
164 static const uint8_t PCF8575_PLAYER_A_PIN = 1; // PCF8575 P01, button to GND.
165 static const uint8_t PCF8575_PLAYER_B_PIN = 7; // PCF8575 P07, button to GND.
166 static const uint8_t MAX17048_I2C_ADDRESS = 0x36;
167 static const uint8_t DS3231_I2C_ADDRESS = 0x68;
168
169 // Rotary encoder placeholders. This sketch uses up/down buttons by default.
170 static const uint8_t PIN_ENCODER_A = 255;
171 static const uint8_t PIN_ENCODER_B = 255;
172
173 // Battery divider calibration placeholders.
174 static const float BATTERY_ADC_REF_VOLTS = 3.30f;
175 static const float BATTERY_DIVIDER_RATIO = 2.00f; // Vbat = Vadc * ratio.
176 static const float BATTERY_EMPTY_VOLTS = 3.30f;
177 static const float BATTERY_FULL_VOLTS = 4.20f;
178
179 // Display/backlight defaults.
180 static const uint8_t DEFAULT_BRIGHTNESS = 210; // 0..255 PWM duty.
181 static const bool TFT_BACKLIGHT_ACTIVE_HIGH = true;
182 static const uint16_t ILI9341_BLACK = 0x0000;
183 static const uint16_t ILI9341_WHITE = 0xFFFF;
184 static const uint16_t ILI9341_RED = 0xF800;
185 static const uint16_t ILI9341_GREEN = 0x07E0;
186 static const uint16_t ILI9341_BLUE = 0x001F;
187 static const uint16_t ILI9341_CYAN = 0x07FF;
188 static const uint16_t ILI9341_YELLOW = 0xFFE0;
189 static const uint16_t TFT_BACKGROUND = ILI9341_BLACK;
190 static const uint16_t TFT_PANEL = 0x2104;
191 static const uint16_t TFT_PANEL_ALT = 0x3186;
192 static const uint16_t TFT_BORDER = 0x7BEF;
193 static const uint16_t TFT_BUTTON_BORDER = 0xD6BA;
194 static const uint16_t TFT_TEXT = ILI9341_WHITE;
195 static const uint16_t TFT_TEXT_DARK = ILI9341_BLACK;
196 static const uint16_t TFT_ACTIVE_TEXT = 0x4208;
197 static const uint16_t TFT_MUTED = 0xBDF7;
198 static const uint16_t TFT_ACCENT_A = ILI9341_CYAN;
199 static const uint16_t TFT_ACCENT_B = 0xFD20;
200 static const uint16_t TFT_MAGENTA = 0xF81F;
201 static const uint16_t TFT_WARN = ILI9341_YELLOW;
202 static const uint16_t TFT_DANGER = ILI9341_RED;
203 static const uint16_t TFT_OK = ILI9341_GREEN;
204 static const uint16_t PLAYER_COLOR_RED = 0x7800; // #7C0A02 approximated in RGB565.
205 static const uint16_t PLAYER_COLOR_BLUE = 0x194A; // #1C2951 approximated in RGB565.

```

```

206 static const uint16_t PLAYER_COLOR_GREEN = 0x19A5; // #1C352D approximated in RGB565.
207 static const uint16_t PLAYER_COLOR_YELLOW = 0xFD20; // #FFA700 approximated in RGB565.
208 static const uint16_t PLAYER_COLOR_GRAY = 0x4208;
209 static const uint16_t PLAYER_COLOR_WHITE = ILI9341_WHITE;
210
211 // Time behavior.
212 static const uint32_t DEFAULT_INITIAL_TIME_SECONDS = 15UL * 60UL;
213 static const uint8_t DEFAULT_DELAY_SECONDS = 12;
214 static const uint8_t DEFAULT_INCREMENT_SECONDS = 0;
215 static const uint32_t LOW_TIME_WARNING_SECONDS = 30;
216 static const uint32_t CRITICAL_TIME_WARNING_SECONDS = 10;
217 static const uint32_t SLEEP_AFTER_IDLE_MS = 10UL * 60UL * 1000UL;
218 static const uint32_t LCD_PIXEL_CLOCK_HZ = 20000000;
219 static const uint16_t LCD_BOUNCE_BUFFER_LINES = 10;
220 static const uint8_t PLAYER_NAME_MAX_LEN = 10;
221
222 // Debounce intervals. Large buttons stay snappy; score/menu buttons are slower.
223 static const uint16_t PLAYER_BUTTON_DEBOUNCE_MS = 25;
224 static const uint16_t SMALL_BUTTON_DEBOUNCE_MS = 90;
225 static const uint16_t DISPLAY_REFRESH_MS = 1000;
226 static const uint16_t SETTINGS_SAVE_DELAY_MS = 1500;
227 static const char *FIRMWARE_VERSION = "lcd-refresh-2026-06-07";
228 static const uint8_t MATCH_LOG_MAX_ENTRIES = 8;
229
230 void logLine(const char *message);
231 void logPrintf(const char *format, ...);
232
233 // -----
234 // Types
235 // -----
236 enum Player : uint8_t {
237     PLAYER_A = 0,
238     PLAYER_B = 1,
239     PLAYER_NONE = 255
240 };
241
242 enum TimingMode : uint8_t {
243     SIMPLE_DELAY,
244     BRONSTEIN_DELAY,
245     FISCHER_INCREMENT
246 };
247
248 enum TimerScope : uint8_t {
249     TIMER_BY_GAME,
250     TIMER_BY_MATCH
251 };
252
253 enum DeviceMode : uint8_t {
254     MODE_MENU,
255     MODE_MATCH,
256     MODE_MATCH_OVER,
257     MODE_TIMEOUT,
258     MODE_NAME_EDIT,
259     MODE_LOG,
260     MODE_CLOCK_SET,
261     MODE_CONFIRM_END_MATCH
262 };
263
264 enum MenuItem : uint8_t {
265     MENU_MATCH_TARGET,
266     MENU_INITIAL_TIME,
267     MENU_DELAY,
268     MENU_INCREMENT,
269     MENU_BRIGHTNESS,
270     MENU_START_MATCH,
271     MENU_RESET_MATCH,
272     MENU_COUNT
273 };
274
275 struct DebouncedButton {
276     uint8_t pin;
277     bool activeLow;
278     bool stablePressed;
279     bool lastRawPressed;
280     bool pressedEvent;
281     uint32_t lastChangeMs;
282     uint16_t debounceMs;
283
284     void begin(uint8_t buttonPin, uint16_t debounce, bool useActiveLow = true) {
285         pin = buttonPin;
286         activeLow = useActiveLow;
287         stablePressed = false;
288         lastRawPressed = false;
289         pressedEvent = false;
290         lastChangeMs = 0;
291         debounceMs = debounce;

```

```

292     if (pin != 255) {
293         pinMode(pin, activeLow ? INPUT_PULLUP : INPUT);
294     }
295 }
296
297 void update(uint32_t nowMs) {
298     pressedEvent = false;
299     if (pin == 255) {
300         return;
301     }
302
303     bool rawPressed = activeLow ? (digitalRead(pin) == LOW) : (digitalRead(pin) == HIGH);
304     if (rawPressed != lastRawPressed) {
305         lastRawPressed = rawPressed;
306         lastChangeMs = nowMs;
307     }
308
309     if ((nowMs - lastChangeMs) >= debounceMs && rawPressed != stablePressed) {
310         stablePressed = rawPressed;
311         if (stablePressed) {
312             pressedEvent = true;
313         }
314     }
315 }
316
317 bool wasPressed() const {
318     return pressedEvent;
319 }
320 };
321
322 struct ExpanderButton {
323     uint8_t bit;
324     bool stablePressed;
325     bool lastRawPressed;
326     bool pressedEvent;
327     uint32_t lastChangeMs;
328     uint16_t debounceMs;
329
330     void begin(uint8_t expanderBit, uint16_t debounce) {
331         bit = expanderBit;
332         stablePressed = false;
333         lastRawPressed = false;
334         pressedEvent = false;
335         lastChangeMs = 0;
336         debounceMs = debounce;
337     }
338
339     void update(uint32_t nowMs, uint16_t portState) {
340         pressedEvent = false;
341         if (bit >= 16) {
342             return;
343         }
344
345         bool rawPressed = (portState & (1U << bit)) == 0;
346         if (rawPressed != lastRawPressed) {
347             lastRawPressed = rawPressed;
348             lastChangeMs = nowMs;
349         }
350
351         if ((nowMs - lastChangeMs) >= debounceMs && rawPressed != stablePressed) {
352             stablePressed = rawPressed;
353             if (stablePressed) {
354                 pressedEvent = true;
355             }
356         }
357     }
358 }
359
360 bool wasPressed() const {
361     return pressedEvent;
362 }
363 };
364
365 struct Rect {
366     int16_t x;
367     int16_t y;
368     int16_t w;
369     int16_t h;
370
371     bool contains(int16_t px, int16_t py) const {
372         return px >= x && py >= y && px < x + w && py < y + h;
373     }
374
375     bool containsPadded(int16_t px, int16_t py, int16_t padX, int16_t padY) const {
376         return px >= x - padX && py >= y - padY && px < x + w + padX && py < y + h + padY;
377     }

```

```

378 };
379
380 struct TouchPoint {
381     bool touched;
382     uint16_t x;
383     uint16_t y;
384 };
385
386 // -----
387 // Globals
388 // -----
389 class CrowPanelRGBDisplay {
390 public:
391     CrowPanelRGBDisplay(uint8_t csPin, uint8_t dcPin, uint8_t rstPin)
392         : cs(csPin), dc(dcPin), rst(rstPin) {}
393
394     void begin() {
395         (void)cs;
396         (void)dc;
397         (void)rst;
398
399         esp_lcd_rgb_panel_config_t panelConfig = {};
400         panelConfig.clk_src = LCD_CLK_SRC_DEFAULT;
401         panelConfig.timings.pclk_hz = LCD_PIXEL_CLOCK_HZ;
402         panelConfig.timings.h_res = PANEL_WIDTH;
403         panelConfig.timings.v_res = PANEL_HEIGHT;
404         panelConfig.timings.hsync_pulse_width = 4;
405         panelConfig.timings.hsync_back_porch = 43;
406         panelConfig.timings.hsync_front_porch = 8;
407         panelConfig.timings.vsync_pulse_width = 4;
408         panelConfig.timings.vsync_back_porch = 12;
409         panelConfig.timings.vsync_front_porch = 8;
410         panelConfig.timings.flags.hsync_idle_low = 0;
411         panelConfig.timings.flags.vsync_idle_low = 0;
412         panelConfig.timings.flags.de_idle_high = 0;
413         panelConfig.timings.flags.pclk_active_neg = 1;
414         panelConfig.timings.flags.pclk_idle_high = 0;
415         panelConfig.data_width = 16;
416         panelConfig.bits_per_pixel = 16;
417         panelConfig.num_fbs = 1;
418         panelConfig.bounce_buffer_size_px = PANEL_WIDTH * LCD_BOUNCE_BUFFER_LINES;
419         panelConfig.dma_burst_size = 32;
420         panelConfig.hsync_gpio_num = PIN_LCD_HSYNC;
421         panelConfig.vsync_gpio_num = PIN_LCD_VSYNC;
422         panelConfig.de_gpio_num = PIN_LCD_DE;
423         panelConfig.pclk_gpio_num = PIN_LCD_PCLK;
424         panelConfig.disp_gpio_num = -1;
425         panelConfig.data_gpio_nums[0] = PIN_LCD_D0;
426         panelConfig.data_gpio_nums[1] = PIN_LCD_D1;
427         panelConfig.data_gpio_nums[2] = PIN_LCD_D2;
428         panelConfig.data_gpio_nums[3] = PIN_LCD_D3;
429         panelConfig.data_gpio_nums[4] = PIN_LCD_D4;
430         panelConfig.data_gpio_nums[5] = PIN_LCD_D5;
431         panelConfig.data_gpio_nums[6] = PIN_LCD_D6;
432         panelConfig.data_gpio_nums[7] = PIN_LCD_D7;
433         panelConfig.data_gpio_nums[8] = PIN_LCD_D8;
434         panelConfig.data_gpio_nums[9] = PIN_LCD_D9;
435         panelConfig.data_gpio_nums[10] = PIN_LCD_D10;
436         panelConfig.data_gpio_nums[11] = PIN_LCD_D11;
437         panelConfig.data_gpio_nums[12] = PIN_LCD_D12;
438         panelConfig.data_gpio_nums[13] = PIN_LCD_D13;
439         panelConfig.data_gpio_nums[14] = PIN_LCD_D14;
440         panelConfig.data_gpio_nums[15] = PIN_LCD_D15;
441         panelConfig.flags.fb_in_psram = 1;
442
443         esp_err_t err = esp_lcd_new_rgb_panel(&panelConfig, &panelHandle);
444         logPrintf("esp_lcd_new_rgb_panel: %d\n", err);
445         if (err != ESP_OK) {
446             panelHandle = nullptr;
447             return;
448         }
449
450         err = esp_lcd_panel_reset(panelHandle);
451         logPrintf("esp_lcd_panel_reset: %d\n", err);
452         err = esp_lcd_panel_init(panelHandle);
453         logPrintf("esp_lcd_panel_init: %d\n", err);
454
455         lineBuffer = (uint16_t *)heap_caps_malloc(PANEL_WIDTH * sizeof(uint16_t), MALLOC_CAP_INTERNAL | MALLOC_CAP_8BIT);
456         if (lineBuffer == nullptr) {
457             lineBuffer = (uint16_t *)heap_caps_malloc(PANEL_WIDTH * sizeof(uint16_t), MALLOC_CAP_8BIT);
458         }
459         logPrintf("LCD line buffer: %s\n", lineBuffer == nullptr ? "FAILED" : "OK");
460     }
461
462     void setRotation(uint8_t rotation) {
463         (void)rotation;

```

```

464     screenWidth = PANEL_WIDTH;
465     screenHeight = PANEL_HEIGHT;
466 }
467
468 void fillScreen(uint16_t color) {
469     fillRect(0, 0, screenWidth, screenHeight, color);
470 }
471
472 void setTextSize(uint8_t size) {
473     textSize = (size == 0) ? 1 : size;
474 }
475
476 void setTextColor(uint16_t color, uint16_t background) {
477     textColor = color;
478     textBackground = background;
479 }
480
481 void setFont(const GFXfont *font = nullptr) {
482     gfxFont = font;
483 }
484
485 void setCursor(int16_t x, int16_t y) {
486     cursorX = x;
487     cursorY = y;
488 }
489
490 void print(const String &text) {
491     print(text.c_str());
492 }
493
494 void print(const char *text) {
495     if (text == nullptr) {
496         return;
497     }
498     while (*text) {
499         drawChar(*text++);
500     }
501 }
502
503 void print(int value) {
504     char buffer[16];
505     snprintf(buffer, sizeof(buffer), "%d", value);
506     print(buffer);
507 }
508
509 void print(unsigned int value) {
510     char buffer[16];
511     snprintf(buffer, sizeof(buffer), "%u", value);
512     print(buffer);
513 }
514
515 void print(unsigned long value) {
516     char buffer[20];
517     snprintf(buffer, sizeof(buffer), "%lu", value);
518     print(buffer);
519 }
520
521 void printf(const char *format, ...) {
522     char buffer[80];
523     va_list args;
524     va_start(args, format);
525     vsnprintf(buffer, sizeof(buffer), format, args);
526     va_end(args);
527     print(buffer);
528 }
529
530 int16_t width() const { return screenWidth; }
531 int16_t height() const { return screenHeight; }
532
533 void fillRect(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t color) {
534     if (w <= 0 || h <= 0 || x >= screenWidth || y >= screenHeight) {
535         return;
536     }
537     if (x < 0) {
538         w += x;
539         x = 0;
540     }
541     if (y < 0) {
542         h += y;
543         y = 0;
544     }
545     if (x + w > screenWidth) {
546         w = screenWidth - x;
547     }
548     if (y + h > screenHeight) {
549         h = screenHeight - y;

```

```

550     }
551     if (w <= 0 || h <= 0) {
552         return;
553     }
554
555     if (panelHandle == nullptr || lineBuffer == nullptr) {
556         return;
557     }
558
559     for (int16_t col = 0; col < w; col++) {
560         lineBuffer[col] = color;
561     }
562
563     for (int16_t row = 0; row < h; row++) {
564         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + w, y + row + 1, lineBuffer);
565     }
566 }
567
568 void drawRGB565Image(int16_t x, int16_t y, uint16_t w, uint16_t h, const uint16_t *pixels) {
569     if (panelHandle == nullptr || lineBuffer == nullptr || pixels == nullptr) {
570         return;
571     }
572
573     uint16_t drawWidth = w;
574     uint16_t drawHeight = h;
575     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
576         return;
577     }
578     if (x + drawWidth > screenWidth) {
579         drawWidth = screenWidth - x;
580     }
581     if (y + drawHeight > screenHeight) {
582         drawHeight = screenHeight - y;
583     }
584
585     for (uint16_t row = 0; row < drawHeight; row++) {
586         const uint32_t sourceOffset = (uint32_t)row * (uint32_t)w;
587         for (uint16_t col = 0; col < drawWidth; col++) {
588             lineBuffer[col] = pgm_read_word(&pixels[sourceOffset + col]);
589         }
590         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + drawWidth, y + row + 1, lineBuffer);
591     }
592 }
593
594 void drawRGB565ImageScaled2x(int16_t x, int16_t y, uint16_t w, uint16_t h, const uint16_t *pixels) {
595     if (panelHandle == nullptr || lineBuffer == nullptr || pixels == nullptr) {
596         return;
597     }
598
599     uint16_t drawWidth = w * 2;
600     uint16_t drawHeight = h * 2;
601     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
602         return;
603     }
604     if (x + drawWidth > screenWidth) {
605         drawWidth = screenWidth - x;
606     }
607     if (y + drawHeight > screenHeight) {
608         drawHeight = screenHeight - y;
609     }
610
611     for (uint16_t row = 0; row < drawHeight; row++) {
612         const uint32_t sourceOffset = (uint32_t)(row / 2) * (uint32_t)w;
613         for (uint16_t col = 0; col < drawWidth; col++) {
614             lineBuffer[col] = pgm_read_word(&pixels[sourceOffset + (col / 2)]);
615         }
616         esp_lcd_panel_draw_bitmap(panelHandle, x, y + row, x + drawWidth, y + row + 1, lineBuffer);
617     }
618 }
619
620 void drawRoundRect(int16_t x, int16_t y, int16_t w, int16_t h, int16_t radius, uint16_t color) {
621     (void)radius;
622     fillRect(x, y, w, 1, color);
623     fillRect(x, y + h - 1, w, 1, color);
624     fillRect(x, y, 1, h, color);
625     fillRect(x + w - 1, y, 1, h, color);
626 }
627
628 void getTextBounds(const String &text, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
629     if (gfxFont != nullptr) {
630         getGfxTextBounds(text, x, y, x1, y1, w, h);
631         return;
632     }
633     *x1 = 0;
634     *y1 = 0;
635     *w = text.length() * 6 * textSize;

```

```

636     *h = 8 * textSize;
637 }
638
639 private:
640 void writeCommand(uint8_t command) {
641     (void)command;
642 }
643
644 void writeData(uint8_t data) {
645     (void)data;
646 }
647
648 void writeData16(uint16_t data) {
649     (void)data;
650 }
651
652 void setAddressWindow(uint16_t x0, uint16_t y0, uint16_t x1, uint16_t y1) {
653     writeCommand(0x2A);
654     writeData16(x0);
655     writeData16(x1);
656     writeCommand(0x2B);
657     writeData16(y0);
658     writeData16(y1);
659     writeCommand(0x2C);
660 }
661
662 void drawPixel(int16_t x, int16_t y, uint16_t color) {
663     if (x < 0 || y < 0 || x >= screenWidth || y >= screenHeight) {
664         return;
665     }
666     fillRect(x, y, 1, 1, color);
667 }
668
669 void getGfxTextBounds(const String &text, int16_t x, int16_t y, int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
670     int16_t minX = 32767;
671     int16_t minY = 32767;
672     int16_t maxX = -32768;
673     int16_t maxY = -32768;
674     int16_t cursor = x;
675     uint16_t first = pgm_read_word(&gfxFont->first);
676     uint16_t last = pgm_read_word(&gfxFont->last);
677     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&gfxFont->glyph);
678
679     for (uint16_t i = 0; i < text.length(); i++) {
680         char c = text.charAt(i);
681         if (c < first || c > last) {
682             continue;
683         }
684         GFXglyph glyph;
685         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
686         if (glyph.width > 0 && glyph.height > 0) {
687             int16_t gx1 = cursor + (glyph.xOffset * textSize);
688             int16_t gy1 = y + (glyph.yOffset * textSize);
689             int16_t gx2 = gx1 + ((int16_t)glyph.width * textSize) - 1;
690             int16_t gy2 = gy1 + ((int16_t)glyph.height * textSize) - 1;
691             if (gx1 < minX) minX = gx1;
692             if (gy1 < minY) minY = gy1;
693             if (gx2 > maxX) maxX = gx2;
694             if (gy2 > maxY) maxY = gy2;
695         }
696         cursor += (int16_t)glyph.xAdvance * textSize;
697     }
698
699     if (maxX < minX || maxY < minY) {
700         *x1 = x;
701         *y1 = y;
702         *w = 0;
703         *h = 0;
704         return;
705     }
706
707     *x1 = minX;
708     *y1 = minY;
709     *w = maxX - minX + 1;
710     *h = maxY - minY + 1;
711 }
712
713 const char *glyphRow(char c, uint8_t row) const {
714     if (c >= 'a' && c <= 'z') {
715         c -= 32;
716     }
717     static const char blank[7][6] = {"00000", "00000", "00000", "00000", "00000", "00000", "00000"};
718     static const char unknown[7][6] = {"11111", "10001", "00010", "00100", "00100", "00000", "00100"};
719     #define GLYPH(ch, a,b,c,d,e,f,g) case ch: { static const char p[7][6] = {a,b,c,d,e,f,g}; return p[row]; }
720     switch (c) {
721         GLYPH(' ', "00000", "00000", "00000", "00000", "00000", "00000")

```

```

722 GLYPH('0', "01110", "10001", "10011", "10101", "11001", "10001", "01110")
723 GLYPH('1', "00100", "01100", "00100", "00100", "00100", "00100", "01110")
724 GLYPH('2', "01110", "10001", "00001", "00010", "00100", "01000", "11111")
725 GLYPH('3', "11110", "00001", "00001", "01110", "00001", "00001", "11110")
726 GLYPH('4', "00010", "00110", "01010", "10010", "11111", "00010", "00010")
727 GLYPH('5', "11111", "10000", "10000", "11110", "00001", "00001", "11110")
728 GLYPH('6', "01110", "10000", "10000", "11110", "10001", "10001", "01110")
729 GLYPH('7', "11111", "00001", "00010", "00100", "01000", "01000", "01000")
730 GLYPH('8', "01110", "10001", "10001", "01110", "10001", "10001", "01110")
731 GLYPH('9', "01110", "10001", "10001", "01111", "00001", "00001", "01110")
732 GLYPH('A', "01110", "10001", "10001", "11111", "10001", "10001", "10001")
733 GLYPH('B', "11110", "10001", "10001", "11110", "10001", "10001", "11110")
734 GLYPH('C', "01111", "10000", "10000", "10000", "10000", "10000", "01111")
735 GLYPH('D', "11110", "10001", "10001", "10001", "10001", "10001", "11110")
736 GLYPH('E', "11111", "10000", "10000", "11110", "10000", "10000", "11111")
737 GLYPH('F', "11111", "10000", "10000", "11110", "10000", "10000", "10000")
738 GLYPH('G', "01111", "10000", "10000", "10011", "10001", "10001", "01111")
739 GLYPH('H', "10001", "10001", "10001", "11111", "10001", "10001", "10001")
740 GLYPH('I', "01110", "00100", "00100", "00100", "00100", "00100", "01110")
741 GLYPH('J', "00111", "00010", "00010", "00010", "00010", "10010", "01100")
742 GLYPH('K', "10001", "10010", "10100", "11000", "10100", "10010", "10001")
743 GLYPH('L', "10000", "10000", "10000", "10000", "10000", "10000", "11111")
744 GLYPH('M', "10001", "11011", "10101", "10101", "10001", "10001", "10001")
745 GLYPH('N', "10001", "11001", "10101", "10011", "10001", "10001", "10001")
746 GLYPH('O', "01110", "10001", "10001", "10001", "10001", "10001", "01110")
747 GLYPH('P', "11110", "10001", "10001", "11110", "10000", "10000", "10000")
748 GLYPH('Q', "01110", "10001", "10001", "10001", "10101", "10010", "01101")
749 GLYPH('R', "11110", "10001", "10001", "10001", "11110", "10100", "10010")
750 GLYPH('S', "01111", "10000", "10000", "01110", "00001", "00001", "11110")
751 GLYPH('T', "11111", "00100", "00100", "00100", "00100", "00100", "00100")
752 GLYPH('U', "10001", "10001", "10001", "10001", "10001", "10001", "01110")
753 GLYPH('V', "10001", "10001", "10001", "10001", "10001", "01010", "00100")
754 GLYPH('W', "10001", "10001", "10001", "10101", "10101", "10101", "01010")
755 GLYPH('X', "10001", "10001", "01010", "00100", "01010", "10001", "10001")
756 GLYPH('Y', "10001", "10001", "01010", "00100", "00100", "00100", "00100")
757 GLYPH('Z', "11111", "00001", "00010", "00100", "01000", "10000", "11111")
758 GLYPH('.', "00000", "00100", "00100", "00000", "00100", "00100", "00000")
759 GLYPH('-', "00000", "00000", "00000", "00000", "00000", "01100", "01100")
760 GLYPH('_', "00000", "00000", "00000", "11111", "00000", "00000", "00000")
761 GLYPH('/', "00001", "00010", "00010", "00100", "01000", "01000", "10000")
762 GLYPH('+', "00000", "00100", "00100", "11111", "00100", "00100", "00000")
763 GLYPH('%', "11001", "11010", "00010", "00100", "01000", "01011", "10011")
764 default:
765     return (c == '\0') ? blank[row] : unknown[row];
766 }
767 #undef GLYPH
768 }
769
770 void drawChar(char c) {
771     if (c == '\n') {
772         cursorX = 0;
773         cursorY += gfxFont == nullptr ? (8 * textSize) : (pgm_read_word(&gfxFont->yAdvance) * textSize);
774         return;
775     }
776
777     if (gfxFont != nullptr) {
778         drawGfxChar(c);
779         return;
780     }
781
782     for (uint8_t row = 0; row < 7; row++) {
783         const char *bits = glyphRow(c, row);
784         for (uint8_t col = 0; col < 5; col++) {
785             uint16_t color = (bits[col] == '1') ? textColor : textBackground;
786             fillRect(cursorX + col * textSize, cursorY + row * textSize, textSize, textSize, color);
787         }
788     }
789     fillRect(cursorX + 5 * textSize, cursorY, textSize, 7 * textSize, textBackground);
790     cursorX += 6 * textSize;
791 }
792
793 void drawGfxChar(char c) {
794     uint16_t first = pgm_read_word(&gfxFont->first);
795     uint16_t last = pgm_read_word(&gfxFont->last);
796     if (c < first || c > last) {
797         return;
798     }
799
800     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&gfxFont->glyph);
801     const uint8_t *bitmap = (const uint8_t *)pgm_read_ptr(&gfxFont->bitmap);
802     GFXglyph glyph;
803     memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
804
805     uint16_t bitIndex = 0;
806     uint16_t bitmapOffset = glyph.bitmapOffset;
807     uint8_t bits = 0;

```

```

808     for (uint8_t yy = 0; yy < glyph.height; yy++) {
809         for (uint8_t xx = 0; xx < glyph.width; xx++) {
810             if ((bitIndex & 7) == 0) {
811                 bits = pgm_read_byte(&bitmap[bitmapOffset++]);
812             }
813             if (bits & 0x80) {
814                 fillRect(cursorX + (glyph.xOffset + xx) * textSize, cursorY + (glyph.yOffset + yy) * textSize,
815                     textSize, textSize, textColor);
816             }
817             bits <<= 1;
818             bitIndex++;
819         }
820     }
821     cursorX += (int16_t)glyph.xAdvance * textSize;
822 }
823
824 uint8_t cs;
825 uint8_t dc;
826 uint8_t rst;
827 int16_t screenWidth = PANEL_WIDTH;
828 int16_t screenHeight = PANEL_HEIGHT;
829 int16_t cursorX = 0;
830 int16_t cursorY = 0;
831 uint16_t textColor = ILI9341_WHITE;
832 uint16_t textBackground = ILI9341_BLACK;
833 uint8_t textSize = 1;
834 const GFXfont *gfxFont = nullptr;
835 esp_lcd_panel_handle_t panelHandle = nullptr;
836 uint16_t *lineBuffer = nullptr;
837 };
838
839 CrowPanelRGBDisplay tft(255, 255, 255);
840 Preferences prefs;
841
842 DebouncedButton btnPlayerA;
843 DebouncedButton btnPlayerB;
844 DebouncedButton btnScoreAPlus;
845 DebouncedButton btnScoreAMinus;
846 DebouncedButton btnScoreBPlus;
847 DebouncedButton btnScoreBMinus;
848 DebouncedButton btnSelect;
849 DebouncedButton btnUp;
850 DebouncedButton btnDown;
851 ExpanderButton expanderBtnPlayerA;
852 ExpanderButton expanderBtnPlayerB;
853 ExpanderButton expanderBtnPlayPause;
854
855 int playerA_score = 0;
856 int playerB_score = 0;
857 int match_target = 9;
858 uint32_t playerA_time_remaining = DEFAULT_INITIAL_TIME_SECONDS * 1000UL;
859 uint32_t playerB_time_remaining = DEFAULT_INITIAL_TIME_SECONDS * 1000UL;
860 uint32_t match_elapsed_seconds = 0;
861 Player active_player = PLAYER_NONE;
862 bool clock_running = false;
863 bool crawford_used = false;
864 bool crawford_active = false;
865 bool post_crawford = false;
866 bool cube_enabled = true;
867 uint16_t game_number = 0;
868
869 uint32_t initial_time_seconds = DEFAULT_INITIAL_TIME_SECONDS;
870 uint8_t delay_seconds = DEFAULT_DELAY_SECONDS;
871 uint8_t increment_seconds = DEFAULT_INCREMENT_SECONDS;
872 uint8_t brightness = DEFAULT_BRIGHTNESS;
873 TimingMode timingMode = SIMPLE_DELAY;
874 TimerScope timerScope = TIMER_BY_MATCH;
875 String playerA_name = "PLAYER A";
876 String playerB_name = "PLAYER B";
877 uint16_t playerA_color = PLAYER_COLOR_BLUE;
878 uint16_t playerB_color = PLAYER_COLOR_RED;
879 String matchLog[MATCH_LOG_MAX_ENTRIES];
880 uint8_t matchLogCount = 0;
881
882 DeviceMode mode = MODE_MENU;
883 MenuItem currentMenuItem = MENU_MATCH_TARGET;
884 Player timeoutPlayer = PLAYER_NONE;
885 Player matchWinner = PLAYER_NONE;
886 Player editingNameFor = PLAYER_NONE;
887 String editingName;
888 uint16_t editingPlayerColor = PLAYER_COLOR_BLUE;
889
890 bool pending_crawford_next_game = false;
891 bool game_in_progress = false;
892 bool settingsDirty = false;
893 bool displayDirty = true;

```

```

894 bool displayModeKnown = false;
895 DeviceMode lastDrawMode = MODE_MENU;
896 uint32_t settingsDirtyMs = 0;
897 uint32_t lastTickMs = 0;
898 uint32_t turnStartMs = 0;
899 uint32_t pausedTurnElapsedMs = 0;
900 uint32_t lastDisplayMs = 0;
901 uint32_t lastActivityMs = 0;
902 uint32_t lastLowTimeBeepMs = 0;
903 uint8_t gt911Address = 0;
904 bool touchWasDown = false;
905 uint16_t touchStartX = 0;
906 uint16_t touchStartY = 0;
907 uint16_t touchLastX = 0;
908 uint16_t touchLastY = 0;
909 uint16_t touchMinY = 0;
910 uint16_t touchMaxY = 0;
911 uint32_t touchStartMs = 0;
912 uint32_t touchLastSeenMs = 0;
913 uint32_t touchLastHandledMs = 0;
914 uint8_t pca9557Address = 0;
915 uint8_t pcf8575Address = 0;
916 bool max17048Found = false;
917 bool ds3231Found = false;
918 bool lastTouchSeen = false;
919 float lastChargeCheckVoltage = 0.0f;
920 float lastChargeCheckPercent = 0.0f;
921 bool chargeTrendRising = false;
922 uint32_t lastChargeCheckMs = 0;
923 uint32_t chargeIndicatorUntilMs = 0;
924 String cachedRtcStatus = "DATE UNSET";
925 uint32_t lastRtcStatusMs = 0;
926 uint16_t editingRtcYear = 2026;
927 uint8_t editingRtcMonth = 1;
928 uint8_t editingRtcDay = 1;
929 uint8_t editingRtcHour = 12;
930 uint8_t editingRtcMinute = 0;
931
932 const int MATCH_TARGETS[] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21};
933 const uint8_t MATCH_TARGET_COUNT = sizeof(MATCH_TARGETS) / sizeof(MATCH_TARGETS[0]);
934
935 const Rect REGION_STATUS = {0, 0, 800, 34};
936 const Rect REGION_HEADER_A = {12, 42, 228, 32};
937 const Rect REGION_TIMER_A = {12, 82, 228, 108};
938 const Rect REGION_SCORE_A = {12, 210, 228, 246};
939 const Rect REGION_PLAY_PAUSE = {255, 42, 290, 56};
940 const Rect REGION_END_GAME = {255, 113, 130, 56};
941 const Rect REGION_END_MATCH = {415, 113, 130, 56};
942 const Rect REGION_MATCH_TARGET = {255, 184, 290, 211};
943 const Rect REGION_SETTINGS = {255, 410, 290, 46};
944 const Rect REGION_HEADER_B = {560, 42, 228, 32};
945 const Rect REGION_TIMER_B = {560, 82, 228, 108};
946 const Rect REGION_SCORE_B = {560, 210, 228, 246};
947 const Rect HIT_PLAY_PAUSE = {245, 36, 310, 70};
948 const Rect HIT_END_GAME = {245, 106, 150, 70};
949 const Rect HIT_END_MATCH = {405, 106, 150, 70};
950 const Rect CONFIRM_NO = {172, 306, 190, 62};
951 const Rect CONFIRM_YES = {438, 306, 190, 62};
952
953 const Rect SETTINGS_MATCH_MINUS = {372, 86, 96, 48};
954 const Rect SETTINGS_MATCH_PLUS = {672, 86, 96, 48};
955 const Rect SETTINGS_TIME_MINUS = {372, 166, 96, 48};
956 const Rect SETTINGS_TIME_PLUS = {672, 166, 96, 48};
957 const Rect SETTINGS_DELAY_MINUS = {372, 246, 96, 48};
958 const Rect SETTINGS_DELAY_PLUS = {672, 246, 96, 48};
959 const Rect SETTINGS_MATCH_ROW_MINUS = {52, 82, 348, 58};
960 const Rect SETTINGS_MATCH_ROW_PLUS = {400, 82, 348, 58};
961 const Rect SETTINGS_TIME_ROW_MINUS = {52, 162, 348, 58};
962 const Rect SETTINGS_TIME_ROW_PLUS = {400, 162, 348, 58};
963 const Rect SETTINGS_DELAY_ROW_MINUS = {52, 242, 348, 58};
964 const Rect SETTINGS_DELAY_ROW_PLUS = {400, 242, 348, 58};
965 const Rect SETTINGS_SCOPE_GAME = {255, 306, 120, 36};
966 const Rect SETTINGS_SCOPE_MATCH = {392, 306, 120, 36};
967 const Rect SETTINGS_MODE_SIMPLE = {52, 352, 210, 38};
968 const Rect SETTINGS_MODE_BRONSTEIN = {295, 352, 210, 38};
969 const Rect SETTINGS_MODE_FISCHER = {538, 352, 210, 38};
970 const Rect SETTINGS_LOG = {52, 410, 150, 44};
971 const Rect SETTINGS_CLOCK = {220, 410, 130, 44};
972 const Rect SETTINGS_START = {368, 410, 170, 44};
973 const Rect SETTINGS_END = {556, 410, 170, 44};
974 const Rect SETTINGS_BACK = {24, 24, 108, 44};
975 const Rect LOG_BACK = {24, 24, 108, 44};
976 const Rect LOG_CLEAR = {650, 24, 108, 44};
977 const Rect CLOCK_BACK = {24, 24, 108, 44};
978 const Rect CLOCK_SAVE = {560, 400, 168, 48};
979 const Rect CLOCK_YEAR_MINUS = {372, 84, 96, 44};

```

```

980 const Rect CLOCK_YEAR_PLUS = {672, 84, 96, 44};
981 const Rect CLOCK_MONTH_MINUS = {372, 142, 96, 44};
982 const Rect CLOCK_MONTH_PLUS = {672, 142, 96, 44};
983 const Rect CLOCK_DAY_MINUS = {372, 200, 96, 44};
984 const Rect CLOCK_DAY_PLUS = {672, 200, 96, 44};
985 const Rect CLOCK_HOUR_MINUS = {372, 258, 96, 44};
986 const Rect CLOCK_HOUR_PLUS = {672, 258, 96, 44};
987 const Rect CLOCK_MINUTE_MINUS = {372, 316, 96, 44};
988 const Rect CLOCK_MINUTE_PLUS = {672, 316, 96, 44};
989
990 // -----
991 // Forward declarations requested by the firmware structure.
992 // -----
993 void drawDisplay();
994 void drawTimerPanels();
995 void drawTimerPanel(Player player, const Rect &region);
996 void drawSplashImage();
997 void fillTriangleRows(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color);
998 void fillCircleRows(int16_t cx, int16_t cy, int16_t radius, uint16_t color);
999 void drawCenteredInRect(const String &text, const Rect &region, uint8_t textSize, uint16_t color, uint16_t background);
1000 void drawScoreInRect(int score, const Rect &region, uint16_t color, uint16_t background);
1001 void drawMatchTargetInRect(int target, const Rect &region, uint16_t color, uint16_t background);
1002 void drawGfxTextScaledInRect(const GFXfont *font, const String &text, const Rect &region, uint16_t color, uint8_t scaleNum, uint8_t
    ↳ scaleDen);
1003 void getGfxTextBoundsScaled(const GFXfont *font, const String &text, uint8_t scaleNum, uint8_t scaleDen, int16_t *x1, int16_t *y1,
    ↳ uint16_t *w, uint16_t *h);
1004 void drawGfxTextScaled(const GFXfont *font, const String &text, int16_t x, int16_t y, uint16_t color, uint8_t scaleNum, uint8_t scaleDen)
    ↳ ;
1005 void drawMatchWinnerBanner();
1006 void drawTimeoutBanner();
1007 void drawNameEditor();
1008 void drawMatchLog();
1009 void drawClockSettings();
1010 void drawEndMatchConfirm();
1011 void openClockSettings();
1012 void handleClockSettingsTap(uint16_t x, uint16_t y);
1013 void handleEndMatchConfirmTap(uint16_t x, uint16_t y);
1014 void adjustRtcField(uint8_t field, int delta);
1015 void openNameEditor(Player player);
1016 void handleNameEditorTap(uint16_t x, uint16_t y);
1017 uint16_t colorForPlayer(Player player);
1018 uint16_t textColorForPlayerColor(uint16_t color);
1019 uint16_t playerColorOption(uint8_t index);
1020 uint8_t playerColorOptionCount();
1021 void savePlayerNames();
1022 void recordMatchLog();
1023 void clearMatchLog();
1024 String matchLogDate();
1025 String matchLogPart(const String &entry, uint8_t part);
1026 void handleButtons();
1027 void handleTouch();
1028 void handleTouchTap(uint16_t x, uint16_t y);
1029 void handleTouchSwipe(uint16_t startX, uint16_t startY, uint16_t endX, uint16_t endY);
1030 void updateClock();
1031 void adjustScore(Player player, int delta);
1032 void checkMatchState();
1033 void startNewGame();
1034 void endCurrentGame();
1035 void markCrawfordUsed();
1036 void forceEndMatch();
1037 void resetMatch();
1038 void enterMenu();
1039 void handleMenu();
1040
1041 // Helper declarations.
1042 void loadSettings();
1043 void saveSettingsIfNeeded();
1044 void markSettingsDirty();
1045 void markDisplayDirty();
1046 void setBrightness(uint8_t duty);
1047 void wakeActivity();
1048 void maybeSleep();
1049 void beep(uint16_t frequency, uint16_t durationMs);
1050 void playButtonBeep();
1051 void playTimeoutBeep();
1052 void playMatchOverBeep();
1053 void pauseClock();
1054 void resumeClock();
1055 void togglePlayPause();
1056 void switchClockTo(Player nextPlayer);
1057 void addIncrementTo(Player player);
1058 bool applyTurnElapsed(Player player, uint32_t nowMs);
1059 uint32_t visibleReserveMs(Player player, uint32_t nowMs);
1060 uint32_t *reserveFor(Player player);
1061 uint32_t matchClockSeconds();
1062 uint32_t startingClockMs();

```

```

1063 String formatTimeMs(uint32_t milliseconds);
1064 String timingModeLabel();
1065 String playPauseLabel();
1066 void drawCentered(const String &text, int16_t y, uint8_t textSize, uint16_t color);
1067 void drawPlayerPanel(Player player, int16_t x, int16_t y, int16_t w, int16_t h);
1068 void drawStatusBar();
1069 void drawChargeBolt(int16_t x, int16_t y, uint16_t color, uint16_t background);
1070 void drawStartupTest();
1071 void initCrowPanelExpander();
1072 void initButtonExpander();
1073 void initFuelGauge();
1074 void initRtcClock();
1075 bool pca9557Write(uint8_t address, uint8_t reg, uint8_t value);
1076 uint8_t pca9557Read(uint8_t address, uint8_t reg);
1077 bool pcf8575Write(uint8_t address, uint16_t value);
1078 uint16_t pcf8575Read(uint8_t address);
1079 bool max17048Read16(uint8_t reg, uint16_t *value);
1080 float max17048CellVoltage();
1081 float max17048StateOfCharge();
1082 bool ds3231ReadTime(uint8_t *second, uint8_t *minute, uint8_t *hour, uint8_t *day, uint8_t *month, uint16_t *year);
1083 bool ds3231SetTime(uint8_t second, uint8_t minute, uint8_t hour, uint8_t day, uint8_t month, uint16_t year);
1084 bool rtcDateLooksSet(uint8_t day, uint8_t month, uint16_t year);
1085 uint8_t daysInMonth(uint8_t month, uint16_t year);
1086 uint8_t bcdToDec(uint8_t value);
1087 uint8_t decToBcd(uint8_t value);
1088 String rtcStatusText();
1089 bool updateRtcStatusCache(bool force);
1090 void scanI2C(const char *label);
1091 void resetTouchController(uint8_t outputValue, uint8_t configValue);
1092 void initTouch();
1093 TouchPoint readTouch();
1094 bool gt911Read(uint16_t reg, uint8_t *data, uint8_t len);
1095 bool gt911Write(uint16_t reg, uint8_t value);
1096 String formatTime(uint32_t seconds);
1097 String matchStateLabel();
1098 int batteryPercent();
1099 bool chargingIndicatorOn();
1100 void changeMenuValue(int delta);
1101 void nextMenuItem();
1102 void previousMenuItem();
1103 uint8_t findCurrentTargetIndex();
1104
1105 // -----
1106 // Arduino entry points
1107 // -----
1108 void setup() {
1109     Serial.begin(115200);
1110     Serial0.begin(115200);
1111     delay(200);
1112     logLine("");
1113     logLine("Backgammon Match Clock booting");
1114     logPrintf("Firmware: %s\n", FIRMWARE_VERSION);
1115     logPrintf("PSRAM found: %s\n", psramFound() ? "yes" : "no");
1116     logPrintf("Free PSRAM: %u bytes\n", ESP.getFreePsram());
1117     logPrintf("Total PSRAM: %u bytes\n", ESP.getPsramSize());
1118     if (!psramFound() || ESP.getPsramSize() < 800000) {
1119         logLine("ERROR: PSRAM is not enabled. Set Tools > PSRAM to OPI PSRAM and upload again.");
1120     }
1121     logLine("Buttons are optional for startup; the menu should draw with no buttons connected.");
1122     logPrintf("DIS07050H RGB panel: %ux%u, BL=%u, PCLK=%u HSYNC=%u VSYNC=%u DE=%u\n",
1123         PANEL_WIDTH, PANEL_HEIGHT, PIN_TFT_BACKLIGHT, PIN_LCD_PCLK,
1124         PIN_LCD_HSYNC, PIN_LCD_VSYNC, PIN_LCD_DE);
1125
1126     btnPlayerA.begin(PIN_PLAYER_A_BUTTON, PLAYER_BUTTON_DEBOUNCE_MS);
1127     btnPlayerB.begin(PIN_PLAYER_B_BUTTON, PLAYER_BUTTON_DEBOUNCE_MS);
1128     btnScoreAPlus.begin(PIN_SCORE_A_PLUS, SMALL_BUTTON_DEBOUNCE_MS);
1129     btnScoreAMinus.begin(PIN_SCORE_A_MINUS, SMALL_BUTTON_DEBOUNCE_MS);
1130     btnScoreBPlus.begin(PIN_SCORE_B_PLUS, SMALL_BUTTON_DEBOUNCE_MS);
1131     btnScoreBMinus.begin(PIN_SCORE_B_MINUS, SMALL_BUTTON_DEBOUNCE_MS);
1132     btnSelect.begin(PIN_MENU_SELECT, SMALL_BUTTON_DEBOUNCE_MS);
1133     btnUp.begin(PIN_MENU_UP, SMALL_BUTTON_DEBOUNCE_MS);
1134     btnDown.begin(PIN_MENU_DOWN, SMALL_BUTTON_DEBOUNCE_MS);
1135     expanderBtnPlayerA.begin(PCF8575_PLAYER_A_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
1136     expanderBtnPlayerB.begin(PCF8575_PLAYER_B_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
1137     expanderBtnPlayPause.begin(PCF8575_PLAY_PAUSE_PIN, PLAYER_BUTTON_DEBOUNCE_MS);
1138
1139     if (PIN_BUZZER != 255) {
1140         pinMode(PIN_BUZZER, OUTPUT);
1141         digitalWrite(PIN_BUZZER, LOW);
1142     }
1143     if (PIN_CHARGE_DETECT != 255) {
1144         pinMode(PIN_CHARGE_DETECT, INPUT_PULLUP);
1145     }
1146
1147     pinMode(PIN_TFT_BACKLIGHT, OUTPUT);
1148     setBrightness(255);

```

```

1149 analogReadResolution(12);
1150
1151
1152 initCrowPanelExpander();
1153 initButtonExpander();
1154 initFuelGauge();
1155 initRtcClock();
1156 initTouch();
1157 tft.begin();
1158 tft.setRotation(1);
1159 drawStartupTest();
1160
1161 loadSettings();
1162 setBrightness(brightness);
1163 resetMatch();
1164 mode = MODE_MATCH;
1165 markDisplayDirty();
1166
1167 lastTickMs = millis();
1168 lastActivityMs = lastTickMs;
1169 updateRtcStatusCache(true);
1170 drawDisplay();
1171 displayDirty = false;
1172 lastDisplayMs = lastTickMs;
1173 }
1174
1175 void loop() {
1176   handleTouch();
1177   handleButtons();
1178
1179   if (mode == MODE_MATCH) {
1180     updateClock();
1181   }
1182
1183   saveSettingsIfNeeded();
1184   maybeSleep();
1185
1186   uint32_t nowMs = millis();
1187   if (!displayDirty && nowMs - lastRtcStatusMs >= 1000UL) {
1188     bool rtcChanged = updateRtcStatusCache(false);
1189     if (rtcChanged && (mode == MODE_MATCH || mode == MODE_MATCH_OVER || mode == MODE_TIMEOUT)) {
1190       drawStatusBar();
1191     }
1192   }
1193
1194   bool timeDisplayDue = (clock_running && nowMs - lastDisplayMs >= DISPLAY_REFRESH_MS);
1195   if (displayDirty) {
1196     drawDisplay();
1197     lastDisplayMs = nowMs;
1198     displayDirty = false;
1199   } else if (timeDisplayDue) {
1200     drawTimerPanels();
1201     lastDisplayMs = nowMs;
1202   }
1203 }
1204
1205 // -----
1206 // Button handling
1207 // -----
1208 void handleButtons() {
1209   uint32_t nowMs = millis();
1210
1211   btnPlayerA.update(nowMs);
1212   btnPlayerB.update(nowMs);
1213   btnScoreAPlus.update(nowMs);
1214   btnScoreAMinus.update(nowMs);
1215   btnScoreBPlus.update(nowMs);
1216   btnScoreBMinus.update(nowMs);
1217   btnSelect.update(nowMs);
1218   btnUp.update(nowMs);
1219   btnDown.update(nowMs);
1220
1221   uint16_t expanderState = (pcf8575Address == 0) ? 0xFFFF : pcf8575Read(pcf8575Address);
1222   expanderBtnPlayerA.update(nowMs, expanderState);
1223   expanderBtnPlayerB.update(nowMs, expanderState);
1224   expanderBtnPlayPause.update(nowMs, expanderState);
1225
1226   bool gpioPlayerAPressed = btnPlayerA.wasPressed();
1227   bool gpioPlayerBPressed = btnPlayerB.wasPressed();
1228   bool expanderPlayerAPressed = expanderBtnPlayerA.wasPressed();
1229   bool expanderPlayerBPressed = expanderBtnPlayerB.wasPressed();
1230   bool expanderPlayPausePressed = expanderBtnPlayPause.wasPressed();
1231
1232   bool anyPress = gpioPlayerAPressed || gpioPlayerBPressed ||
1233                  expanderPlayerAPressed || expanderPlayerBPressed ||
1234                  expanderPlayPausePressed ||

```

```

1235         btnScoreAPlus.wasPressed() || btnScoreAMinus.wasPressed() ||
1236         btnScoreBPlus.wasPressed() || btnScoreBMinus.wasPressed() ||
1237         btnSelect.wasPressed() || btnUp.wasPressed() || btnDown.wasPressed();
1238
1239     if (anyPress) {
1240         wakeActivity();
1241         playButtonBeep();
1242     }
1243
1244     if (mode == MODE_MENU) {
1245         handleMenu();
1246         return;
1247     }
1248
1249     if (mode == MODE_NAME_EDIT) {
1250         if (btnSelect.wasPressed()) {
1251             mode = MODE_MATCH;
1252             editingNameFor = PLAYER_NONE;
1253             markDisplayDirty();
1254         }
1255         return;
1256     }
1257
1258     if (mode == MODE_LOG) {
1259         if (btnSelect.wasPressed()) {
1260             mode = MODE_MENU;
1261             markDisplayDirty();
1262         }
1263         return;
1264     }
1265
1266     if (mode == MODE_CONFIRM_END_MATCH) {
1267         if (btnSelect.wasPressed()) {
1268             mode = MODE_MATCH;
1269             markDisplayDirty();
1270         }
1271         return;
1272     }
1273
1274     // The menu/select button pauses or resumes menu access during a match.
1275     if (btnSelect.wasPressed()) {
1276         enterMenu();
1277         return;
1278     }
1279
1280     if (mode == MODE_MATCH) {
1281         if (expanderPlayPausePressed) {
1282             togglePlayPause();
1283             return;
1284         }
1285
1286         if (gpioPlayerAPressed) {
1287             if (clock_running && active_player == PLAYER_A) {
1288                 switchClockTo(PLAYER_B);
1289             } else {
1290                 switchClockTo(PLAYER_A);
1291             }
1292         }
1293         if (gpioPlayerBPressed) {
1294             switchClockTo(PLAYER_A);
1295         }
1296         if (expanderPlayerAPressed) {
1297             switchClockTo(PLAYER_B);
1298         }
1299         if (expanderPlayerBPressed) {
1300             switchClockTo(PLAYER_A);
1301         }
1302
1303         if (btnScoreAPlus.wasPressed()) {
1304             adjustScore(PLAYER_A, +1);
1305         }
1306         if (btnScoreAMinus.wasPressed()) {
1307             adjustScore(PLAYER_A, -1);
1308         }
1309         if (btnScoreBPlus.wasPressed()) {
1310             adjustScore(PLAYER_B, +1);
1311         }
1312         if (btnScoreBMinus.wasPressed()) {
1313             adjustScore(PLAYER_B, -1);
1314         }
1315     }
1316
1317     // After timeout or match over, select returns to menu so the device can reset.
1318     if ((mode == MODE_TIMEOUT || mode == MODE_MATCH_OVER) && btnSelect.wasPressed()) {
1319         enterMenu();
1320     }

```

```

1321 }
1322
1323 void handleTouch() {
1324     const uint32_t TOUCH_RELEASE_GRACE_MS = 140;
1325     const uint32_t TOUCH_REPEAT_GUARD_MS = 220;
1326     const int16_t TOUCH_SWIPE_THRESHOLD = 32;
1327
1328     TouchPoint point = readTouch();
1329     uint32_t nowMs = millis();
1330
1331     if (point.touched) {
1332         touchLastX = point.x;
1333         touchLastY = point.y;
1334         if (point.y < touchMinY) {
1335             touchMinY = point.y;
1336         }
1337         if (point.y > touchMaxY) {
1338             touchMaxY = point.y;
1339         }
1340         touchLastSeenMs = nowMs;
1341         lastTouchSeen = true;
1342         if (!touchWasDown) {
1343             touchWasDown = true;
1344             touchStartX = point.x;
1345             touchStartY = point.y;
1346             touchMinY = point.y;
1347             touchMaxY = point.y;
1348             touchStartMs = nowMs;
1349         }
1350         return;
1351     }
1352
1353     if (!touchWasDown) {
1354         return;
1355     }
1356
1357     // The GT911 can briefly report no touch while a finger is still moving.
1358     // Waiting a moment here keeps swipes from being split into tiny taps.
1359     if (nowMs - touchLastSeenMs < TOUCH_RELEASE_GRACE_MS) {
1360         return;
1361     }
1362
1363     touchWasDown = false;
1364     wakeActivity();
1365
1366     if (nowMs - touchLastHandledMs < TOUCH_REPEAT_GUARD_MS) {
1367         return;
1368     }
1369     touchLastHandledMs = nowMs;
1370
1371     if (mode == MODE_MENU || mode == MODE_LOG || mode == MODE_NAME_EDIT || mode == MODE_CLOCK_SET) {
1372         handleTouchTap(touchLastX, touchLastY);
1373         return;
1374     }
1375
1376     int16_t dx = (int16_t)touchLastX - (int16_t)touchStartX;
1377     int16_t upDistance = (int16_t)touchStartY - (int16_t)touchMinY;
1378     int16_t downDistance = (int16_t)touchMaxY - (int16_t)touchStartY;
1379     int16_t dy = (downDistance > upDistance) ? downDistance : -upDistance;
1380     if (abs(dy) >= TOUCH_SWIPE_THRESHOLD && abs(dy) > abs(dx)) {
1381         uint16_t gestureEndY = touchStartY;
1382         if (dy > 0) {
1383             uint32_t candidate = (uint32_t)touchStartY + (uint32_t)abs(dy);
1384             gestureEndY = (candidate >= PANEL_HEIGHT) ? PANEL_HEIGHT - 1 : (uint16_t)candidate;
1385         } else {
1386             gestureEndY = (touchStartY > abs(dy)) ? touchStartY - abs(dy) : 0;
1387         }
1388         handleTouchSwipe(touchStartX, touchStartY, touchLastX, gestureEndY);
1389     } else {
1390         handleTouchTap(touchLastX, touchLastY);
1391     }
1392 }
1393
1394 void handleTouchTap(uint16_t x, uint16_t y) {
1395     if (mode == MODE_NAME_EDIT) {
1396         handleNameEditorTap(x, y);
1397         return;
1398     }
1399
1400     if (mode == MODE_CLOCK_SET) {
1401         handleClockSettingsTap(x, y);
1402         return;
1403     }
1404
1405     if (mode == MODE_CONFIRM_END_MATCH) {
1406         handleEndMatchConfirmTap(x, y);

```

```

1407     return;
1408 }
1409
1410 if (mode == MODE_LOG) {
1411     if (LOG_BACK.containsPadded(x, y, 18, 14)) {
1412         mode = MODE_MENU;
1413         markDisplayDirty();
1414         return;
1415     }
1416     if (LOG_CLEAR.containsPadded(x, y, 18, 14)) {
1417         clearMatchLog();
1418         markDisplayDirty();
1419         return;
1420     }
1421     return;
1422 }
1423
1424 if (mode == MODE_MENU) {
1425     if (SETTINGS_BACK.containsPadded(x, y, 18, 14)) {
1426         mode = MODE_MATCH;
1427         markDisplayDirty();
1428         return;
1429     }
1430
1431     if (SETTINGS_MATCH_MINUS.containsPadded(x, y, 40, 18) ||
1432         SETTINGS_MATCH_ROW_MINUS.contains(x, y)) {
1433         currentMenuItem = MENU_MATCH_TARGET;
1434         changeMenuValue(-1);
1435         return;
1436     }
1437     if (SETTINGS_MATCH_PLUS.containsPadded(x, y, 40, 18) ||
1438         SETTINGS_MATCH_ROW_PLUS.contains(x, y)) {
1439         currentMenuItem = MENU_MATCH_TARGET;
1440         changeMenuValue(+1);
1441         return;
1442     }
1443     if (SETTINGS_TIME_MINUS.containsPadded(x, y, 40, 18) ||
1444         SETTINGS_TIME_ROW_MINUS.contains(x, y)) {
1445         currentMenuItem = MENU_INITIAL_TIME;
1446         changeMenuValue(-1);
1447         return;
1448     }
1449     if (SETTINGS_TIME_PLUS.containsPadded(x, y, 40, 18) ||
1450         SETTINGS_TIME_ROW_PLUS.contains(x, y)) {
1451         currentMenuItem = MENU_INITIAL_TIME;
1452         changeMenuValue(+1);
1453         return;
1454     }
1455     if (SETTINGS_DELAY_MINUS.containsPadded(x, y, 40, 18) ||
1456         SETTINGS_DELAY_ROW_MINUS.contains(x, y)) {
1457         currentMenuItem = MENU_DELAY;
1458         changeMenuValue(-1);
1459         return;
1460     }
1461     if (SETTINGS_DELAY_PLUS.containsPadded(x, y, 40, 18) ||
1462         SETTINGS_DELAY_ROW_PLUS.contains(x, y)) {
1463         currentMenuItem = MENU_DELAY;
1464         changeMenuValue(+1);
1465         return;
1466     }
1467     if (SETTINGS_SCOPE_GAME.containsPadded(x, y, 18, 10)) {
1468         timerScope = TIMER_BY_GAME;
1469         playerA_time_remaining = startingClockMs();
1470         playerB_time_remaining = startingClockMs();
1471         markSettingsDirty();
1472         markDisplayDirty();
1473         return;
1474     }
1475     if (SETTINGS_SCOPE_MATCH.containsPadded(x, y, 18, 10)) {
1476         timerScope = TIMER_BY_MATCH;
1477         playerA_time_remaining = startingClockMs();
1478         playerB_time_remaining = startingClockMs();
1479         markSettingsDirty();
1480         markDisplayDirty();
1481         return;
1482     }
1483     if (SETTINGS_MODE_SIMPLE.containsPadded(x, y, 12, 10)) {
1484         timingMode = SIMPLE_DELAY;
1485         markSettingsDirty();
1486         markDisplayDirty();
1487         return;
1488     }
1489     if (SETTINGS_MODE_BRONSTEIN.containsPadded(x, y, 12, 10)) {
1490         timingMode = BRONSTEIN_DELAY;
1491         markSettingsDirty();
1492         markDisplayDirty();

```

```

1493     return;
1494 }
1495 if (SETTINGS_MODE_FISCHER.containsPadded(x, y, 12, 10)) {
1496     timingMode = FISCHER_INCREMENT;
1497     markSettingsDirty();
1498     markDisplayDirty();
1499     return;
1500 }
1501 if (SETTINGS_LOG.containsPadded(x, y, 18, 14)) {
1502     mode = MODE_LOG;
1503     markDisplayDirty();
1504     return;
1505 }
1506 if (SETTINGS_CLOCK.containsPadded(x, y, 18, 14)) {
1507     openClockSettings();
1508     return;
1509 }
1510 if (SETTINGS_START.containsPadded(x, y, 18, 14)) {
1511     resetMatch();
1512     startNewGame();
1513     mode = MODE_MATCH;
1514     markDisplayDirty();
1515     return;
1516 }
1517 if (SETTINGS_END.containsPadded(x, y, 18, 14)) {
1518     resetMatch();
1519     mode = MODE_MATCH;
1520     markDisplayDirty();
1521     return;
1522 }
1523 return;
1524 }
1525
1526 bool timerAHit = REGION_TIMER_A.containsPadded(x, y, 12, 14) ||
1527     REGION_HEADER_A.containsPadded(x, y, 12, 10);
1528 bool timerBHit = REGION_TIMER_B.containsPadded(x, y, 12, 14) ||
1529     REGION_HEADER_B.containsPadded(x, y, 12, 10);
1530
1531 if (!clock_running && REGION_HEADER_A.containsPadded(x, y, 12, 10)) {
1532     openNameEditor(PPLAYER_A);
1533     return;
1534 }
1535
1536 if (!clock_running && REGION_HEADER_B.containsPadded(x, y, 12, 10)) {
1537     openNameEditor(PPLAYER_B);
1538     return;
1539 }
1540
1541 if (mode == MODE_MATCH && (REGION_SETTINGS.contains(x, y) || REGION_MATCH_TARGET.contains(x, y))) {
1542     enterMenu();
1543     return;
1544 }
1545
1546 if (HIT_PLAY_PAUSE.contains(x, y)) {
1547     togglePlayPause();
1548     return;
1549 }
1550
1551 if (HIT_END_GAME.contains(x, y)) {
1552     if (mode == MODE_TIMEOUT) {
1553         mode = MODE_MATCH;
1554         startNewGame();
1555         return;
1556     }
1557     if (game_in_progress || clock_running) {
1558         endCurrentGame();
1559     }
1560     return;
1561 }
1562
1563 if (HIT_END_MATCH.contains(x, y)) {
1564     if (mode == MODE_MATCH_OVER) {
1565         resetMatch();
1566         startNewGame();
1567     } else {
1568         pauseClock();
1569         mode = MODE_CONFIRM_END_MATCH;
1570         markDisplayDirty();
1571     }
1572     return;
1573 }
1574
1575 if (timerAHit) {
1576     switchClockTo(PPLAYER_B);
1577     return;
1578 }

```

```

1579
1580     if (timerBHit) {
1581         switchClockTo(PLAYER_A);
1582         return;
1583     }
1584
1585     if (!clock_running && REGION_SCORE_A.contains(x, y)) {
1586         int delta = (y < REGION_SCORE_A.y + (REGION_SCORE_A.h / 2)) ? +1 : -1;
1587         adjustScore(PLAYER_A, delta);
1588         return;
1589     }
1590
1591     if (!clock_running && REGION_SCORE_B.contains(x, y)) {
1592         int delta = (y < REGION_SCORE_B.y + (REGION_SCORE_B.h / 2)) ? +1 : -1;
1593         adjustScore(PLAYER_B, delta);
1594         return;
1595     }
1596 }
1597
1598 void handleTouchSwipe(uint16_t startX, uint16_t startY, uint16_t endX, uint16_t endY) {
1599     if ((mode != MODE_MATCH && mode != MODE_TIMEOUT) || clock_running) {
1600         return;
1601     }
1602
1603     int delta = (endY < startY) ? +1 : -1;
1604     bool scoreA = REGION_SCORE_A.contains(startX, startY);
1605     bool scoreB = REGION_SCORE_B.contains(startX, startY);
1606
1607     if (!scoreA && !scoreB) {
1608         return;
1609     }
1610
1611     adjustScore(scoreA ? PLAYER_A : PLAYER_B, delta);
1612     markDisplayDirty();
1613 }
1614
1615 void pauseClock() {
1616     if (!clock_running || active_player == PLAYER_NONE) {
1617         return;
1618     }
1619
1620     uint32_t nowMs = millis();
1621     pausedTurnElapsedMs = nowMs - turnStartMs;
1622     clock_running = false;
1623     lastTickMs = nowMs;
1624     markDisplayDirty();
1625 }
1626
1627 void resumeClock() {
1628     if (clock_running || active_player == PLAYER_NONE || matchWinner != PLAYER_NONE || timeoutPlayer != PLAYER_NONE) {
1629         return;
1630     }
1631
1632     uint32_t nowMs = millis();
1633     turnStartMs = nowMs - pausedTurnElapsedMs;
1634     lastTickMs = nowMs;
1635     clock_running = true;
1636     markDisplayDirty();
1637 }
1638
1639 void togglePlayPause() {
1640     if (mode != MODE_MATCH) {
1641         return;
1642     }
1643
1644     if (clock_running) {
1645         pauseClock();
1646     } else if (!game_in_progress) {
1647         startNewGame();
1648     } else {
1649         resumeClock();
1650     }
1651 }
1652
1653 void switchClockTo(Player nextPlayer) {
1654     uint32_t nowMs = millis();
1655
1656     if (mode != MODE_MATCH || matchWinner != PLAYER_NONE || timeoutPlayer != PLAYER_NONE) {
1657         return;
1658     }
1659
1660     if (!game_in_progress) {
1661         startNewGame();
1662     }
1663
1664     // Once the clock is running, only the active player's own large button

```

```

1665 // should end the turn. Presses on the inactive side are ignored so an
1666 // accidental bump cannot reset the current player's delay window.
1667 if (clock_running && active_player == nextPlayer) {
1668     return;
1669 }
1670
1671 if (clock_running && active_player != PLAYER_NONE && active_player != nextPlayer) {
1672     if (!applyTurnElapsed(active_player, nowMs)) {
1673         return;
1674     }
1675 }
1676
1677 active_player = nextPlayer;
1678 clock_running = true;
1679 turnStartMs = nowMs;
1680 pausedTurnElapsedMs = 0;
1681 lastTickMs = nowMs;
1682 markDisplayDirty();
1683 }
1684
1685 // -----
1686 // Clock behavior
1687 // -----
1688 void updateClock() {
1689     if (!clock_running || active_player == PLAYER_NONE) {
1690         lastTickMs = millis();
1691         return;
1692     }
1693
1694     uint32_t nowMs = millis();
1695     uint32_t elapsedMs = nowMs - lastTickMs;
1696     if (elapsedMs < 1000) {
1697         return;
1698     }
1699
1700     uint32_t wholeSeconds = elapsedMs / 1000;
1701     lastTickMs += wholeSeconds * 1000;
1702     match_elapsed_seconds += wholeSeconds;
1703
1704     if (visibleReserveMs(active_player, nowMs) == 0) {
1705         uint32_t *remaining = reserveFor(active_player);
1706         if (remaining != nullptr) {
1707             *remaining = 0;
1708         }
1709         clock_running = false;
1710         timeoutPlayer = active_player;
1711         active_player = PLAYER_NONE;
1712         if (crawford_active) {
1713             markCrawfordUsed();
1714         }
1715         game_in_progress = false;
1716         mode = MODE_TIMEOUT;
1717         playTimeoutBeep();
1718         markDisplayDirty();
1719         return;
1720     }
1721
1722     // Low-time warning. The critical zone beeps more often.
1723     uint32_t visibleSeconds = visibleReserveMs(active_player, nowMs) / 1000UL;
1724     if (visibleSeconds <= LOW_TIME_WARNING_SECONDS) {
1725         uint32_t interval = (visibleSeconds <= CRITICAL_TIME_WARNING_SECONDS) ? 1000UL : 5000UL;
1726         if (nowMs - lastLowTimeBeepMs >= interval) {
1727             beep(2200, 45);
1728             lastLowTimeBeepMs = nowMs;
1729         }
1730     }
1731 }
1732
1733 void addIncrementTo(Player player) {
1734     if (increment_seconds == 0) {
1735         return;
1736     }
1737
1738     if (player == PLAYER_A) {
1739         playerA_time_remaining += (uint32_t)increment_seconds * 1000UL;
1740     } else if (player == PLAYER_B) {
1741         playerB_time_remaining += (uint32_t)increment_seconds * 1000UL;
1742     }
1743     markDisplayDirty();
1744 }
1745
1746 bool applyTurnElapsed(Player player, uint32_t nowMs) {
1747     uint32_t *reserve = reserveFor(player);
1748     if (reserve == nullptr) {
1749         return true;
1750     }

```

```

1751     uint32_t elapsedMs = nowMs - turnStartMs;
1752     uint32_t delayOrIncrementMs = (uint32_t)delay_seconds * 1000UL;
1753     uint32_t deductionMs = 0;
1754
1755     if (timingMode == FISCHER_INCREMENT) {
1756         deductionMs = elapsedMs;
1757     } else {
1758         deductionMs = (elapsedMs > delayOrIncrementMs) ? (elapsedMs - delayOrIncrementMs) : 0;
1759     }
1760
1761     if (deductionMs >= *reserve) {
1762         *reserve = 0;
1763         clock_running = false;
1764         timeoutPlayer = player;
1765         active_player = PLAYER_NONE;
1766         if (crawford_active) {
1767             markCrawfordUsed();
1768         }
1769         game_in_progress = false;
1770         mode = MODE_TIMEOUT;
1771         playTimeoutBeep();
1772         markDisplayDirty();
1773         return false;
1774     }
1775
1776     *reserve -= deductionMs;
1777     if (timingMode == FISCHER_INCREMENT) {
1778         *reserve += delayOrIncrementMs;
1779     }
1780     return true;
1781 }
1782
1783 uint32_t visibleReserveMs(Player player, uint32_t nowMs) {
1784     uint32_t *reserve = reserveFor(player);
1785     if (reserve == nullptr) {
1786         return 0;
1787     }
1788     if (!clock_running || active_player != player) {
1789         return *reserve;
1790     }
1791
1792     uint32_t elapsedMs = nowMs - turnStartMs;
1793     uint32_t delayOrIncrementMs = (uint32_t)delay_seconds * 1000UL;
1794     uint32_t deductionMs = 0;
1795
1796     if (timingMode == SIMPLE_DELAY) {
1797         deductionMs = (elapsedMs > delayOrIncrementMs) ? (elapsedMs - delayOrIncrementMs) : 0;
1798     } else {
1799         deductionMs = elapsedMs;
1800     }
1801
1802     return (deductionMs >= *reserve) ? 0 : (*reserve - deductionMs);
1803 }
1804
1805 uint32_t *reserveFor(Player player) {
1806     if (player == PLAYER_A) {
1807         return &playerA_time_remaining;
1808     }
1809     if (player == PLAYER_B) {
1810         return &playerB_time_remaining;
1811     }
1812     return nullptr;
1813 }
1814
1815 uint32_t matchClockSeconds() {
1816     return initial_time_seconds * (uint32_t)match_target;
1817 }
1818
1819 uint32_t startingClockMs() {
1820     uint32_t seconds = (timerScope == TIMER_BY_MATCH) ? matchClockSeconds() : initial_time_seconds;
1821     return seconds * 1000UL;
1822 }
1823
1824 // -----
1825 // Score and match state
1826 // -----
1827 void adjustScore(Player player, int delta) {
1828     if ((mode != MODE_MATCH && mode != MODE_TIMEOUT) || clock_running) {
1829         return;
1830     }
1831 }
1832
1833 bool scorerIsOpponentOfCrawfordPlayer =
1834     (match_target > 1) &&
1835     ((player == PLAYER_A && playerB_score == match_target - 1) ||
1836      (player == PLAYER_B && playerA_score == match_target - 1));

```

```

1837
1838 if (delta > 0 && (crawford_active || scorerIsOpponentOfCrawfordPlayer)) {
1839     // Any score recorded during the Crawford game ends Crawford immediately.
1840     // Also catch the real scoreboard pattern: the one-away player did not
1841     // score, so the opponent's score changed and the next game is post-Crawford.
1842     markCrawfordUsed();
1843 }
1844
1845 if (delta > 0 && game_in_progress) {
1846     // A positive score adjustment is treated as the end of the current game.
1847     // Manual corrections with minus buttons do not advance Crawford state.
1848     endCurrentGame();
1849 }
1850
1851 int *score = (player == PLAYER_A) ? &playerA_score : &playerB_score;
1852 *score += delta;
1853 if (*score < 0) {
1854     *score = 0;
1855 }
1856 if (*score > match_target) {
1857     *score = match_target;
1858 }
1859
1860 checkMatchState();
1861 markDisplayDirty();
1862
1863 if (matchWinner != PLAYER_NONE || mode == MODE_MATCH_OVER) {
1864     return;
1865 }
1866
1867 // New games are counted only when startNewGame() is explicitly called.
1868 // Scoring corrections should not advance the game counter.
1869 }
1870
1871 void checkMatchState() {
1872     if (playerA_score >= match_target || playerB_score >= match_target) {
1873         matchWinner = (playerA_score >= match_target) ? PLAYER_A : PLAYER_B;
1874         clock_running = false;
1875         active_player = PLAYER_NONE;
1876         game_in_progress = false;
1877         recordMatchLog();
1878         mode = MODE_MATCH_OVER;
1879         playMatchOverBeep();
1880         markDisplayDirty();
1881         return;
1882     }
1883
1884     if (crawford_used) {
1885         pending_crawford_next_game = false;
1886         if (!crawford_active) {
1887             post_crawford = true;
1888             cube_enabled = true;
1889         }
1890         return;
1891     }
1892
1893     // Crawford is only meaningful for match targets above 1.
1894     if (match_target > 1 && !crawford_used && !crawford_active) {
1895         bool someoneAtCrawfordScore =
1896             (playerA_score == match_target - 1) || (playerB_score == match_target - 1);
1897
1898         pending_crawford_next_game = someoneAtCrawfordScore;
1899     }
1900 }
1901
1902 void startNewGame() {
1903     if (mode == MODE_MATCH_OVER) {
1904         return;
1905     }
1906
1907     game_number++;
1908     game_in_progress = true;
1909     clock_running = false;
1910     active_player = PLAYER_NONE;
1911     timeoutPlayer = PLAYER_NONE;
1912     if (timerScope == TIMER_BY_GAME) {
1913         playerA_time_remaining = startingClockMs();
1914         playerB_time_remaining = startingClockMs();
1915     }
1916     pausedTurnElapsedMs = 0;
1917     lastLowTimeBeepMs = 0;
1918
1919     if (pending_crawford_next_game && !crawford_used) {
1920         crawford_active = true;
1921         post_crawford = false;
1922         cube_enabled = false;

```

```

1923     pending_crawford_next_game = false;
1924 } else if (crawford_used) {
1925     crawford_active = false;
1926     post_crawford = true;
1927     cube_enabled = true;
1928 } else {
1929     crawford_active = false;
1930     post_crawford = false;
1931     cube_enabled = true;
1932 }
1933
1934 lastTickMs = millis();
1935 turnStartMs = lastTickMs;
1936 markDisplayDirty();
1937 }
1938
1939 void endCurrentGame() {
1940     clock_running = false;
1941     active_player = PLAYER_NONE;
1942     pausedTurnElapsedMs = 0;
1943
1944     if (crawford_active) {
1945         markCrawfordUsed();
1946     }
1947
1948     game_in_progress = false;
1949     markDisplayDirty();
1950 }
1951
1952 void markCrawfordUsed() {
1953     crawford_active = false;
1954     crawford_used = true;
1955     post_crawford = true;
1956     pending_crawford_next_game = false;
1957     cube_enabled = true;
1958 }
1959
1960 void forceEndMatch() {
1961     clock_running = false;
1962     active_player = PLAYER_NONE;
1963     pausedTurnElapsedMs = 0;
1964     game_in_progress = false;
1965
1966     matchWinner = (playerA_score == playerB_score) ? PLAYER_NONE :
1967                   ((playerA_score > playerB_score) ? PLAYER_A : PLAYER_B);
1968     recordMatchLog();
1969     mode = MODE_MATCH_OVER;
1970     playMatchOverBeep();
1971     markDisplayDirty();
1972 }
1973
1974 void resetMatch() {
1975     playerA_score = 0;
1976     playerB_score = 0;
1977     playerA_time_remaining = startingClockMs();
1978     playerB_time_remaining = startingClockMs();
1979     match_elapsed_seconds = 0;
1980     active_player = PLAYER_NONE;
1981     clock_running = false;
1982     pausedTurnElapsedMs = 0;
1983     crawford_used = false;
1984     crawford_active = false;
1985     post_crawford = false;
1986     cube_enabled = true;
1987     game_number = 0;
1988     timeoutPlayer = PLAYER_NONE;
1989     matchWinner = PLAYER_NONE;
1990     pending_crawford_next_game = false;
1991     game_in_progress = false;
1992     mode = MODE_MATCH;
1993     markDisplayDirty();
1994 }
1995
1996 // -----
1997 // Menu behavior
1998 // -----
1999 void enterMenu() {
2000     clock_running = false;
2001     active_player = PLAYER_NONE;
2002     mode = MODE_MENU;
2003     markDisplayDirty();
2004 }
2005
2006 void handleMenu() {
2007     if (btnPlayerA.wasPressed() || btnUp.wasPressed()) {
2008         changeMenuValue(+1);

```

```

2009     }
2010
2011     if (btnPlayerB.wasPressed() || btnDown.wasPressed()) {
2012         changeMenuValue(-1);
2013     }
2014
2015     if (btnScoreAPlus.wasPressed() || btnScoreBPlus.wasPressed()) {
2016         nextMenuItem();
2017     }
2018
2019     if (btnScoreAMinus.wasPressed() || btnScoreBMinus.wasPressed()) {
2020         previousMenuItem();
2021     }
2022
2023     if (btnSelect.wasPressed()) {
2024         if (currentMenuItem == MENU_START_MATCH) {
2025             resetMatch();
2026             startNewGame();
2027         } else if (currentMenuItem == MENU_RESET_MATCH) {
2028             resetMatch();
2029             enterMenu();
2030         } else {
2031             nextMenuItem();
2032         }
2033     }
2034 }
2035
2036 void changeMenuValue(int delta) {
2037     switch (currentMenuItem) {
2038     case MENU_MATCH_TARGET: {
2039         int index = findCurrentTargetIndex();
2040         index += delta;
2041         if (index < 0) {
2042             index = MATCH_TARGET_COUNT - 1;
2043         }
2044         if (index >= MATCH_TARGET_COUNT) {
2045             index = 0;
2046         }
2047         match_target = MATCH_TARGETS[index];
2048         playerA_time_remaining = startingClockMs();
2049         playerB_time_remaining = startingClockMs();
2050         markSettingsDirty();
2051         break;
2052     }
2053
2054     case MENU_INITIAL_TIME: {
2055         int32_t next = (int32_t)initial_time_seconds + (delta * 30);
2056         if (next < 60) {
2057             next = 60;
2058         }
2059         if (next > 99L * 60L) {
2060             next = 99L * 60L;
2061         }
2062         initial_time_seconds = (uint32_t)next;
2063         playerA_time_remaining = startingClockMs();
2064         playerB_time_remaining = startingClockMs();
2065         markSettingsDirty();
2066         break;
2067     }
2068
2069     case MENU_DELAY: {
2070         int next = (int)delay_seconds + delta;
2071         if (next < 0) {
2072             next = 0;
2073         }
2074         if (next > 60) {
2075             next = 60;
2076         }
2077         delay_seconds = (uint8_t)next;
2078         markSettingsDirty();
2079         break;
2080     }
2081
2082     case MENU_INCREMENT: {
2083         int next = (int)increment_seconds + delta;
2084         if (next < 0) {
2085             next = 0;
2086         }
2087         if (next > 60) {
2088             next = 60;
2089         }
2090         increment_seconds = (uint8_t)next;
2091         markSettingsDirty();
2092         break;
2093     }
2094 }

```

```

2095     case MENU_BRIGHTNESS: {
2096         int next = (int)brightness + (delta * 15);
2097         if (next < 15) {
2098             next = 15;
2099         }
2100         if (next > 255) {
2101             next = 255;
2102         }
2103         brightness = (uint8_t)next;
2104         setBrightness(brightness);
2105         markSettingsDirty();
2106         break;
2107     }
2108
2109     default:
2110         break;
2111 }
2112 markDisplayDirty();
2113 }
2114
2115 void nextMenuItem() {
2116     currentMenuItem = (MenuItem)(((uint8_t)currentMenuItem + 1) % MENU_COUNT);
2117     markDisplayDirty();
2118 }
2119
2120 void previousMenuItem() {
2121     uint8_t item = (uint8_t)currentMenuItem;
2122     currentMenuItem = (MenuItem)((item == 0) ? (MENU_COUNT - 1) : (item - 1));
2123     markDisplayDirty();
2124 }
2125
2126 uint8_t findCurrentTargetIndex() {
2127     for (uint8_t i = 0; i < MATCH_TARGET_COUNT; i++) {
2128         if (MATCH_TARGETS[i] == match_target) {
2129             return i;
2130         }
2131     }
2132     return 4; // Default to match to 9.
2133 }
2134
2135 // -----
2136 // Display
2137 // -----
2138 void drawDisplay() {
2139     bool modeChanged = !displayModeKnown || mode != lastDrawMode;
2140     if (modeChanged) {
2141         tft.fillScreen(TFT_BACKGROUND);
2142     }
2143     displayModeKnown = true;
2144     lastDrawMode = mode;
2145
2146     if (mode == MODE_NAME_EDIT) {
2147         drawNameEditor();
2148         return;
2149     }
2150
2151     if (mode == MODE_LOG) {
2152         drawMatchLog();
2153         return;
2154     }
2155
2156     if (mode == MODE_CLOCK_SET) {
2157         drawClockSettings();
2158         return;
2159     }
2160
2161     if (mode == MODE_MENU) {
2162         tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2163         tft.setTextSize(3);
2164         tft.setCursor(248, 28);
2165         tft.print("SETTINGS");
2166
2167         tft.fillRect(SETTINGS_BACK.x, SETTINGS_BACK.y, SETTINGS_BACK.w, SETTINGS_BACK.h, TFT_PANEL);
2168         tft.drawRoundRect(SETTINGS_BACK.x, SETTINGS_BACK.y, SETTINGS_BACK.w, SETTINGS_BACK.h, 6, TFT_BORDER);
2169         tft.setTextSize(2);
2170         tft.setCursor(44, 38);
2171         tft.print("BACK");
2172
2173         tft.fillRect(52, 82, 696, 58, TFT_PANEL);
2174         tft.fillRect(52, 162, 696, 58, TFT_PANEL);
2175         tft.fillRect(52, 242, 696, 58, TFT_PANEL);
2176         tft.drawRoundRect(52, 82, 696, 58, 6, TFT_BORDER);
2177         tft.drawRoundRect(52, 162, 696, 58, 6, TFT_BORDER);
2178         tft.drawRoundRect(52, 242, 696, 58, 6, TFT_BORDER);
2179
2180         tft.setTextColor(TFT_TEXT, TFT_PANEL);

```

```

2181 tft.setTextSize(2);
2182 tft.setCursor(76, 102);
2183 tft.print("MATCH TARGET");
2184 tft.setCursor(536, 102);
2185 tft.printf("%d", match_target);
2186
2187 tft.setCursor(76, 182);
2188 tft.print("TIME / GAME");
2189 tft.setCursor(520, 182);
2190 tft.print(formatTime(initial_time_seconds));
2191 tft.setTextColor(TFT_MUTED, TFT_PANEL);
2192 tft.setTextSize(1);
2193 tft.setCursor(76, 207);
2194 tft.print("TOTAL ");
2195 tft.print(formatTime(matchClockSeconds()));
2196 tft.setTextColor(TFT_TEXT, TFT_PANEL);
2197 tft.setTextSize(2);
2198
2199 tft.setCursor(76, 262);
2200 tft.print("BONUS / DELAY");
2201 tft.setCursor(536, 262);
2202 tft.printf("%u", delay_seconds);
2203
2204 const Rect controls[] = {
2205     SETTINGS_MATCH_MINUS, SETTINGS_MATCH_PLUS,
2206     SETTINGS_TIME_MINUS, SETTINGS_TIME_PLUS,
2207     SETTINGS_DELAY_MINUS, SETTINGS_DELAY_PLUS
2208 };
2209 const char *symbols[] = {"-", "+", "-", "+", "-", "+"};
2210 for (uint8_t i = 0; i < 6; i++) {
2211     tft.fillRect(controls[i].x, controls[i].y, controls[i].w, controls[i].h, TFT_PANEL_ALT);
2212     tft.drawRoundRect(controls[i].x, controls[i].y, controls[i].w, controls[i].h, 6, TFT_BORDER);
2213     tft.setTextColor(TFT_TEXT, TFT_PANEL_ALT);
2214     tft.setTextSize(3);
2215     tft.setCursor(controls[i].x + 40, controls[i].y + 14);
2216     tft.print(symbols[i]);
2217 }
2218
2219 tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2220 tft.setTextSize(2);
2221 tft.setCursor(76, 315);
2222 tft.print("TIMER");
2223 tft.fillRect(SETTINGS_SCOPE_GAME.x, SETTINGS_SCOPE_GAME.y, SETTINGS_SCOPE_GAME.w, SETTINGS_SCOPE_GAME.h,
2224     timerScope == TIMER_BY_GAME ? TFT_OK : TFT_PANEL_ALT);
2225 tft.drawRoundRect(SETTINGS_SCOPE_GAME.x, SETTINGS_SCOPE_GAME.y, SETTINGS_SCOPE_GAME.w, SETTINGS_SCOPE_GAME.h, 6, TFT_BORDER);
2226 drawCenteredInRect("GAME", SETTINGS_SCOPE_GAME, 2, timerScope == TIMER_BY_GAME ? ILI9341_BLACK : TFT_TEXT,
2227     timerScope == TIMER_BY_GAME ? TFT_OK : TFT_PANEL_ALT);
2228 tft.fillRect(SETTINGS_SCOPE_MATCH.x, SETTINGS_SCOPE_MATCH.y, SETTINGS_SCOPE_MATCH.w, SETTINGS_SCOPE_MATCH.h,
2229     timerScope == TIMER_BY_MATCH ? TFT_OK : TFT_PANEL_ALT);
2230 tft.drawRoundRect(SETTINGS_SCOPE_MATCH.x, SETTINGS_SCOPE_MATCH.y, SETTINGS_SCOPE_MATCH.w, SETTINGS_SCOPE_MATCH.h, 6, TFT_BORDER);
2231 drawCenteredInRect("MATCH", SETTINGS_SCOPE_MATCH, 2, timerScope == TIMER_BY_MATCH ? ILI9341_BLACK : TFT_TEXT,
2232     timerScope == TIMER_BY_MATCH ? TFT_OK : TFT_PANEL_ALT);
2233
2234 const Rect modeButtons[] = {SETTINGS_MODE_SIMPLE, SETTINGS_MODE_BRONSTEIN, SETTINGS_MODE_FISCHER};
2235 const char *modeLabels[] = {"SIMPLE", "BRONSTEIN", "FISCHER"};
2236 for (uint8_t i = 0; i < 3; i++) {
2237     bool selected = (uint8_t)timingMode == i;
2238     tft.fillRect(modeButtons[i].x, modeButtons[i].y, modeButtons[i].w, modeButtons[i].h,
2239         selected ? TFT_OK : TFT_PANEL_ALT);
2240     tft.drawRoundRect(modeButtons[i].x, modeButtons[i].y, modeButtons[i].w, modeButtons[i].h, 6, TFT_BORDER);
2241     drawCenteredInRect(modeLabels[i], modeButtons[i], 2, selected ? ILI9341_BLACK : TFT_TEXT,
2242         selected ? TFT_OK : TFT_PANEL_ALT);
2243 }
2244
2245 tft.fillRect(SETTINGS_LOG.x, SETTINGS_LOG.y, SETTINGS_LOG.w, SETTINGS_LOG.h, TFT_PANEL_ALT);
2246 tft.drawRoundRect(SETTINGS_LOG.x, SETTINGS_LOG.y, SETTINGS_LOG.w, SETTINGS_LOG.h, 6, TFT_BORDER);
2247 drawCenteredInRect("MATCH LOG", SETTINGS_LOG, 2, TFT_TEXT, TFT_PANEL_ALT);
2248
2249 tft.fillRect(SETTINGS_CLOCK.x, SETTINGS_CLOCK.y, SETTINGS_CLOCK.w, SETTINGS_CLOCK.h, TFT_PANEL_ALT);
2250 tft.drawRoundRect(SETTINGS_CLOCK.x, SETTINGS_CLOCK.y, SETTINGS_CLOCK.w, SETTINGS_CLOCK.h, 6, TFT_BORDER);
2251 drawCenteredInRect("CLOCK", SETTINGS_CLOCK, 2, TFT_TEXT, TFT_PANEL_ALT);
2252
2253 tft.fillRect(SETTINGS_START.x, SETTINGS_START.y, SETTINGS_START.w, SETTINGS_START.h, TFT_OK);
2254 tft.fillRect(SETTINGS_END.x, SETTINGS_END.y, SETTINGS_END.w, SETTINGS_END.h, TFT_DANGER);
2255 tft.drawRoundRect(SETTINGS_START.x, SETTINGS_START.y, SETTINGS_START.w, SETTINGS_START.h, 6, TFT_BORDER);
2256 tft.drawRoundRect(SETTINGS_END.x, SETTINGS_END.y, SETTINGS_END.w, SETTINGS_END.h, 6, TFT_BORDER);
2257 drawCenteredInRect("START MATCH", SETTINGS_START, 2, ILI9341_BLACK, TFT_OK);
2258 drawCenteredInRect("END MATCH", SETTINGS_END, 2, TFT_TEXT, TFT_DANGER);
2259
2260 return;
2261 }
2262
2263 drawStatusBar();
2264
2265 bool winnerA = (mode == MODE_MATCH_OVER && matchWinner == PLAYER_A);
2266 bool winnerB = (mode == MODE_MATCH_OVER && matchWinner == PLAYER_B);

```

```

2267 uint16_t headerABg = winnerA ? ILI9341_WHITE : TFT_PANEL;
2268 uint16_t headerBBg = winnerB ? ILI9341_WHITE : TFT_PANEL;
2269 uint16_t headerAText = winnerA ? TFT_TEXT_DARK : TFT_TEXT;
2270 uint16_t headerBText = winnerB ? TFT_TEXT_DARK : TFT_TEXT;
2271
2272 tft.fillRect(REGION_HEADER_A.x, REGION_HEADER_A.y, REGION_HEADER_A.w, REGION_HEADER_A.h, headerABg);
2273 tft.fillRect(REGION_HEADER_B.x, REGION_HEADER_B.y, REGION_HEADER_B.w, REGION_HEADER_B.h, headerBBg);
2274 drawTimerPanel(PYER_A, REGION_TIMER_A);
2275 tft.fillRect(REGION_PLAY_PAUSE.x, REGION_PLAY_PAUSE.y, REGION_PLAY_PAUSE.w, REGION_PLAY_PAUSE.h, TFT_PANEL);
2276 tft.fillRect(REGION_END_GAME.x, REGION_END_GAME.y, REGION_END_GAME.w, REGION_END_GAME.h, TFT_PANEL);
2277 tft.fillRect(REGION_END_MATCH.x, REGION_END_MATCH.y, REGION_END_MATCH.w, REGION_END_MATCH.h,
2278     mode == MODE_MATCH_OVER ? TFT_DANGER : TFT_PANEL);
2279 tft.fillRect(REGION_SCORE_A.x, REGION_SCORE_A.y, REGION_SCORE_A.w, REGION_SCORE_A.h, playerA_color);
2280 tft.fillRect(REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y, REGION_MATCH_TARGET.w, REGION_MATCH_TARGET.h, TFT_PANEL);
2281 tft.fillRect(REGION_SETTINGS.x, REGION_SETTINGS.y, REGION_SETTINGS.w, REGION_SETTINGS.h, TFT_PANEL);
2282 tft.fillRect(REGION_SCORE_B.x, REGION_SCORE_B.y, REGION_SCORE_B.w, REGION_SCORE_B.h, playerB_color);
2283
2284 tft.drawRoundRect(REGION_HEADER_A.x, REGION_HEADER_A.y, REGION_HEADER_A.w, REGION_HEADER_A.h, 4, TFT_ACCENT_A);
2285 tft.drawRoundRect(REGION_HEADER_B.x, REGION_HEADER_B.y, REGION_HEADER_B.w, REGION_HEADER_B.h, 4, TFT_ACCENT_B);
2286 tft.drawRoundRect(REGION_PLAY_PAUSE.x, REGION_PLAY_PAUSE.y, REGION_PLAY_PAUSE.w, REGION_PLAY_PAUSE.h, 6, TFT_BUTTON_BORDER);
2287 tft.drawRoundRect(REGION_END_GAME.x, REGION_END_GAME.y, REGION_END_GAME.w, REGION_END_GAME.h, 6, TFT_BUTTON_BORDER);
2288 tft.drawRoundRect(REGION_END_MATCH.x, REGION_END_MATCH.y, REGION_END_MATCH.w, REGION_END_MATCH.h, 6, TFT_BUTTON_BORDER);
2289 drawTimerPanel(PYER_B, REGION_TIMER_B);
2290 tft.drawRoundRect(REGION_SCORE_A.x, REGION_SCORE_A.y, REGION_SCORE_A.w, REGION_SCORE_A.h, 6, TFT_ACCENT_A);
2291 tft.drawRoundRect(REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y, REGION_MATCH_TARGET.w, REGION_MATCH_TARGET.h, 6, TFT_BUTTON_BORDER);
2292 tft.drawRoundRect(REGION_SETTINGS.x, REGION_SETTINGS.y, REGION_SETTINGS.w, REGION_SETTINGS.h, 6, TFT_BUTTON_BORDER);
2293 tft.drawRoundRect(REGION_SCORE_B.x, REGION_SCORE_B.y, REGION_SCORE_B.w, REGION_SCORE_B.h, 6, TFT_ACCENT_B);
2294
2295 tft.setTextSize(2);
2296 tft.setTextColor(TFT_TEXT, TFT_PANEL);
2297 drawCenteredInRect(playerA_name, REGION_HEADER_A, playerA_name.length() > 9 ? 1 : 2, headerAText, headerABg);
2298 drawCenteredInRect(playerB_name, REGION_HEADER_B, playerB_name.length() > 9 ? 1 : 2, headerBText, headerBBg);
2299
2300 drawCenteredInRect(playPauseLabel(), REGION_PLAY_PAUSE, 2, TFT_TEXT, TFT_PANEL);
2301
2302 drawCenteredInRect(mode == MODE_TIMEOUT ? "New Game" : "End Game", REGION_END_GAME, 2, TFT_TEXT, TFT_PANEL);
2303 drawCenteredInRect(mode == MODE_MATCH_OVER ? "Reset" : "End Match", REGION_END_MATCH, 2, TFT_TEXT, mode == MODE_MATCH_OVER ? TFT_DANGER
    ↪ : TFT_PANEL);
2304 tft.setTextColor(TFT_TEXT, TFT_PANEL);
2305
2306 drawScoreInRect(playerA_score, REGION_SCORE_A, textColorForPlayerColor(playerA_color), playerA_color);
2307 drawScoreInRect(playerB_score, REGION_SCORE_B, textColorForPlayerColor(playerB_color), playerB_color);
2308
2309 drawCenteredInRect("Match to:", {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 22, REGION_MATCH_TARGET.w, 26}, 2, TFT_TEXT, TFT_PANEL)
    ↪ ;
2310 drawMatchTargetInRect(match_target, {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 56, REGION_MATCH_TARGET.w, 74}, TFT_TEXT, TFT_PANEL
    ↪ );
2311 drawCenteredInRect(String("Game ") + String(game_number), {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 136, REGION_MATCH_TARGET.w,
    ↪ 28}, 2, TFT_TEXT, TFT_PANEL);
2312 String stateLabel = "";
2313 uint16_t stateColor = TFT_MAGENTA;
2314 if (mode == MODE_MATCH_OVER) {
2315     stateLabel = "Match Ended";
2316     stateColor = TFT_WARN;
2317 } else if (crawford_active || pending_crawford_next_game) {
2318     stateLabel = "CRAWFORD";
2319 } else if (post_crawford || crawford_used) {
2320     stateLabel = "POST CRAWFORD";
2321 }
2322 drawCenteredInRect(stateLabel, {REGION_MATCH_TARGET.x, REGION_MATCH_TARGET.y + 166, REGION_MATCH_TARGET.w, 30}, 2, stateColor,
    ↪ TFT_PANEL);
2323
2324 tft.setTextColor(TFT_TEXT, TFT_PANEL);
2325 drawCenteredInRect("SETTINGS", REGION_SETTINGS, 2, TFT_TEXT, TFT_PANEL);
2326
2327 if (mode == MODE_TIMEOUT) {
2328     drawTimeoutBanner();
2329 }
2330
2331 if (mode == MODE_CONFIRM_END_MATCH) {
2332     drawEndMatchConfirm();
2333 }
2334
2335 }
2336
2337 void drawMatchWinnerBanner() {
2338     const Rect banner = {6, 204, 788, 260};
2339     const Rect nameLine = {24, 270, 752, 76};
2340     const Rect winnerLine = {24, 348, 752, 76};
2341     String winnerName = (matchWinner == PLAYER_A) ? playerA_name : playerB_name;
2342
2343     tft.fillRect(banner.x, banner.y, banner.w, banner.h, TFT_PANEL);
2344     tft.drawRoundRect(banner.x, banner.y, banner.w, banner.h, 6, TFT_OK);
2345     drawCenteredInRect(winnerName, nameLine, winnerName.length() > 8 ? 6 : 7, TFT_TEXT, TFT_PANEL);
2346     drawCenteredInRect("IS THE WINNER", winnerLine, 5, TFT_OK, TFT_PANEL);
2347 }

```

```

2348
2349 void drawTimeoutBanner() {
2350     const Rect banner = {42, 224, 716, 174};
2351     Player loser = timeoutPlayer;
2352     String loserName = (loser == PLAYER_A) ? playerA_name : playerB_name;
2353
2354     tft.fillRect(banner.x, banner.y, banner.w, banner.h, TFT_PANEL);
2355     tft.drawRoundRect(banner.x, banner.y, banner.w, banner.h, 6, TFT_DANGER);
2356     drawCenteredInRect(loserName, {banner.x + 16, banner.y + 24, banner.w - 32, 48}, loserName.length() > 8 ? 4 : 5, TFT_TEXT, TFT_PANEL);
2357     drawCenteredInRect("RAN OUT OF TIME", {banner.x + 16, banner.y + 80, banner.w - 32, 36}, 3, TFT_DANGER, TFT_PANEL);
2358     drawCenteredInRect("LOSES ONE POINT", {banner.x + 16, banner.y + 122, banner.w - 32, 34}, 3, TFT_WARN, TFT_PANEL);
2359 }
2360
2361 void drawEndMatchConfirm() {
2362     const Rect dialog = {118, 132, 564, 236};
2363
2364     tft.fillRect(dialog.x, dialog.y, dialog.w, dialog.h, TFT_PANEL);
2365     tft.drawRoundRect(dialog.x, dialog.y, dialog.w, dialog.h, 8, TFT_BUTTON_BORDER);
2366
2367     drawCenteredInRect("END MATCH?", {dialog.x, dialog.y + 28, dialog.w, 34}, 3, TFT_WARN, TFT_PANEL);
2368     drawCenteredInRect("This will finish the match", {dialog.x, dialog.y + 84, dialog.w, 24}, 2, TFT_TEXT, TFT_PANEL);
2369     drawCenteredInRect("and record the current leader.", {dialog.x, dialog.y + 112, dialog.w, 24}, 2, TFT_TEXT, TFT_PANEL);
2370
2371     tft.fillRect(CONFIRM_NO.x, CONFIRM_NO.y, CONFIRM_NO.w, CONFIRM_NO.h, TFT_PANEL_ALT);
2372     tft.drawRoundRect(CONFIRM_NO.x, CONFIRM_NO.y, CONFIRM_NO.w, CONFIRM_NO.h, 6, TFT_BUTTON_BORDER);
2373     drawCenteredInRect("NO", CONFIRM_NO, 3, TFT_TEXT, TFT_PANEL_ALT);
2374
2375     tft.fillRect(CONFIRM_YES.x, CONFIRM_YES.y, CONFIRM_YES.w, CONFIRM_YES.h, TFT_DANGER);
2376     tft.drawRoundRect(CONFIRM_YES.x, CONFIRM_YES.y, CONFIRM_YES.w, CONFIRM_YES.h, 6, TFT_BUTTON_BORDER);
2377     drawCenteredInRect("YES", CONFIRM_YES, 3, TFT_TEXT, TFT_DANGER);
2378 }
2379
2380 void drawMatchLog() {
2381     tft.fillScreen(TFT_BACKGROUND);
2382
2383     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2384     tft.setTextSize(3);
2385     tft.setCursor(286, 28);
2386     tft.print("MATCH LOG");
2387
2388     tft.fillRect(LOG_BACK.x, LOG_BACK.y, LOG_BACK.w, LOG_BACK.h, TFT_PANEL);
2389     tft.drawRoundRect(LOG_BACK.x, LOG_BACK.y, LOG_BACK.w, LOG_BACK.h, 6, TFT_BORDER);
2390     drawCenteredInRect("BACK", LOG_BACK, 2, TFT_TEXT, TFT_PANEL);
2391
2392     tft.fillRect(LOG_CLEAR.x, LOG_CLEAR.y, LOG_CLEAR.w, LOG_CLEAR.h, TFT_DANGER);
2393     tft.drawRoundRect(LOG_CLEAR.x, LOG_CLEAR.y, LOG_CLEAR.w, LOG_CLEAR.h, 6, TFT_BORDER);
2394     drawCenteredInRect("CLEAR", LOG_CLEAR, 2, TFT_TEXT, TFT_DANGER);
2395
2396     if (matchLogCount == 0) {
2397         drawCenteredInRect("NO MATCHES YET", {120, 210, 560, 70}, 3, TFT_MUTED, TFT_BACKGROUND);
2398         return;
2399     }
2400
2401     for (uint8_t i = 0; i < matchLogCount && i < MATCH_LOG_MAX_ENTRIES; i++) {
2402         const int16_t y = 88 + (i * 46);
2403         const Rect row = {34, y, 732, 40};
2404         String date = matchLogPart(matchLog[i], 0);
2405         String nameA = matchLogPart(matchLog[i], 1);
2406         String nameB = matchLogPart(matchLog[i], 2);
2407         String score = matchLogPart(matchLog[i], 3);
2408         String names = nameA + " VS " + nameB;
2409         String scoreLine = "SCORE " + score;
2410
2411         tft.fillRect(row.x, row.y, row.w, row.h, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
2412         tft.drawRoundRect(row.x, row.y, row.w, row.h, 4, TFT_BORDER);
2413         tft.setTextColor(TFT_MUTED, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
2414         tft.setTextSize(1);
2415         tft.setCursor(row.x + 12, row.y + 8);
2416         tft.print(date);
2417         tft.setCursor(row.x + 152, row.y + 8);
2418         tft.print(names);
2419         tft.setTextColor(TFT_TEXT, (i % 2 == 0) ? TFT_PANEL : TFT_PANEL_ALT);
2420         tft.setTextSize(2);
2421         tft.setCursor(row.x + 560, row.y + 12);
2422         tft.print(scoreLine);
2423     }
2424 }
2425
2426 void drawClockSettings() {
2427     tft.fillScreen(TFT_BACKGROUND);
2428
2429     tft.setTextColor(TFT_TEXT, TFT_BACKGROUND);
2430     tft.setTextSize(3);
2431     tft.setCursor(248, 28);
2432     tft.print("CLOCK SET");
2433 }

```

```

2434 tft.fillRect(CLOCK_BACK.x, CLOCK_BACK.y, CLOCK_BACK.w, CLOCK_BACK.h, TFT_PANEL);
2435 tft.drawRoundRect(CLOCK_BACK.x, CLOCK_BACK.y, CLOCK_BACK.w, CLOCK_BACK.h, 6, TFT_BORDER);
2436 drawCenteredInRect("BACK", CLOCK_BACK, 2, TFT_TEXT, TFT_PANEL);
2437
2438 const Rect rows[] = {
2439     {52, 82, 696, 48},
2440     {52, 140, 696, 48},
2441     {52, 198, 696, 48},
2442     {52, 256, 696, 48},
2443     {52, 314, 696, 48}
2444 };
2445 const Rect minusButtons[] = {
2446     CLOCK_YEAR_MINUS,
2447     CLOCK_MONTH_MINUS,
2448     CLOCK_DAY_MINUS,
2449     CLOCK_HOUR_MINUS,
2450     CLOCK_MINUTE_MINUS
2451 };
2452 const Rect plusButtons[] = {
2453     CLOCK_YEAR_PLUS,
2454     CLOCK_MONTH_PLUS,
2455     CLOCK_DAY_PLUS,
2456     CLOCK_HOUR_PLUS,
2457     CLOCK_MINUTE_PLUS
2458 };
2459 const char *labels[] = {"YEAR", "MONTH", "DAY", "HOUR", "MINUTE"};
2460 uint16_t values[] = {editingRtcYear, editingRtcMonth, editingRtcDay, editingRtcHour, editingRtcMinute};
2461
2462 for (uint8_t i = 0; i < 5; i++) {
2463     tft.fillRect(rows[i].x, rows[i].y, rows[i].w, rows[i].h, TFT_PANEL);
2464     tft.drawRoundRect(rows[i].x, rows[i].y, rows[i].w, rows[i].h, 6, TFT_BORDER);
2465     tft.setTextColor(TFT_TEXT, TFT_PANEL);
2466     tft.setTextSize(2);
2467     tft.setCursor(76, rows[i].y + 16);
2468     tft.print(labels[i]);
2469     tft.setCursor(i == 0 ? 514 : 536, rows[i].y + 16);
2470     if (i == 0) {
2471         tft.printf("%u", values[i]);
2472     } else {
2473         tft.printf("%02u", values[i]);
2474     }
2475
2476     tft.fillRect(minusButtons[i].x, minusButtons[i].y, minusButtons[i].w, minusButtons[i].h, TFT_PANEL_ALT);
2477     tft.drawRoundRect(minusButtons[i].x, minusButtons[i].y, minusButtons[i].w, minusButtons[i].h, 6, TFT_BORDER);
2478     drawCenteredInRect("-", minusButtons[i], 3, TFT_TEXT, TFT_PANEL_ALT);
2479     tft.fillRect(plusButtons[i].x, plusButtons[i].y, plusButtons[i].w, plusButtons[i].h, TFT_PANEL_ALT);
2480     tft.drawRoundRect(plusButtons[i].x, plusButtons[i].y, plusButtons[i].w, plusButtons[i].h, 6, TFT_BORDER);
2481     drawCenteredInRect("+", plusButtons[i], 3, TFT_TEXT, TFT_PANEL_ALT);
2482 }
2483
2484 tft.setTextColor(ds3231Found ? TFT_MUTED : TFT_DANGER, TFT_BACKGROUND);
2485 tft.setTextSize(1);
2486 tft.setCursor(72, 380);
2487 tft.print(ds3231Found ? "SAVE writes this date/time to the DS3231 RTC." : "DS3231 RTC not detected on I2C address 0x68.");
2488
2489 tft.fillRect(CLOCK_SAVE.x, CLOCK_SAVE.y, CLOCK_SAVE.w, CLOCK_SAVE.h, ds3231Found ? TFT_OK : TFT_PANEL_ALT);
2490 tft.drawRoundRect(CLOCK_SAVE.x, CLOCK_SAVE.y, CLOCK_SAVE.w, CLOCK_SAVE.h, 6, TFT_BORDER);
2491 drawCenteredInRect("SAVE", CLOCK_SAVE, 2, ds3231Found ? ILI9341_BLACK : TFT_MUTED, ds3231Found ? TFT_OK : TFT_PANEL_ALT);
2492 }
2493
2494 void drawNameEditor() {
2495     const Rect title = {70, 24, 660, 42};
2496     const Rect input = {92, 78, 616, 64};
2497     const Rect cancel = {72, 420, 168, 44};
2498     const Rect clearKey = {258, 420, 168, 44};
2499     const Rect ok = {560, 420, 168, 44};
2500     const char *rows[] = {"ABCDEFGHJIJ", "KLMNOPQRST", "UVWXYZ"};
2501     const int16_t rowY[] = {148, 208, 268};
2502     const int16_t keyW = 66;
2503     const int16_t keyH = 44;
2504     const int16_t gap = 8;
2505     const int16_t swatchY = 326;
2506     const int16_t swatchSize = 36;
2507     const int16_t swatchGap = 14;
2508
2509     tft.fillScreen(TFT_BACKGROUND);
2510     tft.fillRect(48, 14, 704, 452, TFT_PANEL);
2511     tft.drawRoundRect(48, 14, 704, 452, 8, TFT_BORDER);
2512     drawCenteredInRect(editingNameFor == PLAYER_A ? "PLAYER A NAME" : "PLAYER B NAME", title, 3, TFT_TEXT, TFT_PANEL);
2513
2514     tft.fillRect(input.x, input.y, input.w, input.h, TFT_BACKGROUND);
2515     tft.drawRoundRect(input.x, input.y, input.w, input.h, 6, TFT_OK);
2516     drawCenteredInRect(editingName.length() == 0 ? " " : editingName, input, 4, TFT_TEXT, TFT_BACKGROUND);
2517
2518     for (uint8_t r = 0; r < 3; r++) {
2519         uint8_t count = strlen(rows[r]);

```

```

2520 int16_t startX = (PANEL_WIDTH - ((count * keyW) + ((count - 1) * gap))) / 2;
2521 for (uint8_t i = 0; i < count; i++) {
2522     Rect key = {startX + (int16_t)i * (keyW + gap), rowY[r], keyW, keyH};
2523     tft.fillRect(key.x, key.y, key.w, key.h, TFT_PANEL_ALT);
2524     tft.drawRoundRect(key.x, key.y, key.w, key.h, 6, TFT_BORDER);
2525     String label;
2526     label += rows[r][i];
2527     drawCenteredInRect(label, key, 3, TFT_TEXT, TFT_PANEL_ALT);
2528 }
2529 }
2530
2531 for (uint8_t i = 0; i < playerColorOptionCount(); i++) {
2532     int16_t startX = (PANEL_WIDTH - (playerColorOptionCount() * swatchSize + (playerColorOptionCount() - 1) * swatchGap)) / 2;
2533     Rect swatch = {startX + (int16_t)i * (swatchSize + swatchGap), swatchY, swatchSize, swatchSize};
2534     uint16_t color = playerColorOption(i);
2535     tft.fillRect(swatch.x, swatch.y, swatch.w, swatch.h, color);
2536     tft.drawRoundRect(swatch.x, swatch.y, swatch.w, swatch.h, 5, color == editingPlayerColor ? TFT_OK : TFT_BORDER);
2537     if (color == PLAYER_COLOR_WHITE) {
2538         tft.drawRoundRect(swatch.x + 3, swatch.y + 3, swatch.w - 6, swatch.h - 6, 4, TFT_MUTED);
2539     }
2540 }
2541
2542 const Rect spaceKey = {166, 372, 244, 38};
2543 const Rect delKey = {430, 372, 204, 38};
2544 tft.fillRect(spaceKey.x, spaceKey.y, spaceKey.w, spaceKey.h, TFT_PANEL_ALT);
2545 tft.drawRoundRect(spaceKey.x, spaceKey.y, spaceKey.w, spaceKey.h, 6, TFT_BORDER);
2546 drawCenteredInRect("SPACE", spaceKey, 2, TFT_TEXT, TFT_PANEL_ALT);
2547 tft.fillRect(delKey.x, delKey.y, delKey.w, delKey.h, TFT_PANEL_ALT);
2548 tft.drawRoundRect(delKey.x, delKey.y, delKey.w, delKey.h, 6, TFT_BORDER);
2549 drawCenteredInRect("DEL", delKey, 2, TFT_TEXT, TFT_PANEL_ALT);
2550
2551 tft.fillRect(cancel.x, cancel.y, cancel.w, cancel.h, TFT_DANGER);
2552 tft.drawRoundRect(cancel.x, cancel.y, cancel.w, cancel.h, 6, TFT_BORDER);
2553 drawCenteredInRect("CANCEL", cancel, 2, TFT_TEXT, TFT_DANGER);
2554 tft.fillRect(clearKey.x, clearKey.y, clearKey.w, clearKey.h, TFT_PANEL_ALT);
2555 tft.drawRoundRect(clearKey.x, clearKey.y, clearKey.w, clearKey.h, 6, TFT_BORDER);
2556 drawCenteredInRect("CLEAR", clearKey, 2, TFT_TEXT, TFT_PANEL_ALT);
2557 tft.fillRect(ok.x, ok.y, ok.w, ok.h, TFT_OK);
2558 tft.drawRoundRect(ok.x, ok.y, ok.w, ok.h, 6, TFT_BORDER);
2559 drawCenteredInRect("OK", ok, 2, ILI9341_BLACK, TFT_OK);
2560 }
2561
2562 void openNameEditor(Player player) {
2563     if (player != PLAYER_A && player != PLAYER_B) {
2564         return;
2565     }
2566
2567     clock_running = false;
2568     active_player = PLAYER_NONE;
2569     editingNameFor = player;
2570     editingName = (player == PLAYER_A) ? playerA_name : playerB_name;
2571     editingPlayerColor = colorForPlayer(player);
2572     mode = MODE_NAME_EDIT;
2573     markDisplayDirty();
2574 }
2575
2576 void openClockSettings() {
2577     uint8_t second = 0;
2578     uint8_t minute = 0;
2579     uint8_t hour = 0;
2580     uint8_t day = 0;
2581     uint8_t month = 0;
2582     uint16_t year = 0;
2583     if (ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) && rtcDateLooksSet(day, month, year)) {
2584         editingRtcYear = year;
2585         editingRtcMonth = month;
2586         editingRtcDay = day;
2587         editingRtcHour = hour;
2588         editingRtcMinute = minute;
2589     } else {
2590         editingRtcYear = 2026;
2591         editingRtcMonth = 1;
2592         editingRtcDay = 1;
2593         editingRtcHour = 12;
2594         editingRtcMinute = 0;
2595     }
2596     mode = MODE_CLOCK_SET;
2597     markDisplayDirty();
2598 }
2599
2600 void handleClockSettingsTap(uint16_t x, uint16_t y) {
2601     if (CLOCK_BACK.containsPadded(x, y, 18, 14)) {
2602         mode = MODE_MENU;
2603         markDisplayDirty();
2604         return;
2605     }

```

```

2606     const Rect minusButtons[] = {
2607         CLOCK_YEAR_MINUS,
2608         CLOCK_MONTH_MINUS,
2609         CLOCK_DAY_MINUS,
2610         CLOCK_HOUR_MINUS,
2611         CLOCK_MINUTE_MINUS
2612     };
2613
2614     const Rect plusButtons[] = {
2615         CLOCK_YEAR_PLUS,
2616         CLOCK_MONTH_PLUS,
2617         CLOCK_DAY_PLUS,
2618         CLOCK_HOUR_PLUS,
2619         CLOCK_MINUTE_PLUS
2620     };
2621
2622     for (uint8_t i = 0; i < 5; i++) {
2623         if (minusButtons[i].containsPadded(x, y, 36, 12)) {
2624             adjustRtcField(i, -1);
2625             return;
2626         }
2627         if (plusButtons[i].containsPadded(x, y, 36, 12)) {
2628             adjustRtcField(i, +1);
2629             return;
2630         }
2631     }
2632
2633     if (CLOCK_SAVE.containsPadded(x, y, 18, 14) && ds3231Found) {
2634         if (ds3231SetTime(0, editingRtcMinute, editingRtcHour, editingRtcDay, editingRtcMonth, editingRtcYear)) {
2635             logPrintf("DS3231 RTC set to %04u-%02u-%02u %02u:%02u:00\n",
2636                 editingRtcYear, editingRtcMonth, editingRtcDay, editingRtcHour, editingRtcMinute);
2637             updateRtcStatusCache(true);
2638             mode = MODE_MENU;
2639         } else {
2640             logLine("DS3231 RTC save failed.");
2641         }
2642         markDisplayDirty();
2643         return;
2644     }
2645 }
2646
2647 void handleEndMatchConfirmTap(uint16_t x, uint16_t y) {
2648     if (CONFIRM_NO.containsPadded(x, y, 18, 14)) {
2649         mode = MODE_MATCH;
2650         markDisplayDirty();
2651         return;
2652     }
2653
2654     if (CONFIRM_YES.containsPadded(x, y, 18, 14)) {
2655         forceEndMatch();
2656         return;
2657     }
2658 }
2659
2660 void adjustRtcField(uint8_t field, int delta) {
2661     switch (field) {
2662         case 0:
2663             editingRtcYear = constrain((int)editingRtcYear + delta, 2024, 2099);
2664             break;
2665         case 1:
2666             editingRtcMonth = constrain((int)editingRtcMonth + delta, 1, 12);
2667             break;
2668         case 2:
2669             editingRtcDay = constrain((int)editingRtcDay + delta, 1, (int)daysInMonth(editingRtcMonth, editingRtcYear));
2670             break;
2671         case 3:
2672             editingRtcHour = (uint8_t)((((int)editingRtcHour + delta + 24) % 24);
2673             break;
2674         case 4:
2675             editingRtcMinute = (uint8_t)((((int)editingRtcMinute + delta + 60) % 60);
2676             break;
2677         default:
2678             break;
2679     }
2680
2681     uint8_t maxDay = daysInMonth(editingRtcMonth, editingRtcYear);
2682     if (editingRtcDay > maxDay) {
2683         editingRtcDay = maxDay;
2684     }
2685     markDisplayDirty();
2686 }
2687
2688 void handleNameEditorTap(uint16_t x, uint16_t y) {
2689     const Rect cancel = {72, 420, 168, 44};
2690     const Rect clearKey = {258, 420, 168, 44};
2691     const Rect ok = {560, 420, 168, 44};

```

```

2692 const Rect spaceKey = {166, 372, 244, 38};
2693 const Rect delKey = {430, 372, 204, 38};
2694 const char *rows[] = {"ABCDEFGHJIJ", "KLMNOPQRST", "UVWXYZ"};
2695 const int16_t rowY[] = {148, 208, 268};
2696 const int16_t keyW = 66;
2697 const int16_t keyH = 44;
2698 const int16_t gap = 8;
2699 const int16_t swatchY = 326;
2700 const int16_t swatchSize = 36;
2701 const int16_t swatchGap = 14;
2702
2703 if (cancel.contains(x, y)) {
2704     editingNameFor = PLAYER_NONE;
2705     mode = MODE_MATCH;
2706     markDisplayDirty();
2707     return;
2708 }
2709
2710 if (ok.contains(x, y)) {
2711     if (editingName.length() == 0) {
2712         editingName = (editingNameFor == PLAYER_A) ? "PLAYER A" : "PLAYER B";
2713     }
2714     if (editingNameFor == PLAYER_A) {
2715         playerA_name = editingName;
2716         playerA_color = editingPlayerColor;
2717     } else if (editingNameFor == PLAYER_B) {
2718         playerB_name = editingName;
2719         playerB_color = editingPlayerColor;
2720     }
2721     savePlayerNames();
2722     editingNameFor = PLAYER_NONE;
2723     mode = MODE_MATCH;
2724     markDisplayDirty();
2725     return;
2726 }
2727
2728 if (clearKey.contains(x, y)) {
2729     editingName = "";
2730     markDisplayDirty();
2731     return;
2732 }
2733
2734 if (delKey.contains(x, y)) {
2735     if (editingName.length() > 0) {
2736         editingName.remove(editingName.length() - 1);
2737     }
2738     markDisplayDirty();
2739     return;
2740 }
2741
2742 if (spaceKey.contains(x, y)) {
2743     if (editingName.length() < PLAYER_NAME_MAX_LEN && editingName.length() > 0) {
2744         editingName += ' ';
2745     }
2746     markDisplayDirty();
2747     return;
2748 }
2749
2750 int16_t swatchStartX = (PANEL_WIDTH - (playerColorOptionCount() * swatchSize + (playerColorOptionCount() - 1) * swatchGap)) / 2;
2751 for (uint8_t i = 0; i < playerColorOptionCount(); i++) {
2752     Rect swatch = {swatchStartX + (int16_t)i * (swatchSize + swatchGap), swatchY, swatchSize, swatchSize};
2753     if (swatch.contains(x, y)) {
2754         editingPlayerColor = playerColorOption(i);
2755         markDisplayDirty();
2756         return;
2757     }
2758 }
2759
2760 for (uint8_t r = 0; r < 3; r++) {
2761     uint8_t count = strlen(rows[r]);
2762     int16_t startX = (PANEL_WIDTH - ((count * keyW) + ((count - 1) * gap))) / 2;
2763     for (uint8_t i = 0; i < count; i++) {
2764         Rect key = {startX + (int16_t)i * (keyW + gap), rowY[r], keyW, keyH};
2765         if (key.contains(x, y) && editingName.length() < PLAYER_NAME_MAX_LEN) {
2766             editingName += rows[r][i];
2767             markDisplayDirty();
2768             return;
2769         }
2770     }
2771 }
2772 }
2773
2774 uint16_t colorForPlayer(Player player) {
2775     if (player == PLAYER_A) {
2776         return playerA_color;
2777     }

```

```

2778     if (player == PLAYER_B) {
2779         return playerB_color;
2780     }
2781     return TFT_PANEL;
2782 }
2783
2784 uint16_t textColorForPlayerColor(uint16_t color) {
2785     return (color == PLAYER_COLOR_WHITE || color == PLAYER_COLOR_YELLOW) ? TFT_TEXT_DARK : TFT_TEXT;
2786 }
2787
2788 uint16_t playerColorOption(uint8_t index) {
2789     switch (index) {
2790     case 0:
2791         return PLAYER_COLOR_RED;
2792     case 1:
2793         return PLAYER_COLOR_BLUE;
2794     case 2:
2795         return PLAYER_COLOR_GREEN;
2796     case 3:
2797         return PLAYER_COLOR_YELLOW;
2798     case 4:
2799         return PLAYER_COLOR_GRAY;
2800     case 5:
2801         return PLAYER_COLOR_WHITE;
2802     default:
2803         return PLAYER_COLOR_BLUE;
2804     }
2805 }
2806
2807 uint8_t playerColorOptionCount() {
2808     return 6;
2809 }
2810
2811 void drawTimerPanels() {
2812     if (mode != MODE_MATCH) {
2813         return;
2814     }
2815
2816     drawStatusBar();
2817     drawTimerPanel(PLAYER_A, REGION_TIMER_A);
2818     drawTimerPanel(PLAYER_B, REGION_TIMER_B);
2819 }
2820
2821 void drawTimerPanel(Player player, const Rect &region) {
2822     bool isA = (player == PLAYER_A);
2823     bool active = (active_player == player && clock_running);
2824     uint16_t accent = isA ? TFT_ACCENT_A : TFT_ACCENT_B;
2825     uint16_t panelColor = active ? ILI9341_WHITE : TFT_PANEL;
2826     uint32_t remaining = visibleReserveMs(player, millis());
2827     String value = formatTimeMs(remaining);
2828     uint16_t valueColor = active ? TFT_ACTIVE_TEXT : TFT_TEXT;
2829     bool showingDelay = false;
2830
2831     if (active && timingMode == SIMPLE_DELAY && delay_seconds > 0) {
2832         uint32_t elapsedMs = millis() - turnStartMs;
2833         uint32_t delayMs = (uint32_t)delay_seconds * 1000UL;
2834         if (elapsedMs < delayMs) {
2835             uint32_t delayRemaining = (delayMs - elapsedMs + 999UL) / 1000UL;
2836             value = String(delayRemaining);
2837             valueColor = active ? TFT_ACTIVE_TEXT : TFT_WARN;
2838             showingDelay = true;
2839         }
2840     }
2841
2842     tft.fillRect(region.x, region.y, region.w, region.h, panelColor);
2843     tft.drawRoundRect(region.x, region.y, region.w, region.h, 6, accent);
2844
2845     #if BGMC_HAS_CLOCK_FONT
2846     tft.setFont(&FreeMonoBold24pt7b);
2847     tft.setTextSize(1);
2848     tft.setTextColor(valueColor, panelColor);
2849     int16_t x1;
2850     int16_t y1;
2851     uint16_t w;
2852     uint16_t h;
2853     tft.getTextBounds(value, 0, 0, &x1, &y1, &w, &h);
2854     int16_t valueX = region.x + ((region.w - (int16_t)w) / 2) - x1;
2855     int16_t valueY = region.y + ((region.h - (int16_t)h) / 2) - y1;
2856     tft.setCursor(valueX, valueY);
2857     tft.print(value);
2858     tft.setFont();
2859     #else
2860     tft.setTextSize(showingDelay ? 6 : 4);
2861     tft.setTextColor(valueColor, panelColor);
2862     int16_t valueX = showingDelay ? (region.x + 104) : (region.x + 48);
2863     if (!showingDelay && remaining >= 10UL * 60UL * 1000UL) {

```

```

2864     valueX = region.x + 36;
2865 }
2866 tft.setCursor(valueX, showingDelay ? (region.y + 30) : (region.y + 38));
2867 tft.print(value);
2868 #endif
2869 }
2870
2871 void drawPlayerPanel(Player player, int16_t x, int16_t y, int16_t w, int16_t h) {
2872     bool isA = (player == PLAYER_A);
2873     bool active = (active_player == player && clock_running);
2874     uint16_t accent = isA ? TFT_ACCENT_A : TFT_ACCENT_B;
2875     uint16_t border = active ? TFT_OK : accent;
2876     uint32_t remaining = isA ? playerA_time_remaining : playerB_time_remaining;
2877     int score = isA ? playerA_score : playerB_score;
2878
2879     tft.fillRect(x, y, w, h, TFT_PANEL);
2880     tft.drawRoundRect(x, y, w, h, 6, border);
2881     tft.drawRoundRect(x + 3, y + 3, w - 6, h - 6, 6, TFT_PANEL_ALT);
2882     if (active) {
2883         tft.fillRect(x + 8, y + 8, w - 16, 10, TFT_OK);
2884     }
2885
2886     tft.setTextColor(accent, TFT_PANEL);
2887     tft.setTextSize(3);
2888     tft.setCursor(x + 24, y + 34);
2889     tft.print(isA ? "PLAYER A" : "PLAYER B");
2890
2891     tft.setTextColor((remaining <= LOW_TIME_WARNING_SECONDS) ? TFT_DANGER : TFT_TEXT, TFT_PANEL);
2892     tft.setTextSize(6);
2893     tft.setCursor(x + 24, y + 98);
2894     tft.print(formatTimeMs(remaining));
2895
2896     tft.fillRect(x + 24, y + 188, 150, 34, active ? TFT_OK : TFT_PANEL_ALT);
2897     tft.setTextColor(active ? ILI9341_BLACK : TFT_TEXT, active ? TFT_OK : TFT_PANEL_ALT);
2898     tft.setTextSize(2);
2899     tft.setCursor(x + 40, y + 198);
2900     tft.printf("SCORE %d", score);
2901
2902     tft.setTextColor(TFT_MUTED, TFT_PANEL);
2903     tft.setTextSize(1);
2904     tft.setCursor(x + 212, y + 204);
2905     tft.print(active ? "RUNNING" : "WAITING");
2906 }
2907
2908 void drawStatusBar() {
2909     tft.fillRect(REGION_STATUS.x, REGION_STATUS.y, REGION_STATUS.w, REGION_STATUS.h, TFT_PANEL);
2910     tft.fillRect(0, REGION_STATUS.h - 1, tft.width(), 1, TFT_BORDER);
2911     tft.setTextColor(TFT_TEXT, TFT_PANEL);
2912     tft.setTextSize(2);
2913     tft.setCursor(18, 10);
2914     tft.print("BATTERY");
2915     tft.setCursor(114, 10);
2916     int pct = batteryPercent();
2917     if (pct < 0) {
2918         tft.print("--%");
2919     } else {
2920         tft.printf("%d%%", pct);
2921     }
2922     if (chargingIndicatorOn()) {
2923         drawChargeBolt(168, 6, TFT_WARN, TFT_PANEL);
2924     }
2925
2926     tft.setTextColor(TFT_MUTED, TFT_PANEL);
2927     tft.setCursor(300, 10);
2928     tft.print("ELAPSED");
2929     tft.setTextColor(TFT_TEXT, TFT_PANEL);
2930     tft.setCursor(410, 10);
2931     tft.print(formatTime(match_elapsed_seconds));
2932
2933     tft.setTextColor(TFT_TEXT, TFT_PANEL);
2934     tft.setTextSize(2);
2935     tft.setCursor(tft.width() - 168, 10);
2936     tft.print(cachedRtcStatus);
2937 }
2938
2939 void drawChargeBolt(int16_t x, int16_t y, uint16_t color, uint16_t background) {
2940     tft.fillRect(x, y, 20, 24, background);
2941     fillTriangleRows(x + 11, y + 3, x + 5, y + 12, x + 11, y + 12, color);
2942     fillTriangleRows(x + 9, y + 10, x + 16, y + 10, x + 6, y + 21, color);
2943 }
2944
2945 void drawStartupTest() {
2946     logLine("Drawing startup splash image");
2947
2948     setBrightness(255);
2949     drawSplashImage();

```

```

2950     delay(4000);
2951 }
2952
2953 void drawSplashImage() {
2954     if (SPLASH_IMAGE_SCALE == 2) {
2955         tft.drawRGB565ImageScaled2x(0, 0, SPLASH_IMAGE_WIDTH, SPLASH_IMAGE_HEIGHT, SPLASH_IMAGE_RGB565);
2956     } else {
2957         tft.drawRGB565Image(0, 0, SPLASH_IMAGE_WIDTH, SPLASH_IMAGE_HEIGHT, SPLASH_IMAGE_RGB565);
2958     }
2959 }
2960
2961 void drawGeneratedSplashImage() {
2962     const uint16_t board = 0x0841;
2963     const uint16_t wood = 0x3922;
2964     const uint16_t woodLite = 0x7A84;
2965     const uint16_t gold = 0xDDA6;
2966     const uint16_t amber = 0xB3A0;
2967     const uint16_t cream = 0xDEB2;
2968     const uint16_t orange = 0x8A80;
2969     const uint16_t blackChecker = 0x10A2;
2970     const uint16_t whiteChecker = 0xF6D8;
2971     const uint16_t clockBg = 0x0000;
2972     const uint16_t silver = 0xD6BA;
2973
2974     tft.fillScreen(board);
2975     tft.fillRect(0, 0, 58, PANEL_HEIGHT, wood);
2976     tft.fillRect(742, 0, 58, PANEL_HEIGHT, wood);
2977     for (int16_t x = 8; x < 58; x += 12) {
2978         tft.fillRect(x, 0, 2, PANEL_HEIGHT, woodLite);
2979         tft.fillRect(742 + x, 0, 2, PANEL_HEIGHT, woodLite);
2980     }
2981
2982     const int16_t pointTop = 0;
2983     const int16_t pointBottom = PANEL_HEIGHT - 1;
2984     const int16_t pointH = 220;
2985     const int16_t leftPoints[] = {88, 140, 190};
2986     const int16_t rightPoints[] = {610, 662, 714};
2987     for (uint8_t i = 0; i < 3; i++) {
2988         uint16_t color = (i == 1) ? orange : cream;
2989         fillTriangleRows(leftPoints[i], pointTop, leftPoints[i] + 44, pointTop, leftPoints[i] + 22, pointTop + pointH, color);
2990         fillTriangleRows(rightPoints[i], pointTop, rightPoints[i] + 44, pointTop, rightPoints[i] + 22, pointTop + pointH, color);
2991         fillTriangleRows(leftPoints[i], pointBottom, leftPoints[i] + 44, pointBottom, leftPoints[i] + 22, pointBottom - pointH, color);
2992         fillTriangleRows(rightPoints[i], pointBottom, rightPoints[i] + 44, pointBottom, rightPoints[i] + 22, pointBottom - pointH, color);
2993     }
2994
2995     fillCircleRows(372, 120, 32, whiteChecker);
2996     fillCircleRows(418, 128, 28, blackChecker);
2997     fillCircleRows(372, 120, 22, 0xE5B1);
2998     fillCircleRows(418, 128, 19, 0x2945);
2999     tft.fillRect(296, 138, 78, 2, gold);
3000     tft.fillRect(450, 138, 106, 2, gold);
3001     tft.fillRect(314, 122, 36, 2, gold);
3002     tft.fillRect(318, 110, 24, 2, gold);
3003
3004     drawCenteredInRect("BACKGAMMON", {110, 166, 580, 60}, 6, TFT_TEXT, board);
3005     tft.fillRect(250, 236, 58, 2, gold);
3006     tft.fillRect(492, 236, 58, 2, gold);
3007     drawCenteredInRect("CLOCK", {298, 220, 204, 48}, 4, amber, board);
3008
3009     tft.fillRect(224, 288, 352, 78, clockBg);
3010     tft.drawRoundRect(224, 288, 352, 78, 8, gold);
3011     drawCenteredInRect("15:00 : 15:00", {242, 300, 316, 52}, 4, silver, clockBg);
3012     tft.setTextColor(amber, board);
3013     tft.setTextSize(2);
3014     tft.setCursor(254, 390);
3015     tft.print("FOCUS. STRATEGY. EVERY MOVE COUNTS.");
3016     tft.setCursor(286, 434);
3017     tft.print("PLAY FAIR. PLAY STRONG.");
3018 }
3019
3020 void fillTriangleRows(int16_t x0, int16_t y0, int16_t x1, int16_t y1, int16_t x2, int16_t y2, uint16_t color) {
3021     if (y1 < y0) {
3022         int16_t tx = x0; x0 = x1; x1 = tx;
3023         int16_t ty = y0; y0 = y1; y1 = ty;
3024     }
3025     if (y2 < y0) {
3026         int16_t tx = x0; x0 = x2; x2 = tx;
3027         int16_t ty = y0; y0 = y2; y2 = ty;
3028     }
3029     if (y2 < y1) {
3030         int16_t tx = x1; x1 = x2; x2 = tx;
3031         int16_t ty = y1; y1 = y2; y2 = ty;
3032     }
3033
3034     int16_t totalHeight = y2 - y0;
3035     if (totalHeight == 0) {

```

```

3036     return;
3037 }
3038
3039 for (int16_t y = y0; y <= y2; y++) {
3040     bool secondHalf = y > y1 || y1 == y0;
3041     int16_t segmentHeight = secondHalf ? (y2 - y1) : (y1 - y0);
3042     if (segmentHeight == 0) {
3043         continue;
3044     }
3045     float alpha = (float)(y - y0) / (float)totalHeight;
3046     float beta = secondHalf ? (float)(y - y1) / (float)segmentHeight : (float)(y - y0) / (float)segmentHeight;
3047     int16_t ax = x0 + (int16_t)((x2 - x0) * alpha);
3048     int16_t bx = secondHalf ? x1 + (int16_t)((x2 - x1) * beta) : x0 + (int16_t)((x1 - x0) * beta);
3049     if (ax > bx) {
3050         int16_t tmp = ax;
3051         ax = bx;
3052         bx = tmp;
3053     }
3054     tft.fillRect(ax, y, bx - ax + 1, 1, color);
3055 }
3056 }
3057
3058 void fillCircleRows(int16_t cx, int16_t cy, int16_t radius, uint16_t color) {
3059     int16_t r2 = radius * radius;
3060     for (int16_t y = -radius; y <= radius; y++) {
3061         int16_t x = 0;
3062         while ((x + 1) * (x + 1) + y * y <= r2) {
3063             x++;
3064         }
3065         tft.fillRect(cx - x, cy + y, (x * 2) + 1, 1, color);
3066     }
3067     tft.drawRoundRect(cx - radius, cy - radius, radius * 2, radius * 2, radius, TFT_BORDER);
3068 }
3069
3070 void initCrowPanelExpander() {
3071     Wire.begin(PIN_I2C_SDA, PIN_I2C_SCL);
3072     Wire.setClock(400000);
3073     scanI2C("before PCA9557 init");
3074
3075     uint8_t pcaAddress = 0;
3076     for (uint8_t address = 0x18; address <= 0x1F; address++) {
3077         Wire.beginTransmission(address);
3078         if (Wire.endTransmission() == 0) {
3079             pcaAddress = address;
3080             break;
3081         }
3082     }
3083
3084     if (pcaAddress == 0) {
3085         logLine("PCA9557 not found on I2C; continuing without expander init");
3086         return;
3087     }
3088
3089     logPrintf("PCA9557 found at 0x%02X\n", pcaAddress);
3090     pca9557Address = pcaAddress;
3091
3092     // Elecrow V3 sequence: I00/I01 low, short delay, I00 high, I01 input.
3093     // Registers: 1=output port, 2=polarity, 3=configuration (1=input, 0=output).
3094     bool ok = true;
3095     ok &= pca9557Write(pcaAddress, 0x02, 0x00);
3096     ok &= pca9557Write(pcaAddress, 0x03, 0x00);
3097     ok &= pca9557Write(pcaAddress, 0x01, 0x00);
3098     delay(20);
3099     ok &= pca9557Write(pcaAddress, 0x01, 0x01);
3100     delay(100);
3101     ok &= pca9557Write(pcaAddress, 0x03, 0x02);
3102
3103     logPrintf("PCA9557 init: %s\n", ok ? "OK" : "FAILED");
3104     scanI2C("after PCA9557 init");
3105 }
3106
3107 bool pca9557Write(uint8_t address, uint8_t reg, uint8_t value) {
3108     Wire.beginTransmission(address);
3109     Wire.write(reg);
3110     Wire.write(value);
3111     return Wire.endTransmission() == 0;
3112 }
3113
3114 uint8_t pca9557Read(uint8_t address, uint8_t reg) {
3115     Wire.beginTransmission(address);
3116     Wire.write(reg);
3117     if (Wire.endTransmission(false) != 0) {
3118         return 0xFF;
3119     }
3120     if (Wire.requestFrom((int)address, 1) != 1) {
3121         return 0xFF;
3122     }

```

```

3122 }
3123 return Wire.read();
3124 }
3125
3126 void initButtonExpander() {
3127     uint8_t foundAddress = 0;
3128     for (uint8_t address = 0x20; address <= 0x27; address++) {
3129         Wire.beginTransaction(address);
3130         if (Wire.endTransmission() == 0) {
3131             foundAddress = address;
3132             break;
3133         }
3134     }
3135
3136     if (foundAddress == 0) {
3137         logLine("PCF8575 button expander not found; physical I2C buttons disabled.");
3138         return;
3139     }
3140
3141     pcf8575Address = foundAddress;
3142     bool ok = pcf8575Write(pcf8575Address, 0xFFFF);
3143     logPrintf("PCF8575 button expander found at 0x%02X: %s\n", pcf8575Address, ok ? "OK" : "FAILED");
3144     logPrintf("PCF8575 buttons: Play/Pause=P%02u Player A=P%02u Player B=P%02u\n",
3145         PCF8575_PLAY_PAUSE_PIN, PCF8575_PLAYER_A_PIN, PCF8575_PLAYER_B_PIN);
3146     scanI2C("after PCF8575 init");
3147 }
3148
3149 bool pcf8575Write(uint8_t address, uint16_t value) {
3150     Wire.beginTransaction(address);
3151     Wire.write(value & 0xFF);
3152     Wire.write((value >> 8) & 0xFF);
3153     return Wire.endTransmission() == 0;
3154 }
3155
3156 uint16_t pcf8575Read(uint8_t address) {
3157     if (Wire.requestFrom((int)address, 2) != 2) {
3158         return 0xFFFF;
3159     }
3160     uint8_t low = Wire.read();
3161     uint8_t high = Wire.read();
3162     return (uint16_t)low | ((uint16_t)high << 8);
3163 }
3164
3165 void initFuelGauge() {
3166     Wire.beginTransaction(MAX17048_I2C_ADDRESS);
3167     if (Wire.endTransmission() != 0) {
3168         max17048Found = false;
3169         logLine("MAX17048 fuel gauge not found; battery display will use fallback.");
3170         return;
3171     }
3172
3173     uint16_t version = 0;
3174     max17048Found = true;
3175     max17048Read16(0x08, &version);
3176     logPrintf("MAX17048 fuel gauge found at 0x%02X version=0x%04X\n", MAX17048_I2C_ADDRESS, version);
3177     logPrintf("MAX17048 battery: %.2fV %.1f%%\n", max17048CellVoltage(), max17048StateOfCharge());
3178     scanI2C("after MAX17048 init");
3179 }
3180
3181 bool max17048Read16(uint8_t reg, uint16_t *value) {
3182     if (value == nullptr) {
3183         return false;
3184     }
3185
3186     Wire.beginTransaction(MAX17048_I2C_ADDRESS);
3187     Wire.write(reg);
3188     if (Wire.endTransmission(false) != 0) {
3189         return false;
3190     }
3191
3192     if (Wire.requestFrom((int)MAX17048_I2C_ADDRESS, 2) != 2) {
3193         return false;
3194     }
3195
3196     uint8_t msb = Wire.read();
3197     uint8_t lsb = Wire.read();
3198     *value = ((uint16_t)msb << 8) | lsb;
3199     return true;
3200 }
3201
3202 float max17048CellVoltage() {
3203     uint16_t raw = 0;
3204     if (!max17048Found || !max17048Read16(0x02, &raw)) {
3205         return 0.0f;
3206     }
3207     return (float)(raw >> 4) * 0.00125f;

```

```

3208 }
3209
3210 float max17048StateOfCharge() {
3211     uint16_t raw = 0;
3212     if (!max17048Found || !max17048Read16(0x04, &raw)) {
3213         return 0.0f;
3214     }
3215     float pct = (float)(raw >> 8) + ((float)(raw & 0xFF) / 256.0f);
3216     if (pct < 0.0f) {
3217         pct = 0.0f;
3218     }
3219     if (pct > 100.0f) {
3220         pct = 100.0f;
3221     }
3222     return pct;
3223 }
3224
3225 void initRtcClock() {
3226     Wire.beginTransmission(DS3231_I2C_ADDRESS);
3227     if (Wire.endTransmission() != 0) {
3228         ds3231Found = false;
3229         logLine("DS3231 RTC not found; match log dates will be DATE UNSET.");
3230         return;
3231     }
3232
3233     ds3231Found = true;
3234     uint8_t second = 0;
3235     uint8_t minute = 0;
3236     uint8_t hour = 0;
3237     uint8_t day = 0;
3238     uint8_t month = 0;
3239     uint16_t year = 0;
3240     if (ds3231ReadTime(&second, &minute, &hour, &day, &month, &year)) {
3241         if (rtcDateLooksSet(day, month, year)) {
3242             logPrintf("DS3231 RTC found at 0x%02X date=%04u-%02u-%02u time=%02u:%02u:%02u\n",
3243                 DS3231_I2C_ADDRESS, year, month, day, hour, minute, second);
3244         } else {
3245             logPrintf("DS3231 RTC found at 0x%02X but date looks unset: %04u-%02u-%02u\n",
3246                 DS3231_I2C_ADDRESS, year, month, day);
3247         }
3248     } else {
3249         logPrintf("DS3231 RTC found at 0x%02X but time read failed.\n", DS3231_I2C_ADDRESS);
3250     }
3251     scanI2C("after DS3231 init");
3252 }
3253
3254 bool ds3231ReadTime(uint8_t *second, uint8_t *minute, uint8_t *hour, uint8_t *day, uint8_t *month, uint16_t *year) {
3255     if (second == nullptr || minute == nullptr || hour == nullptr || day == nullptr || month == nullptr || year == nullptr) {
3256         return false;
3257     }
3258
3259     Wire.beginTransmission(DS3231_I2C_ADDRESS);
3260     Wire.write(0x00);
3261     if (Wire.endTransmission(false) != 0) {
3262         return false;
3263     }
3264
3265     if (Wire.requestFrom((int)DS3231_I2C_ADDRESS, 7) != 7) {
3266         return false;
3267     }
3268
3269     uint8_t rawSecond = Wire.read();
3270     uint8_t rawMinute = Wire.read();
3271     uint8_t rawHour = Wire.read();
3272     Wire.read(); // Day of week, unused for match logs.
3273     uint8_t rawDay = Wire.read();
3274     uint8_t rawMonth = Wire.read();
3275     uint8_t rawYear = Wire.read();
3276
3277     *second = bcdToDec(rawSecond & 0x7F);
3278     *minute = bcdToDec(rawMinute & 0x7F);
3279     if ((rawHour & 0x40) != 0) {
3280         uint8_t hour12 = bcdToDec(rawHour & 0x1F);
3281         bool pm = (rawHour & 0x20) != 0;
3282         *hour = (hour12 % 12) + (pm ? 12 : 0);
3283     } else {
3284         *hour = bcdToDec(rawHour & 0x3F);
3285     }
3286     *day = bcdToDec(rawDay & 0x3F);
3287     *month = bcdToDec(rawMonth & 0x1F);
3288     *year = 2000 + bcdToDec(rawYear);
3289     return true;
3290 }
3291
3292 bool ds3231SetTime(uint8_t second, uint8_t minute, uint8_t hour, uint8_t day, uint8_t month, uint16_t year) {
3293     if (!ds3231Found || !rtcDateLooksSet(day, month, year) || hour > 23 || minute > 59 || second > 59) {

```

```

3294     return false;
3295 }
3296
3297 Wire.beginTransmission(DS3231_I2C_ADDRESS);
3298 Wire.write(0x00);
3299 Wire.write(decToBcd(second));
3300 Wire.write(decToBcd(minute));
3301 Wire.write(decToBcd(hour)); // 24-hour mode.
3302 Wire.write(decToBcd(1)); // Day of week is not used by this firmware.
3303 Wire.write(decToBcd(day));
3304 Wire.write(decToBcd(month));
3305 Wire.write(decToBcd(uint8_t)(year - 2000));
3306 return Wire.endTransmission() == 0;
3307 }
3308
3309 bool rtcDateLooksSet(uint8_t day, uint8_t month, uint16_t year) {
3310     return year >= 2024 && year <= 2099 && month >= 1 && month <= 12 && day >= 1 && day <= 31;
3311 }
3312
3313 uint8_t daysInMonth(uint8_t month, uint16_t year) {
3314     switch (month) {
3315         case 4:
3316         case 6:
3317         case 9:
3318         case 11:
3319             return 30;
3320         case 2: {
3321             bool leap = ((year % 4 == 0) && (year % 100 != 0)) || (year % 400 == 0);
3322             return leap ? 29 : 28;
3323         }
3324         default:
3325             return 31;
3326     }
3327 }
3328
3329 uint8_t bcdToDec(uint8_t value) {
3330     return ((value >> 4) * 10) + (value & 0x0F);
3331 }
3332
3333 uint8_t decToBcd(uint8_t value) {
3334     return ((value / 10) << 4) | (value % 10);
3335 }
3336
3337 String rtcStatusText() {
3338     uint8_t second = 0;
3339     uint8_t minute = 0;
3340     uint8_t hour = 0;
3341     uint8_t day = 0;
3342     uint8_t month = 0;
3343     uint16_t year = 0;
3344     if (!ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) || !rtcDateLooksSet(day, month, year)) {
3345         return "DATE UNSET";
3346     }
3347
3348     char buffer[18];
3349     snprintf(buffer, sizeof(buffer), "%02u/%02u %02u:%02u", month, day, hour, minute);
3350     return String(buffer);
3351 }
3352
3353 bool updateRtcStatusCache(bool force) {
3354     uint32_t nowMs = millis();
3355     if (!force && nowMs - lastRtcStatusMs < 1000UL) {
3356         return false;
3357     }
3358
3359     String next = rtcStatusText();
3360     bool changed = force || next != cachedRtcStatus;
3361     cachedRtcStatus = next;
3362     lastRtcStatusMs = nowMs;
3363     return changed;
3364 }
3365
3366 void scanI2C(const char *label) {
3367     logPrintf("I2C scan %s:", label);
3368     bool any = false;
3369     for (uint8_t address = 1; address < 127; address++) {
3370         Wire.beginTransmission(address);
3371         if (Wire.endTransmission() == 0) {
3372             logPrintf(" 0x%02X", address);
3373             any = true;
3374         }
3375     }
3376     logLine(any ? "" : " none");
3377 }
3378
3379 void resetTouchController(uint8_t outputValue, uint8_t configValue) {

```

```

3380 if (pca9557Address == 0) {
3381     return;
3382 }
3383 pca9557Write(pca9557Address, 0x03, 0x00);
3384 pca9557Write(pca9557Address, 0x01, outputValue);
3385 delay(20);
3386 pca9557Write(pca9557Address, 0x03, configValue);
3387 delay(80);
3388 }
3389
3390 void initTouch() {
3391     scanI2C("before GT911 init");
3392
3393     const uint8_t addresses[] = {0x5D, 0x14};
3394     for (uint8_t i = 0; i < sizeof(addresses); i++) {
3395         Wire.beginTransmission(addresses[i]);
3396         if (Wire.endTransmission() == 0) {
3397             gt911Address = addresses[i];
3398             break;
3399         }
3400     }
3401
3402     if (gt911Address == 0) {
3403         const uint8_t resetOutputs[] = {0x01, 0x00, 0x03, 0x02};
3404         const uint8_t resetConfigs[] = {0x02, 0x02, 0x00, 0x00};
3405         for (uint8_t attempt = 0; attempt < 4 && gt911Address == 0; attempt++) {
3406             resetTouchController(resetOutputs[attempt], resetConfigs[attempt]);
3407             scanI2C("after touch reset attempt");
3408             for (uint8_t i = 0; i < sizeof(addresses); i++) {
3409                 Wire.beginTransmission(addresses[i]);
3410                 if (Wire.endTransmission() == 0) {
3411                     gt911Address = addresses[i];
3412                     break;
3413                 }
3414             }
3415         }
3416     }
3417
3418     if (gt911Address == 0) {
3419         logLine("GT911 touch controller not found. Touch disabled.");
3420         return;
3421     }
3422
3423     uint8_t productId[4] = {};
3424     gt911Read(0x8140, productId, sizeof(productId));
3425     logPrintf("GT911 found at 0x%02X id=%c%c%c%c\n", gt911Address,
3426             productId[0], productId[1], productId[2], productId[3]);
3427     gt911Write(0x814E, 0x00);
3428 }
3429
3430 TouchPoint readTouch() {
3431     TouchPoint point = {false, 0, 0};
3432     if (gt911Address == 0) {
3433         return point;
3434     }
3435
3436     uint8_t status = 0;
3437     if (!gt911Read(0x814E, &status, 1)) {
3438         return point;
3439     }
3440
3441     uint8_t count = status & 0x0F;
3442     if (count == 0) {
3443         if (status & 0x80) {
3444             gt911Write(0x814E, 0x00);
3445         }
3446         return point;
3447     }
3448     if (count > 5) {
3449         gt911Write(0x814E, 0x00);
3450         return point;
3451     }
3452
3453     uint8_t data[7] = {};
3454     if (gt911Read(0x8150, data, sizeof(data))) {
3455         uint16_t xLowHigh = ((uint16_t)data[1] << 8) | data[0];
3456         uint16_t yLowHigh = ((uint16_t)data[3] << 8) | data[2];
3457         uint16_t xHighLow = ((uint16_t)data[0] << 8) | data[1];
3458         uint16_t yHighLow = ((uint16_t)data[2] << 8) | data[3];
3459         bool lowHighValid = xLowHigh < PANEL_WIDTH && yLowHigh < PANEL_HEIGHT;
3460         bool highLowValid = xHighLow < PANEL_WIDTH && yHighLow < PANEL_HEIGHT;
3461
3462         if (lowHighValid || highLowValid) {
3463             point.touched = true;
3464             point.x = lowHighValid ? xLowHigh : xHighLow;
3465             point.y = lowHighValid ? yLowHigh : yHighLow;

```

```

3466 } else {
3467     uint16_t shiftedXLowHigh = ((uint16_t)data[2] << 8) | data[1];
3468     uint16_t shiftedYLowHigh = ((uint16_t)data[4] << 8) | data[3];
3469     uint16_t shiftedXHighLow = ((uint16_t)data[1] << 8) | data[2];
3470     uint16_t shiftedYHighLow = ((uint16_t)data[3] << 8) | data[4];
3471     bool shiftedLowHighValid = shiftedXLowHigh < PANEL_WIDTH && shiftedYLowHigh < PANEL_HEIGHT;
3472     bool shiftedHighLowValid = shiftedXHighLow < PANEL_WIDTH && shiftedYHighLow < PANEL_HEIGHT;
3473
3474     if (shiftedLowHighValid || shiftedHighLowValid) {
3475         point.touched = true;
3476         point.x = shiftedLowHighValid ? shiftedXLowHigh : shiftedXHighLow;
3477         point.y = shiftedLowHighValid ? shiftedYLowHigh : shiftedYHighLow;
3478     }
3479 }
3480 }
3481
3482 gt911Write(0x814E, 0x00);
3483 return point;
3484 }
3485
3486 bool gt911Read(uint16_t reg, uint8_t *data, uint8_t len) {
3487     Wire.beginTransmission(gt911Address);
3488     Wire.write(reg >> 8);
3489     Wire.write(reg & 0xFF);
3490     if (Wire.endTransmission(false) != 0) {
3491         return false;
3492     }
3493     uint8_t received = Wire.requestFrom((int)gt911Address, (int)len);
3494     if (received != len) {
3495         return false;
3496     }
3497     for (uint8_t i = 0; i < len; i++) {
3498         data[i] = Wire.read();
3499     }
3500     return true;
3501 }
3502
3503 bool gt911Write(uint16_t reg, uint8_t value) {
3504     Wire.beginTransmission(gt911Address);
3505     Wire.write(reg >> 8);
3506     Wire.write(reg & 0xFF);
3507     Wire.write(value);
3508     return Wire.endTransmission() == 0;
3509 }
3510
3511 void logLine(const char *message) {
3512     Serial.println(message);
3513     Serial0.println(message);
3514 }
3515
3516 void logPrintf(const char *format, ...) {
3517     char buffer[160];
3518     va_list args;
3519     va_start(args, format);
3520     vsnprintf(buffer, sizeof(buffer), format, args);
3521     va_end(args);
3522
3523     Serial.print(buffer);
3524     Serial0.print(buffer);
3525 }
3526
3527 void drawCentered(const String &text, int16_t y, uint8_t textSize, uint16_t color) {
3528     tft.setTextSize(textSize);
3529     tft.setTextColor(color, TFT_BACKGROUND);
3530     int16_t x1;
3531     int16_t y1;
3532     uint16_t w;
3533     uint16_t h;
3534     tft.getTextBounds(text, 0, y, &x1, &y1, &w, &h);
3535     tft.setCursor((tft.width() - w) / 2, y);
3536     tft.print(text);
3537 }
3538
3539 void drawCenteredInRect(const String &text, const Rect &region, uint8_t textSize, uint16_t color, uint16_t background) {
3540     #if BGMC_HAS_SMALL_UI_FONT
3541     bool useUIFont = (textSize >= 2 && textSize <= 3);
3542     if (useUIFont) {
3543         tft.setFont(&FreeSans12pt7b);
3544         tft.setTextSize(1);
3545     } else {
3546         tft.setFont();
3547         tft.setTextSize(textSize);
3548     }
3549     #elif BGMC_HAS_UI_FONT
3550     bool useUIFont = (textSize >= 2 && textSize <= 3);

```

```

3552     if (useUIFont) {
3553         tft.setFont(&FreeSans18pt7b);
3554         tft.setTextSize(1);
3555     } else {
3556         tft.setFont();
3557         tft.setTextSize(textSize);
3558     }
3559 #else
3560     tft.setFont();
3561     tft.setTextSize(textSize);
3562 #endif
3563     tft.setTextColor(color, background);
3564     int16_t x1;
3565     int16_t y1;
3566     uint16_t w;
3567     uint16_t h;
3568     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3569     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3570     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3571     tft.setCursor(x, y);
3572     tft.print(text);
3573     tft.setFont();
3574 }
3575
3576 void drawScoreInRect(int score, const Rect &region, uint16_t color, uint16_t background) {
3577     String text = String(score);
3578     #if BGMC_HAS_COLLEGE_SCORE_FONT
3579         tft.setFont(&collegeb96pt7b);
3580         tft.setTextSize(1);
3581     #elif BGMC_HAS_SCORE_96_FONT
3582         tft.setFont(&FreeSansBold96pt7b);
3583         tft.setTextSize(1);
3584     #elif BGMC_HAS_SCORE_FONT
3585         tft.setFont(&FreeSansBold24pt7b);
3586         tft.setTextSize(4);
3587     #elif BGMC_HAS_UI_FONT
3588         tft.setFont(&FreeSans18pt7b);
3589         tft.setTextSize(3);
3590     #else
3591         tft.setFont();
3592         tft.setTextSize(10);
3593     #endif
3594     tft.setTextColor(color, background);
3595     int16_t x1;
3596     int16_t y1;
3597     uint16_t w;
3598     uint16_t h;
3599     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3600     #if BGMC_HAS_COLLEGE_SCORE_FONT || BGMC_HAS_SCORE_96_FONT
3601         if (w > region.w - 28 || h > region.h - 28) {
3602             #if BGMC_HAS_SCORE_FONT
3603                 tft.setFont(&FreeSansBold24pt7b);
3604                 tft.setTextSize(4);
3605             #else
3606                 tft.setFont();
3607                 tft.setTextSize(10);
3608             #endif
3609             tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3610         }
3611     #endif
3612     #if BGMC_HAS_SCORE_FONT
3613         uint8_t scoreSize = 4;
3614         while (scoreSize > 1 && (w > region.w - 28 || h > region.h - 28)) {
3615             scoreSize--;
3616             tft.setTextSize(scoreSize);
3617             tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3618         }
3619     #endif
3620     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3621     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3622     tft.setCursor(x, y);
3623     tft.print(text);
3624     tft.setFont();
3625 }
3626
3627 void drawMatchTargetInRect(int target, const Rect &region, uint16_t color, uint16_t background) {
3628     String text = String(target);
3629     #if BGMC_HAS_MATCH_TARGET_48_FONT
3630     {
3631         tft.setFont(&FreeSansBold48pt7b);
3632         tft.setTextSize(1);
3633         tft.setTextColor(color, background);
3634         int16_t x1;
3635         int16_t y1;
3636         uint16_t w;
3637         uint16_t h;

```

```

3638     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3639     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3640     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3641     tft.setCursor(x, y);
3642     tft.print(text);
3643     tft.setFont();
3644 }
3645 return;
3646 #elif BGMC_HAS_SCORE_FONT
3647     (void)background;
3648     drawGfxTextScaledInRect(&FreeSansBold24pt7b, text, region, color, 3, 2);
3649     return;
3650 #elif BGMC_HAS_SMALL_UI_FONT
3651     tft.setFont(&FreeSans12pt7b);
3652     tft.setTextSize(3);
3653 #elif BGMC_HAS_UI_FONT
3654     tft.setFont(&FreeSans18pt7b);
3655     tft.setTextSize(2);
3656 #else
3657     tft.setFont();
3658     tft.setTextSize(7);
3659 #endif
3660     tft.setTextColor(color, background);
3661     int16_t x1;
3662     int16_t y1;
3663     uint16_t w;
3664     uint16_t h;
3665     tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3666 #if BGMC_HAS_SMALL_UI_FONT
3667     if (w > region.w - 24 || h > region.h - 8) {
3668         tft.setTextSize(2);
3669         tft.getTextBounds(text, 0, 0, &x1, &y1, &w, &h);
3670     }
3671 #endif
3672     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3673     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3674     tft.setCursor(x, y);
3675     tft.print(text);
3676     tft.setFont();
3677 }
3678
3679 void drawGfxTextScaledInRect(const GFXfont *font, const String &text, const Rect &region, uint16_t color,
3680                             uint8_t scaleNum, uint8_t scaleDen) {
3681     int16_t x1;
3682     int16_t y1;
3683     uint16_t w;
3684     uint16_t h;
3685     getGfxTextBoundsScaled(font, text, scaleNum, scaleDen, &x1, &y1, &w, &h);
3686
3687     if (w > region.w - 24 || h > region.h - 8) {
3688         scaleNum = 1;
3689         scaleDen = 1;
3690         getGfxTextBoundsScaled(font, text, scaleNum, scaleDen, &x1, &y1, &w, &h);
3691     }
3692
3693     int16_t x = region.x + ((region.w - (int16_t)w) / 2) - x1;
3694     int16_t y = region.y + ((region.h - (int16_t)h) / 2) - y1;
3695     drawGfxTextScaled(font, text, x, y, color, scaleNum, scaleDen);
3696 }
3697
3698 void getGfxTextBoundsScaled(const GFXfont *font, const String &text, uint8_t scaleNum, uint8_t scaleDen,
3699                             int16_t *x1, int16_t *y1, uint16_t *w, uint16_t *h) {
3700     int16_t minX = 32767;
3701     int16_t minY = 32767;
3702     int16_t maxX = -32768;
3703     int16_t maxY = -32768;
3704     int16_t cursor = 0;
3705     uint16_t first = pgm_read_word(&font->first);
3706     uint16_t last = pgm_read_word(&font->last);
3707     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&font->glyph);
3708
3709     for (uint16_t i = 0; i < text.length(); i++) {
3710         char c = text.charAt(i);
3711         if (c < first || c > last) {
3712             continue;
3713         }
3714         GFXglyph glyph;
3715         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
3716         if (glyph.width > 0 && glyph.height > 0) {
3717             int16_t gx1 = cursor + ((int32_t)glyph.xOffset * scaleNum) / scaleDen;
3718             int16_t gy1 = ((int32_t)glyph.yOffset * scaleNum) / scaleDen;
3719             int16_t gx2 = cursor + ((int32_t)(glyph.xOffset + glyph.width) * scaleNum + scaleDen - 1) / scaleDen - 1;
3720             int16_t gy2 = ((int32_t)(glyph.yOffset + glyph.height) * scaleNum + scaleDen - 1) / scaleDen - 1;
3721             if (gx1 < minX) minX = gx1;
3722             if (gy1 < minY) minY = gy1;
3723             if (gx2 > maxX) maxX = gx2;

```

```

3724     if (gy2 > maxY) maxY = gy2;
3725 }
3726 cursor += ((int32_t)glyph.xAdvance * scaleNum + scaleDen - 1) / scaleDen;
3727 }
3728
3729 if (maxX < minX || maxY < minY) {
3730     *x1 = 0;
3731     *y1 = 0;
3732     *w = 0;
3733     *h = 0;
3734     return;
3735 }
3736 *x1 = minX;
3737 *y1 = minY;
3738 *w = maxX - minX + 1;
3739 *h = maxY - minY + 1;
3740 }
3741
3742 void drawGfxTextScaled(const GFXfont *font, const String &text, int16_t x, int16_t y, uint16_t color,
3743                      uint8_t scaleNum, uint8_t scaleDen) {
3744     int16_t cursor = x;
3745     uint16_t first = pgm_read_word(&font->first);
3746     uint16_t last = pgm_read_word(&font->last);
3747     const GFXglyph *glyphs = (const GFXglyph *)pgm_read_ptr(&font->glyph);
3748     const uint8_t *bitmap = (const uint8_t *)pgm_read_ptr(&font->bitmap);
3749
3750     for (uint16_t i = 0; i < text.length(); i++) {
3751         char c = text.charAt(i);
3752         if (c < first || c > last) {
3753             continue;
3754         }
3755
3756         GFXglyph glyph;
3757         memcpy_P(&glyph, &glyphs[c - first], sizeof(GFXglyph));
3758         uint16_t bitIndex = 0;
3759         uint16_t bitmapOffset = glyph.bitmapOffset;
3760         uint8_t bits = 0;
3761
3762         for (uint8_t yy = 0; yy < glyph.height; yy++) {
3763             int16_t rowTop = y + ((int32_t)(glyph.yOffset + yy) * scaleNum) / scaleDen;
3764             int16_t rowBottom = y + ((int32_t)(glyph.yOffset + yy + 1) * scaleNum + scaleDen - 1) / scaleDen;
3765             int16_t rowH = max((int16_t)1, (int16_t)(rowBottom - rowTop));
3766
3767             for (uint8_t xx = 0; xx < glyph.width; xx++) {
3768                 if ((bitIndex & 7) == 0) {
3769                     bits = pgm_read_byte(&bitmap[bitmapOffset++]);
3770                 }
3771                 if (bits & 0x80) {
3772                     int16_t colLeft = cursor + ((int32_t)(glyph.xOffset + xx) * scaleNum) / scaleDen;
3773                     int16_t colRight = cursor + ((int32_t)(glyph.xOffset + xx + 1) * scaleNum + scaleDen - 1) / scaleDen;
3774                     int16_t colW = max((int16_t)1, (int16_t)(colRight - colLeft));
3775                     tft.fillRect(colLeft, rowTop, colW, rowH, color);
3776                 }
3777                 bits <<= 1;
3778                 bitIndex++;
3779             }
3780         }
3781         cursor += ((int32_t)glyph.xAdvance * scaleNum + scaleDen - 1) / scaleDen;
3782     }
3783 }
3784
3785 String formatTime(uint32_t seconds) {
3786     uint32_t minutes = seconds / 60;
3787     uint32_t secs = seconds % 60;
3788     char buffer[12];
3789     snprintf(buffer, sizeof(buffer), "%02lu:%02lu", (unsigned long)minutes, (unsigned long)secs);
3790     return String(buffer);
3791 }
3792
3793 String formatTimeMs(uint32_t milliseconds) {
3794     uint32_t seconds = (milliseconds + 999UL) / 1000UL;
3795     return formatTime(seconds);
3796 }
3797
3798 String timingModeLabel() {
3799     switch (timingMode) {
3800     case BRONSTEIN_DELAY:
3801         return "Bronstein";
3802     case FISCHER_INCREMENT:
3803         return "Fischer";
3804     case SIMPLE_DELAY:
3805     default:
3806         return "Simple";
3807     }
3808 }
3809

```

```

3810 String playPauseLabel() {
3811     if (clock_running) {
3812         return "Pause Game";
3813     }
3814     if (game_number == 0 && !game_in_progress) {
3815         return "Start Match";
3816     }
3817     return "Play Game";
3818 }
3819
3820 String matchStateLabel() {
3821     if (crawford_active) {
3822         return "CRAWFORD GAME - CUBE OFF";
3823     }
3824     if (post_crawford) {
3825         return "POST-CRAWFORD - CUBE ON";
3826     }
3827     return "NORMAL";
3828 }
3829
3830 int batteryPercent() {
3831     if (max17048Found) {
3832         return (int)(max17048StateOfCharge() + 0.5f);
3833     }
3834
3835     if (PIN_BATTERY_ADC == 255) {
3836         return -1;
3837     }
3838
3839     int raw = analogRead(PIN_BATTERY_ADC);
3840     float adcVolts = ((float)raw / 4095.0f) * BATTERY_ADC_REF_VOLTS;
3841     float batteryVolts = adcVolts * BATTERY_DIVIDER_RATIO;
3842     float pct = (batteryVolts - BATTERY_EMPTY_VOLTS) * 100.0f /
3843         (BATTERY_FULL_VOLTS - BATTERY_EMPTY_VOLTS);
3844     if (pct < 0.0f) {
3845         pct = 0.0f;
3846     }
3847     if (pct > 100.0f) {
3848         pct = 100.0f;
3849     }
3850     return (int)(pct + 0.5f);
3851 }
3852
3853 bool chargingIndicatorOn() {
3854     if (PIN_CHARGE_DETECT == 255) {
3855         if (!max17048Found) {
3856             return false;
3857         }
3858
3859         uint32_t nowMs = millis();
3860         if (lastChargeCheckMs == 0 || nowMs - lastChargeCheckMs >= 30000UL) {
3861             float voltage = max17048CellVoltage();
3862             float percent = max17048StateOfCharge();
3863             if (lastChargeCheckMs != 0) {
3864                 chargeTrendRising = (voltage > lastChargeCheckVoltage + 0.006f) ||
3865                     (percent > lastChargeCheckPercent + 0.08f);
3866                 if (chargeTrendRising) {
3867                     chargeIndicatorUntilMs = nowMs + 120000UL;
3868                 }
3869             }
3870             lastChargeCheckVoltage = voltage;
3871             lastChargeCheckPercent = percent;
3872             lastChargeCheckMs = nowMs;
3873         }
3874
3875         return chargeTrendRising || nowMs < chargeIndicatorUntilMs;
3876     }
3877     return digitalRead(PIN_CHARGE_DETECT) == LOW;
3878 }
3879
3880 void recordMatchLog() {
3881     String entry = matchLogDate();
3882     entry += "|";
3883     entry += playerA_name;
3884     entry += "|";
3885     entry += playerB_name;
3886     entry += "|";
3887     entry += String(playerA_score);
3888     entry += "-";
3889     entry += String(playerB_score);
3890
3891     for (int8_t i = MATCH_LOG_MAX_ENTRIES - 1; i > 0; i--) {
3892         matchLog[i] = matchLog[i - 1];
3893     }
3894     matchLog[0] = entry;
3895     if (matchLogCount < MATCH_LOG_MAX_ENTRIES) {

```

```

3896     matchLogCount++;
3897 }
3898
3899 prefs.putUChar("logCount", matchLogCount);
3900 for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
3901     char key[8];
3902     snprintf(key, sizeof(key), "log%u", i);
3903     prefs.putString(key, matchLog[i]);
3904 }
3905 }
3906
3907 void clearMatchLog() {
3908     matchLogCount = 0;
3909     prefs.putUChar("logCount", matchLogCount);
3910     for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
3911         matchLog[i] = "";
3912         char key[8];
3913         snprintf(key, sizeof(key), "log%u", i);
3914         prefs.putString(key, "");
3915     }
3916 }
3917
3918 String matchLogDate() {
3919     uint8_t second = 0;
3920     uint8_t minute = 0;
3921     uint8_t hour = 0;
3922     uint8_t day = 0;
3923     uint8_t month = 0;
3924     uint16_t year = 0;
3925     if (!ds3231ReadTime(&second, &minute, &hour, &day, &month, &year) || !rtcDateLooksSet(day, month, year)) {
3926         return "DATE UNSET";
3927     }
3928
3929     char buffer[12];
3930     snprintf(buffer, sizeof(buffer), "%04u-%02u-%02u", year, month, day);
3931     return String(buffer);
3932 }
3933
3934 String matchLogPart(const String &entry, uint8_t part) {
3935     int start = 0;
3936     for (uint8_t i = 0; i < part; i++) {
3937         int separator = entry.indexOf('|', start);
3938         if (separator < 0) {
3939             return "";
3940         }
3941         start = separator + 1;
3942     }
3943
3944     int end = entry.indexOf('|', start);
3945     if (end < 0) {
3946         return entry.substring(start);
3947     }
3948     return entry.substring(start, end);
3949 }
3950
3951 // -----
3952 // Settings and power management
3953 // -----
3954 void loadSettings() {
3955     prefs.begin("bgclock", false);
3956     match_target = prefs.getInt("target", 9);
3957     initial_time_seconds = prefs.getULong("time", DEFAULT_INITIAL_TIME_SECONDS);
3958     delay_seconds = prefs.getUChar("delay", DEFAULT_DELAY_SECONDS);
3959     if (delay_seconds == 7) {
3960         delay_seconds = DEFAULT_DELAY_SECONDS;
3961         prefs.putUChar("delay", delay_seconds);
3962     }
3963     increment_seconds = prefs.getUChar("inc", DEFAULT_INCREMENT_SECONDS);
3964     brightness = prefs.getUChar("bright", DEFAULT_BRIGHTNESS);
3965     timingMode = (TimingMode)prefs.getUChar("timing", (uint8_t)SIMPLE_DELAY);
3966     if (timingMode > FISCHER_INCREMENT) {
3967         timingMode = SIMPLE_DELAY;
3968     }
3969     timerScope = (TimerScope)prefs.getUChar("scope", (uint8_t)TIMER_BY_MATCH);
3970     if (timerScope > TIMER_BY_MATCH) {
3971         timerScope = TIMER_BY_MATCH;
3972     }
3973     playerA_name = prefs.getString("nameA", "PLAYER A");
3974     playerB_name = prefs.getString("nameB", "PLAYER B");
3975     playerA_color = prefs.getUShort("colorA", PLAYER_COLOR_BLUE);
3976     playerB_color = prefs.getUShort("colorB", PLAYER_COLOR_RED);
3977     if (playerA_name.length() == 0 || playerA_name.length() > PLAYER_NAME_MAX_LEN) {
3978         playerA_name = "PLAYER A";
3979     }
3980     if (playerB_name.length() == 0 || playerB_name.length() > PLAYER_NAME_MAX_LEN) {
3981         playerB_name = "PLAYER B";

```

```

3982 }
3983
3984 matchLogCount = prefs.getUChar("logCount", 0);
3985 if (matchLogCount > MATCH_LOG_MAX_ENTRIES) {
3986     matchLogCount = MATCH_LOG_MAX_ENTRIES;
3987 }
3988 for (uint8_t i = 0; i < MATCH_LOG_MAX_ENTRIES; i++) {
3989     char key[8];
3990     snprintf(key, sizeof(key), "log%u", i);
3991     matchLog[i] = prefs.getString(key, "");
3992 }
3993
3994 bool validTarget = false;
3995 for (uint8_t i = 0; i < MATCH_TARGET_COUNT; i++) {
3996     if (MATCH_TARGETS[i] == match_target) {
3997         validTarget = true;
3998     }
3999 }
4000 if (!validTarget) {
4001     match_target = 9;
4002 }
4003 }
4004
4005 void saveSettingsIfNeeded() {
4006     if (!settingsDirty) {
4007         return;
4008     }
4009
4010     uint32_t nowMs = millis();
4011     if (nowMs - settingsDirtyMs < SETTINGS_SAVE_DELAY_MS) {
4012         return;
4013     }
4014
4015     prefs.putInt("target", match_target);
4016     prefs.putULong("time", initial_time_seconds);
4017     prefs.putUChar("delay", delay_seconds);
4018     prefs.putUChar("inc", increment_seconds);
4019     prefs.putUChar("bright", brightness);
4020     prefs.putUChar("timing", (uint8_t)timingMode);
4021     prefs.putUChar("scope", (uint8_t)timerScope);
4022     settingsDirty = false;
4023 }
4024
4025 void savePlayerNames() {
4026     prefs.putString("nameA", playerA_name);
4027     prefs.putString("nameB", playerB_name);
4028     prefs.putUShort("colorA", playerA_color);
4029     prefs.putUShort("colorB", playerB_color);
4030 }
4031
4032 void markSettingsDirty() {
4033     settingsDirty = true;
4034     settingsDirtyMs = millis();
4035     markDisplayDirty();
4036 }
4037
4038 void markDisplayDirty() {
4039     displayDirty = true;
4040 }
4041
4042 void setBrightness(uint8_t duty) {
4043     if (duty == 0) {
4044         digitalWrite(PIN_TFT_BACKLIGHT, TFT_BACKLIGHT_ACTIVE_HIGH ? LOW : HIGH);
4045         return;
4046     }
4047
4048     digitalWrite(PIN_TFT_BACKLIGHT, TFT_BACKLIGHT_ACTIVE_HIGH ? HIGH : LOW);
4049 }
4050
4051 void wakeActivity() {
4052     lastActivityMs = millis();
4053 }
4054
4055 void maybeSleep() {
4056     if (clock_running) {
4057         return;
4058     }
4059
4060     if (PIN_PLAYER_A_BUTTON == 255 || PIN_PLAYER_B_BUTTON == 255 || PIN_MENU_SELECT == 255) {
4061         return;
4062     }
4063
4064     uint32_t nowMs = millis();
4065     if (nowMs - lastActivityMs < SLEEP_AFTER_IDLE_MS) {
4066         return;
4067     }

```

```
4068 tft.fillScreen(ILI9341_BLACK);
4069 setBrightness(0);
4070 delay(20); // Tiny hardware settle before deep sleep.
4071
4072 // Wake on the two large player buttons and menu button. Add more pins if
4073 // your board supports the required EXT1 wake bitmask for those GPIOs.
4074 uint64_t wakeMask = (1ULL << PIN_PLAYER_A_BUTTON) |
4075                     (1ULL << PIN_PLAYER_B_BUTTON) |
4076                     (1ULL << PIN_MENU_SELECT);
4077 esp_sleep_enable_ext1_wakeup(wakeMask, ESP_EXT1_WAKEUP_ANY_LOW);
4078 esp_deep_sleep_start();
4079 }
4080
4081 // -----
4082 // Buzzer feedback
4083 // -----
4084
4085 void beep(uint16_t frequency, uint16_t durationMs) {
4086     if (PIN_BUZZER == 255) {
4087         return;
4088     }
4089
4090     tone(PIN_BUZZER, frequency, durationMs);
4091 }
4092
4093 void playButtonBeep() {
4094     beep(1800, 20);
4095 }
4096
4097 void playTimeoutBeep() {
4098     beep(900, 300);
4099 }
4100
4101 void playMatchOverBeep() {
4102     beep(1600, 120);
4103     // tone() is non-blocking on most Arduino cores, so keep this simple.
4104 }
```